ĐÀM MẠNH DŨNG

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỎ - ĐỊA CHẤT



ĐÀM MẠNH DŨNG

ĐỒ ÁN TỐT NGHIỆP

NGÀNH CÔNG NGHỆ THÔNG TIN

TÊN ĐỀ TÀI:

XÂY DỰNG NỀN TẢNG AI HỖ TRỢ NGHIÊN CỨU VÀ VIẾT BÀI BÁO KHOA HỌC VINUNI DỰA TRÊN AGENTIC RAG, TRUY XUẤT TRI THỨC HỌC THUẬT VÀ KIỂM CHỨNG TRÍCH DẪN

HỆ THỐNG THÔNG TIN VÀ TRI THỨC

Hà Nội - Năm 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỎ - ĐỊA CHẤT

ĐỒ ÁN TỐT NGHIỆP

CHUYÊN NGÀNH: HỆ THỐNG THÔNG TIN VÀ TRI THỨC

GIÁO VIÊN HƯỚNG DẪN	SINH VIÊN THỰC HIỆN
(Chữ ký)	*(Chữ ký)*
ThS. Vũ Lan Phương	**Đàm Mạnh Dũng**

Hà Nội – Năm 2026

PHIẾU GIAO ĐỀ TÀI TỐT NGHIỆP

1. Thông tin chung

Tên đề tài:	Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn
Họ và tên sinh viên:	Đàm Mạnh Dũng
Mã sinh viên:	2221050359
Điện thoại liên lạc:	0978461865
Email:	dammanhdungvn@gmail.com
Lớp:	DCCTCT67_09B
Hệ đào tạo:	Đại học chính quy
Đồ án tốt nghiệp thực hiện tại:	Trường Đại học Mỏ – Địa chất
Thời gian làm ĐATN:	Năm 2026

2. Mục tiêu của ĐATN

Mục tiêu tổng quát:

- Xây dựng thành công nền tảng Trợ lý AI nghiên cứu khoa học VeriScholar cho học giả VinUni.
- Triệt tiêu ảo giác qua truy xuất tri thức lai (Hybrid RAG) và kiểm chứng trích dẫn tọa độ Bounding Box.
- Cung cấp công cụ soạn thảo, biên dịch LaTeX an toàn trong sandbox E2B và bảo mật khóa BYOK.

Mục tiêu cụ thể:

– **Về sản phẩm:**
  – Phân hệ Khám phá: Tìm kiếm học thuật đa nguồn (Exa AI, arXiv, Semantic Scholar, CrossRef).
  – Phân hệ Đọc sâu: Đọc PDF Canvas hai cột kết hợp kiểm chứng tọa độ Bounding Box trực quan.
  – Phân hệ GraphRAG: Đồ thị tri thức 4 thực thể (Paper, Method, Dataset, MetricResult) và Graph Wiki.
  – Phân hệ Soạn thảo LaTeX: Soạn thảo trực tuyến và tác nhân ReAct tự sửa lỗi trong sandbox E2B.
  – Phân hệ Bảo mật: Cơ chế quản lý khóa API cá nhân BYOK mã hóa AES-256-GCM phía máy chủ.

– **Về kiến thức và kỹ năng:**
  – Vận dụng kiến thức: Kỹ nghệ phần mềm, CSDL nâng cao, Kiến trúc hệ thống và Xử lý ngôn ngữ tự nhiên.
  – Nắm vững quy trình phát triển phần mềm theo phương pháp TDD với 4 Cổng xác minh chất lượng.
  – Làm chủ kỹ thuật Hybrid RAG, đồ thị tri thức GraphRAG và tác nhân AI ReAct tự chủ.
  – Rèn luyện năng lực nghiên cứu độc lập, phân tích tài liệu quốc tế và tư duy liêm chính học thuật.

3. Các nhiệm vụ cụ thể của ĐATN

Để hoàn thành các mục tiêu đề ra, sinh viên thực hiện các nhiệm vụ cụ thể sau:

– Khảo sát quy trình nghiên cứu, viết bài báo tại VinUni; xác định bài toán liêm chính học thuật.
– Đặc tả yêu cầu chức năng và phi chức năng hệ thống (BYOK, trễ RAG $\leq 1{,}5\,\mathrm{s}$, IoU $\geq 0{,}85$).
– Thiết kế kiến trúc Lục giác, mô hình CSDL quan hệ PostgreSQL kết hợp véc-tơ pgvector.
– Cài đặt hệ thống Backend (FastAPI, pgvector, LangGraph) và Frontend (TypeScript, React 19).
– Tích hợp sandbox E2B MicroVM biên dịch LaTeX cô lập an toàn, ngăn chặn mã độc hại.
– Triển khai toàn bộ quy trình kiểm thử TDD với hơn 330 kịch bản kiểm thử tự động.
– Đánh giá thực nghiệm hệ thống trên các bài báo thực tế, đo độ trễ và độ chính xác tọa độ.
– Viết báo cáo tổng kết đồ án tốt nghiệp, chuẩn bị slide thuyết trình và đóng gói sản phẩm bảo vệ Hội đồng.

4. Lời cam đoan của sinh viên:

Em – Đàm Mạnh Dũng – cam kết ĐATN là công trình nghiên cứu của bản thân em, làm dưới sự hướng dẫn của cô Vũ Lan Phương. Các kết quả nêu trên đồ án hoàn toàn là trung thực, không phải là sao chép toàn văn của bất kì công trình nào khác.

Hà Nội, ngày……tháng……năm 2026

Xác nhận của GVHD	**Tác giả ĐATN**
(Chữ ký)	*(Chữ ký)*
ThS. Vũ Lan Phương	**Đàm Mạnh Dũng**

KHOA CÔNG NGHỆ THÔNG TIN
BỘ MÔN HỆ THỐNG THÔNG TIN VÀ TRI THỨC

CỘNG HÒA XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh phúc

ĐỀ CƯƠNG ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC

Sinh viên thực hiện:

TT	Mã sinh viên	Họ và tên	Lớp	Điện thoại	E-mail
1	2221050359	Đàm Mạnh Dũng	DCCTCT67_09B	0978461865	dammanhdungvn@gmail.com

Giảng viên hướng dẫn:

TT	Mã cán bộ	Họ và tên	Đơn vị	Điện thoại	E-mail
1	0802-09	ThS. Vũ Lan Phương	Khoa CNTT	0984223379	vulanphuong@humg.edu.vn

1. Tên đề tài

Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn.

2. Lý do chọn đề tài

Trong bối cảnh bùng nổ nghiên cứu khoa học, giảng viên và học giả tại Trường Đại học VinUni đối mặt với khối lượng lớn tài liệu quốc tế đa nguồn cần đọc hiểu, tổng hợp và thẩm định. Việc tiếp cận nhanh chóng các phương pháp tiên tiến đồng thời duy trì năng suất công bố khoa học chuẩn IEEE/ACM là nhu cầu thường trực của các nhóm nghiên cứu.

Tuy nhiên, các mô hình ngôn ngữ lớn hiện nay thường mắc lỗi ảo giác tri thức (*Hallucination*), tự bịa đặt trích dẫn hoặc dẫn chiếu sai lệch số liệu. Bên cạnh đó, các công cụ phổ biến như ChatPDF làm mất cấu trúc hai cột và không có tọa độ minh chứng, trong khi Overleaf chưa tích hợp AI tự sửa lỗi biên dịch LaTeX. Vì vậy, việc nghiên cứu và xây dựng nền tảng VeriScholar là hết sức cấp thiết, cung cấp giải pháp toàn diện hỗ trợ học giả VinUni từ khâu khám phá tài liệu, đọc sâu kiểm chứng tọa độ, đến biên dịch bản thảo khoa học chuẩn mực.

3. Mục tiêu của đề tài

– Mục tiêu tổng quát:

- Xây dựng Trợ lý AI nghiên cứu khoa học VeriScholar cho học giả VinUni.
- Triệt tiêu ảo giác qua kiểm chứng trích dẫn tọa độ Bounding Box hai tầng.
- Tự động hóa biên dịch LaTeX và sửa lỗi cú pháp trong sandbox E2B.

– Về sản phẩm:

- Hệ thống Backend: REST API và 5 luồng SSE theo Kiến trúc Lục giác.
- Phân hệ Khám phá tài liệu: Tìm kiếm đa nguồn (Exa AI, arXiv, Semantic Scholar, CrossRef) kết hợp truy xuất lai pgvector Dense và BM25 Sparse.
- Phân hệ Đọc sâu PDF: Phân tích 2 cột, hỏi đáp và đối chiếu tọa độ Bounding Box.
- Phân hệ GraphRAG: Trích xuất 4 thực thể (Paper, Method, Dataset, MetricResult) và phân cụm Louvain.
- Phân hệ Soạn thảo & Bảo mật: Tích hợp sandbox E2B, tác nhân LangGraph ReAct tự sửa cú pháp LaTeX và mã hóa khóa BYOK bằng AES-256-GCM.
- Giao diện Web: Phát triển trên React 19, TypeScript, Tailwind và Monaco.

– Về kiến thức và kỹ năng:

- Vận dụng kiến thức: Kỹ nghệ phần mềm, CSDL nâng cao, Học máy và NLP.
- Làm chủ quy trình TDD, bảo đảm 100% các cổng xác minh chất lượng.
- Thành thạo công nghệ: FastAPI, SQLAlchemy, PostgreSQL, React 19, E2B.
- Nâng cao năng lực nghiên cứu độc lập và tư duy liêm chính học thuật.

4. Các nhiệm vụ cụ thể

Các nhiệm vụ cụ thể của đồ án bao gồm:

– Khảo sát bài toán nghiên cứu tại VinUni và lập đề cương chi tiết.
– Đặc tả yêu cầu chức năng, phi chức năng hệ thống (BYOK, IoU $\geq 0{,}85$).
– Thiết kế kiến trúc Lục giác, mô hình quan hệ và CSDL pgvector.
– Thiết kế giao diện (UI/UX) và các biểu đồ tuần tự nghiệp vụ cốt lõi.
– Xây dựng module Backend: Xác thực, phân quyền, thư viện và lưu trữ CAS.
– Xây dựng phân hệ Hybrid RAG (pgvector + BM25) và kiểm chứng Bbox.
– Xây dựng GraphRAG và tác nhân AI sửa lỗi LaTeX trong sandbox E2B.
– Xây dựng Frontend: Monaco Editor, PDF Canvas và streaming SSE.
– Kiểm thử tự động TDD, đánh giá thực nghiệm và đóng gói sản phẩm.

5. Kế hoạch thực hiện

Tuần	Thời gian	Nội dung công việc
1	24/08/2026 – 30/08/2026	Khảo sát bài toán nghiên cứu tại VinUni, thu thập tài liệu tham khảo liên quan, lập đề cương chi tiết và kế hoạch thực hiện đồ án.
2	31/08/2026 – 06/09/2026	Phân tích yêu cầu nghiệp vụ; xác định các tác nhân (Actor), biểu đồ Use Case tổng thể và đặc tả chi tiết các Use Case chính.
3	07/09/2026 – 13/09/2026	Thiết kế kiến trúc Lục giác (Hexagonal Architecture), thiết kế CSDL quan hệ PostgreSQL và cơ sở dữ liệu véc-tơ pgvector.
4	14/09/2026 – 20/09/2026	Thiết kế giao diện người dùng khoa học tối giản (UI/UX); thiết kế biểu đồ tuần tự cho các luồng nghiệp vụ RAG và kiểm chứng Bounding Box.
5	21/09/2026 – 27/09/2026	Xây dựng Backend cốt lõi: Cấu hình môi trường, triển khai tầng Core nghiệp vụ, xác thực JWT, lưu trữ CAS và mã hóa BYOK.
6	28/09/2026 – 04/10/2026	Hiện thực hóa phân hệ Hybrid RAG: Tìm kiếm kết hợp HNSW + BM25, dung hợp RRF, tái xếp hạng Cross-Encoder và trích xuất Bounding Box PDF.
7	05/10/2026 – 11/10/2026	Hiện thực hóa phân hệ Đồ thị tri thức (GraphRAG) và Tác nhân AI ReAct tự động phục hồi lỗi biên dịch LaTeX trong sandbox E2B.
8	12/10/2026 – 18/10/2026	Xây dựng giao diện Frontend (React 19): Monaco Editor, trình hiển thị PDF Canvas thời gian thực, bảng dẫn chứng Bbox và luồng SSE.
9	19/10/2026 – 25/10/2026	Tích hợp toàn diện hệ thống Frontend – Backend; thực hiện kiểm thử tự động TDD (Unit Test, Boundary Test, E2E); tối ưu hóa hiệu năng và bảo mật.
10	26/10/2026 – 02/11/2026	Đánh giá thực nghiệm trên các ấn phẩm khoa học thực tế; hoàn thiện báo cáo tốt nghiệp, slide thuyết trình, đóng gói sản phẩm và nộp báo cáo về Bộ môn Hệ thống thông tin và Tri thức ngày 02/11/2026.

6. Nơi thực hiện đồ án: Trường Đại học Mỏ – Địa chất

Hà Nội, ngày……tháng……năm 2026

Giảng viên hướng dẫn	**Sinh viên thực hiện**
(ký, họ tên)	*(ký, họ tên)*
ThS. Vũ Lan Phương	**Đàm Mạnh Dũng**

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI TỐT NGHIỆP iii
ĐỀ CƯƠNG ĐỒ ÁN TỐT NGHIỆP ĐẠI HỌC vi
MỤC LỤC ix
DANH MỤC HÌNH ẢNH xii
DANH MỤC BẢNG BIỂU xiv
DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT xvi
LỜI CẢM ƠN 1
LỜI MỞ ĐẦU 2
CHƯƠNG 1 TỔNG QUAN VỀ ĐỀ TÀI 4
1.1 Giới thiệu đề tài 4
1.2 Tính cấp thiết của đề tài 4
1.3 Mục tiêu nghiên cứu 7
1.3.1 Mục tiêu tổng quát 7
1.3.2 Mục tiêu kỹ thuật cụ thể 7
1.4 Nội dung nghiên cứu và phương pháp tiếp cận 8
1.4.1 Nội dung nghiên cứu 8
1.4.2 Phương pháp tiếp cận 8
1.5 Phạm vi và đối tượng nghiên cứu 8
1.6 Bố cục của báo cáo đồ án tốt nghiệp 9
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ 11
2.1 Tổng quan về PostgreSQL và Redis 11
2.1.1 PostgreSQL là gì? 11
2.1.2 Extension pgvector và chỉ mục HNSW 12
2.1.3 Redis là gì? 13
2.2 Tổng quan về Hexagonal Architecture 14
2.2.1 Hexagonal Architecture là gì? 14
2.2.2 Kiến trúc Ports and Adapters 14
2.3 Tổng quan về Advanced RAG 16
2.3.1 RAG là gì? (Retrieval-Augmented Generation) 16
2.3.2 Advanced RAG là gì? 17
2.3.3 Hybrid Search và thuật toán RRF 18

2.4 Tổng quan về GraphRAG, Knowledge Graph và Graph Wiki 21
2.4.1 GraphRAG là gì? 21
2.4.2 Knowledge Graph là gì? (Đồ thị tri thức) 22
2.4.3 Graph Wiki là gì? (Pre-compiled Community Wiki) 23
2.5 Tìm hiểu về Agentic AI và ReAct 24
2.5.1 Khái niệm và kiến trúc tổng quan của Agentic AI 24
2.5.2 Mẫu kiến trúc Lập kế hoạch (Planning Pattern) 25
2.5.3 Mẫu kiến trúc Sử dụng Công cụ (Tool Use Pattern) 26
2.5.4 Mẫu kiến trúc Suy luận và Hành động ReAct (ReAct Pattern) ... 27
2.5.5 Cơ chế Reflection và Tự sửa sai Reflexion (Reflection Pattern).. 28
2.5.6 Mẫu kiến trúc Phối hợp Đa Agent (Multi-Agent Pattern)..... 29
2.5.7 Mẫu tương tác Người trong vòng lặp (Human-in-the-Loop Pattern) 30
2.6 Tổng quan về LangGraph 31
2.6.1 LangGraph là gì? 31
2.6.2 Kiến trúc StateGraph điều phối Agent 32
2.7 Tìm hiểu về Azure OpenAI và E2B Sandbox 33
2.7.1 Dịch vụ suy luận OpenAI API và Hạ tầng đám mây Microsoft Azure 33
2.7.2 E2B Sandbox là gì? 34
2.7.3 Mô hình bảo mật BYOK (Bring Your Own Key) 34
2.8 Tổng quan về Langfuse 35
2.8.1 Langfuse là gì? (LLM Tracing & Observability)..... 35
2.8.2 Cơ chế theo dõi chi phí và độ trễ LLM 36
2.9 Các công cụ và ngôn ngữ phát triển hệ thống 37
2.9.1 Python và FastAPI 37
2.9.2 TypeScript và React..... 38
2.9.3 Docker, Pytest và Vitest 39
CHƯƠNG 3 PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG..... 40
3.1 Mô tả bài toán 40
3.2 Khảo sát nghiệp vụ và quy trình xử lý tài liệu nghiên cứu 41
3.2.1 Khảo sát quy trình tìm kiếm và thu thập tài liệu nghiên cứu 41
3.2.2 Khảo sát quy trình đọc sâu và thẩm định nguồn trích dẫn 42
3.2.3 Khảo sát quy trình quản lý và liên kết đồ thị tri thức học thuật ... 42
3.2.4 Khảo sát quy trình biên soạn bản thảo và sửa lỗi biên dịch LaTeX 43

3.2.5 Khảo sát quy trình quản lý hồ sơ học giả và bảo mật khóa cá nhân 43
3.2.6 Thiết kế chi tiết các thuật toán xử lý cốt lõi của hệ thống..... 44
3.3 Giới thiệu các Actor và các Use Case tương ứng trong hệ thống..... 46
3.4 Phân tích các Use Case của hệ thống 47
3.4.1 Phân tích các Use Case phía Học giả..... 48
3.4.2 Phân tích các Use Case phía Quản trị viên 54
3.5 Phân tích một số biểu đồ tuần tự chính của hệ thống..... 56
3.5.1 Biểu đồ tuần tự Use Case "Tra cứu và tải tài liệu nghiên cứu"..... 57
3.5.2 Biểu đồ tuần tự Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ" 57
3.5.3 Biểu đồ tuần tự Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX" 58
3.6 Thiết kế cơ sở dữ liệu cho hệ thống..... 58
3.6.1 Thiết kế cơ sở dữ liệu của hệ thống 58
3.6.2 Đặc tả các bảng cơ sở dữ liệu của hệ thống 59
3.6.3 Lược đồ thực thể quan hệ (ERD) của hệ thống..... 76
3.6.4 Thiết kế hạ tầng lưu trữ vector và đồ thị tri thức phụ trợ..... 77
CHƯƠNG 4 CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM..... 78
4.1 Môi trường thực nghiệm..... 78
4.1.1 Cài đặt Visual Studio Code và môi trường phát triển..... 78
4.1.2 Cài đặt môi trường hạ tầng và cơ sở dữ liệu 79
4.1.3 Triển khai thực nghiệm hệ thống 80
4.2 Kết quả thực nghiệm 81
4.2.1 Một số giao diện của hệ thống 83
4.2.2 Xây dựng kịch bản kiểm thử tự động hệ thống qua 4 Cổng xác minh..... 90
4.2.3 Đánh giá số liệu thực nghiệm định lượng và đối chiếu Langfuse Cloud..... 92
4.2.4 Hệ thống giám sát vận hành và sử dụng tài nguyên (Monitoring) 95
Nhận xét..... 99
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN..... 101
TÀI LIỆU THAM KHẢO..... 104

DANH MỤC HÌNH ẢNH

Hình 2-1 PostgreSQL..... 11
Hình 2-2 Redis 13
Hình 2-3 Mô hình phân rã module và cấu trúc Cổng – Điều hợp (Ports and Adapters) 15
Hình 2-4 Mô hình kiến trúc tạo sinh tăng cường truy xuất truyền thống (Naive RAG) 17
Hình 2-5 Mô hình kiến trúc tạo sinh tăng cường truy xuất nâng cao (Advanced RAG) 18
Hình 2-6 Cơ chế tìm kiếm lai (Hybrid Search) kết hợp thuật toán Hợp nhất Xếp hạng Đảo ngược (RRF) 20
Hình 2-7 Mô hình kiến trúc tạo sinh tăng cường truy xuất dựa trên đồ thị tri thức (GraphRAG) 21
Hình 2-8 Cấu trúc Đồ thị tri thức học thuật (Academic Knowledge Graph) và cơ chế biên dịch trang bách khoa (Graph Wiki)..... 23
Hình 2-9 Kiến trúc AI Agent Độc lập (LLM-Powered Autonomous Agent) . 24
Hình 2-10 Mẫu kiến trúc Lập kế hoạch và Phân rã nhiệm vụ (Planning Pattern) 25
Hình 2-11 Mẫu kiến trúc Agent sử dụng công cụ ngoại vi (Tool Use Pattern) . 26
Hình 2-12 Mẫu kiến trúc suy luận và hành động ReAct (Reasoning and Acting Pattern)..... 27
Hình 2-13 Cơ chế Reflection và tự sửa sai Reflexion (Reflection Pattern) 28
Hình 2-14 Mẫu kiến trúc phối hợp đa Agent chuyên trách (Multi-Agent Collaboration Pattern)..... 29
Hình 2-15 Mẫu tương tác người trong vòng lặp (Human-in-the-Loop Pattern) 30
Hình 2-16 LangGraph..... 31
Hình 2-17 Microsoft Azure 33
Hình 2-18 E2B Sandbox..... 34
Hình 2-19 Langfuse 36
Hình 2-20 Python..... 37
Hình 2-21 FastAPI 37
Hình 2-22 React 38
Hình 2-23 Docker 39

Hình 3-1 Mô hình đối chiếu chuyển đổi quy trình nghiên cứu khoa học AS-IS và TO-BE 44
Hình 3-2 Biểu đồ Use Case tổng quan của hệ thống VeriScholar..... 48
Hình 3-3 Biểu đồ tuần tự Use Case Tra cứu và tải tài liệu nghiên cứu..... 57
Hình 3-4 Biểu đồ tuần tự Use Case Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ..... 57
Hình 3-5 Biểu đồ tuần tự Use Case Biên soạn bản thảo và tự phục hồi lỗi LaTeX 58
Hình 3-6 Mối quan hệ giữa các bảng trong hệ quản trị cơ sở dữ liệu quan hệ (ERD)..... 76

Hình 4-1 Mô hình kiến trúc không gian tương tác người–máy bốn phân hệ VeriScholar 82
Hình 4-2 Giao diện đăng ký tài khoản của hệ thống 83
Hình 4-3 Giao diện đăng nhập hệ thống 84
Hình 4-4 Giao diện bảng điều khiển trung tâm 85
Hình 4-5 Giao diện tìm kiếm và khám phá tài liệu học thuật..... 86
Hình 4-6 Giao diện tra cứu và đối chiếu danh mục trích dẫn..... 86
Hình 4-7 Giao diện quản lý thư viện nghiên cứu cá nhân..... 87
Hình 4-8 Giao diện đọc sâu tài liệu PDF hai cột và Bounding Box viền vàng 88
Hình 4-9 Giao diện mạng đồ thị tri thức học thuật 4 thực thể GraphRAG 89
Hình 4-10 Giao diện kho khóa API cá nhân (BYOK) và hồ sơ học giả 90
Hình 4-11 Giao diện soạn thảo bản thảo bài báo LaTeX và tự sửa lỗi biên dịch 91
Hình 4-12 Nhật ký kiểm chứng và giám sát vết thực thi mô hình trên Langfuse Cloud 95
Hình 4-13 Biểu đồ giám sát tài nguyên máy chủ ảo Azure (Độ sẵn sàng, CPU, RAM và tiến trình) 96
Hình 4-14 Biểu đồ giám sát lưu lượng truy vấn và độ tin cậy API tìm kiếm Exa AI..... 97
Hình 4-15 Báo cáo giám sát lượng token luân chuyển và chi phí API OpenAI (lũy kế chu kỳ tháng 10/2026)..... 98
Hình 4-16 Bảng điều khiển chi phí và phân bổ vết suy luận Langfuse Cost Dashboard (cửa sổ lọc 24 giờ kiểm định)..... 99

DANH MỤC BẢNG BIỂU

Bảng 1-1 Đối chiếu năng lực kỹ thuật giữa các công cụ hỗ trợ nghiên cứu học thuật hiện có và hệ thống VeriScholar 6

Bảng 2-1 So sánh đặc tính giữa tìm kiếm ngữ nghĩa Dense Vector và từ khóa BM25..... 19

Bảng 3-1 Các Use Case chính đối với mỗi Actor trong hệ thống VeriScholar 47
Bảng 3-2 Đặc tả Use Case "Tra cứu và tải tài liệu nghiên cứu"..... 50
Bảng 3-3 Đặc tả Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ" 52
Bảng 3-4 Đặc tả Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX" .. 54
Bảng 3-5 Đặc tả Use Case "Quản trị người dùng và chính sách bảo mật BYOK" 55
Bảng 3-6 Đặc tả Use Case "Giám sát hệ thống và Trace mô hình"..... 56
Bảng 3-7 Các bảng cơ sở dữ liệu của hệ thống VeriScholar 60
Bảng 3-8 Đặc tả bảng users..... 61
Bảng 3-9 Đặc tả bảng user_api_keys 62
Bảng 3-10 Đặc tả bảng scholar_profiles 63
Bảng 3-11 Đặc tả bảng folders 64
Bảng 3-12 Đặc tả bảng documents..... 65
Bảng 3-13 Đặc tả bảng document_storage_blobs..... 66
Bảng 3-14 Đặc tả bảng document_chunks..... 67
Bảng 3-15 Đặc tả bảng reference_items 68
Bảng 3-16 Đặc tả bảng research_sessions 69
Bảng 3-17 Đặc tả bảng session_turns 70
Bảng 3-18 Đặc tả bảng graph_nodes 71
Bảng 3-19 Đặc tả bảng graph_edges 72
Bảng 3-20 Đặc tả bảng folder_community_summaries..... 73
Bảng 3-21 Đặc tả bảng wiki_pages 74
Bảng 3-22 Đặc tả bảng drafts..... 75
Bảng 3-23 Đặc tả bảng draft_messages 76

Bảng 4-1 Cấu trúc phân vùng kho mã nguồn Monorepo và quy tắc ranh giới kiến trúc 80
Bảng 4-2 Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh 91

Bảng 4-3 Kết quả thực nghiệm đối chứng (Ablation Study) giữa các phương pháp truy xuất trên 36 truy vấn chuẩn…… 93
Bảng 4-4 Kết quả thực nghiệm đối chứng (Ablation Study) các cơ chế tự sửa lỗi biên dịch LaTeX trên 25 kịch bản lỗi…… 94

DANH MỤC CÁC KÝ HIỆU VÀ CHỮ VIẾT TẮT

STT	Ký hiệu / viết tắt	Ý nghĩa
1	ACID	Atomicity, Consistency, Isolation, Durability – Bốn thuộc tính toàn vẹn giao dịch cơ sở dữ liệu
2	AI	Artificial Intelligence – Trí tuệ nhân tạo
3	API	Application Programming Interface – Giao diện lập trình ứng dụng
4	ASGI	Asynchronous Server Gateway Interface – Giao diện cổng máy chủ bất đồng bộ Python
5	BBox	Bounding Box – Khung hộp chữ nhật định vị tọa độ
6	BM25	Best Matching 25 – Thuật toán xếp hạng tìm kiếm từ khóa toàn văn
7	BYOK	Bring Your Own Key – Cơ chế người dùng tự cung cấp khóa API cá nhân
8	CAS	Content-Addressable Storage – Hệ thống lưu trữ định danh theo nội dung băm
9	CI/CD	Continuous Integration / Continuous Deployment – Tích hợp và Triển khai liên tục
10	CTE	Common Table Expression – Bảng biểu thức chung trong truy vấn SQL đệ quy
11	DOI	Digital Object Identifier – Định danh đối tượng kỹ thuật số bài báo khoa học
12	ERD	Entity-Relationship Diagram – Sơ đồ quan hệ thực thể cơ sở dữ liệu
13	FTS	Full-Text Search – Tìm kiếm toàn văn cơ sở dữ liệu
14	GIN	Generalized Inverted Index – Chỉ mục đảo tổng quát trong PostgreSQL
15	GraphRAG	Graph Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất qua đồ thị
16	HNSW	Hierarchical Navigable Small World – Cấu trúc đồ thị chỉ mục tìm kiếm vector gần đúng
17	IoU	Intersection over Union – Chỉ số trùng khớp diện tích hộp tọa độ

STT	Ký hiệu / viết tắt	Ý nghĩa
18	JWT	JSON Web Token – Chuỗi mã định danh phiên người dùng an toàn
19	LLM	Large Language Model – Mô hình ngôn ngữ lớn
20	ORCID	Open Researcher and Contributor ID – Mã định danh học giả quốc tế 16 số
21	PDF	Portable Document Format – Định dạng tài liệu số di động
22	RAG	Retrieval-Augmented Generation – Sinh văn bản tăng cường truy xuất thông tin
23	ReAct	Reason + Act – Chu trình AI: Suy luận và Thực thi hành động
24	RRF	Reciprocal Rank Fusion – Thuật toán hợp nhất xếp hạng đảo ngược
25	SHA-256	Secure Hash Algorithm 256-bit – Thuật toán băm mật mã an toàn 256 bit
26	SSE	Server-Sent Events – Giao thức truyền sự kiện thời gian thực từ máy chủ
27	TDD	Test-Driven Development – Phương pháp phát triển hướng kiểm thử
28	TTFT	Time To First Token – Thời gian đáp ứng từ lúc gửi câu hỏi đến token đầu
29	UML	Unified Modeling Language – Ngôn ngữ mô hình hóa thống nhất

LỜI CẢM ƠN

Để hoàn thành bản đồ án tốt nghiệp đại học này, trước hết em xin bày tỏ lòng biết ơn sâu sắc và chân thành nhất tới cô Vũ Lan Phương – Giảng viên Khoa Công nghệ Thông tin, Trường Đại học Mỏ - Địa chất. Cô đã trực tiếp định hướng đề tài, tận tình chỉ bảo, đóng góp nhiều ý kiến học thuật và chuyên môn quý báu, cũng như luôn tạo mọi điều kiện thuận lợi nhất để em nghiên cứu và hiện thực hóa các công nghệ trí tuệ nhân tạo hiện đại vào nền tảng VeriScholar.

Em xin trân trọng gửi lời cảm ơn chân thành tới các Thầy, Cô giáo trong Bộ môn Hệ thống thông tin và Tri thức, Bộ môn Công nghệ Thông tin cùng toàn thể các Thầy, Cô trong Khoa Công nghệ Thông tin – Trường Đại học Mỏ - Địa chất. Những tri thức nền tảng vững chắc, phương pháp luận nghiên cứu khoa học cùng sự dạy dỗ tâm huyết của các Thầy, Cô trong suốt những năm tháng học tập và rèn luyện trên giảng đường đại học chính là hành trang quý giá giúp em hoàn thành công trình tốt nghiệp này.

Cuối cùng, em xin gửi lời tri ân sâu sắc tới bố mẹ và gia đình, đồng thời xin cảm ơn bạn bè đã luôn ở bên động viên, sẻ chia và là điểm tựa tinh thần vững chắc cho em trong suốt quá trình học tập và hoàn thiện đồ án.

Dù đã nỗ lực hết mình để nghiên cứu, xây dựng hệ thống và biên soạn báo cáo một cách khoa học, cẩn trọng nhất, song do giới hạn về thời gian và kinh nghiệm thực tiễn, đồ án chắc chắn khó tránh khỏi những thiếu sót nhất định. Em kính mong nhận được những ý kiến đóng góp, chỉ dẫn quý báu từ quý Thầy, Cô trong Hội đồng chấm đồ án tốt nghiệp để công trình được hoàn thiện hơn nữa.

LỜI MỞ ĐẦU

Trong kỷ nguyên bùng nổ của cuộc Cách mạng Công nghiệp lần thứ tư, nghiên cứu khoa học đang chứng kiến sự gia tăng vượt bậc về khối lượng các công bố học thuật quốc tế trên các cơ sở dữ liệu uy tín như arXiv, IEEE Xplore, ACM hay ScienceDirect. Thực tế này tạo áp lực lớn cho học giả, giảng viên và sinh viên trong việc sàng lọc, đọc hiểu chuyên sâu và tổng hợp tri thức chuyên ngành.

Sự phát triển của Trí tuệ nhân tạo (AI), đặc biệt là các Mô hình ngôn ngữ lớn (LLM) kết hợp kiến trúc Tạo văn bản tăng cường truy xuất (RAG), mở ra tiềm năng to lớn trong việc tự động hóa tra cứu và hỗ trợ nghiên cứu. Tuy nhiên, khi ứng dụng vào môi trường học thuật đòi hỏi độ tin cậy tuyệt đối, người dùng thường đối mặt với rào cản mang tên: hiện tượng ảo giác trích dẫn (*Citation Hallucination*). Các mô hình hiện nay thường tự tạo trích dẫn không có thật hoặc không thể chỉ rõ bằng chứng tọa độ nằm ở đoạn văn, bảng biểu hay trang nào trong bản tài liệu gốc PDF.

Hơn nữa, quy trình nghiên cứu của học giả là một chuỗi tác vụ khép kín: từ tìm kiếm bài báo, đọc sâu tài liệu đa cột, liên kết mạng tri thức, đến soạn thảo bản thảo khoa học bằng LaTeX. Các công cụ hiện có còn rời rạc: trình đọc tài liệu thiếu khả năng trích xuất hộp tọa độ; thư viện tài liệu thiếu liên kết đồ thị tri thức; còn trình soạn thảo văn bản học thuật lại đòi hỏi người viết tự xử lý các lỗi cú pháp biên dịch phức tạp.

Nhằm giải quyết bài toán ảo giác trích dẫn và hỗ trợ quy trình nghiên cứu khép kín, đề tài: *"Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn"* (sản phẩm: VeriScholar) được lựa chọn thực hiện. Đồ án tập trung nghiên cứu và hiện thực hóa giải pháp tích hợp tìm kiếm lai Hybrid RAG (kết hợp Dense Vector và Sparse BM25 với thuật toán RRF $k = 60$), đối chiếu tọa độ hộp chữ chuẩn hóa trên bản PDF gốc, mạng tri thức 4 thực thể (GraphRAG: *Paper, Method, Dataset, MetricResult*), cùng tác nhân AI ReAct tự động sửa lỗi biên dịch LaTeX trong môi trường đám mây cô lập.

Kết quả thực nghiệm của hệ thống đã được kiểm chứng qua 580 kịch bản kiểm thử tự động (340 kiểm thử Backend pytest và 240 kiểm thử Frontend vitest trên 58 tệp, tổng thời gian thực thi $13{,}02$ giây). Trên bộ dữ liệu đối chuẩn *Golden Benchmark* gồm 36 câu hỏi học thuật trải đều trên 12 ấn phẩm kinh điển, cơ chế truy xuất lai Hybrid RRF ($k = 60$) đạt độ thu hồi Recall@5 $= 94{,}44\%$ (vượt trội so với chỉ dùng Dense $80{,}56\%$ và chỉ dùng Sparse $83{,}33\%$), điểm bám sát bằng chứng

Groundedness $= 0{,}9775$, độ liên quan Relevance $= 0{,}9471$ và độ chính xác định vị khung bao tọa độ IoU $= 0{,}9266 \pm 0{,}0126$. Đồng thời, trên 25 ca kiểm thử lỗi biên dịch LaTeX thuộc 5 nhóm lỗi cú pháp phổ biến, chu trình Agentic ReAct kết hợp đạt tỷ lệ tự sửa lỗi thành công 96% ($24/25$ ca), cải thiện rõ rệt so với phương pháp Heuristic (84%) và chỉ dùng biểu thức chính quy AST/Regex (44%). Trong quá trình triển khai, tác giả có sử dụng các công cụ trợ lý AI hỗ trợ sinh mã khung và rà soát biên tập; toàn bộ kiến trúc hệ thống, thuật toán cốt lõi và số liệu thực nghiệm đều do tác giả trực tiếp thiết kế, kiểm định và chịu trách nhiệm học thuật. Báo cáo gồm 4 chương chính phản ánh toàn diện quá trình phân tích, thiết kế, cài đặt và đánh giá hệ thống.

CHƯƠNG 1: TỔNG QUAN VỀ ĐỀ TÀI

1.1 Giới thiệu đề tài

Đề tài tốt nghiệp *"Xây dựng nền tảng AI hỗ trợ nghiên cứu và viết bài báo khoa học VinUni dựa trên Agentic RAG, truy xuất tri thức học thuật và kiểm chứng trích dẫn"* được nghiên cứu và hiện thực hóa thông qua hệ thống phần mềm VeriScholar.

Mục tiêu trọng tâm của đề tài là xây dựng một nền tảng trợ lý trí tuệ nhân tạo hỗ trợ quy trình nghiên cứu khoa học lấy môi trường đại học định hướng nghiên cứu quốc tế như VinUniversity làm bối cảnh đối chuẩn điển hình, đồng thời phục vụ rộng rãi đội ngũ giảng viên, nghiên cứu sinh, học viên cao học và sinh viên đại học. Hệ thống liên kết xuyên suốt bốn không gian làm việc trong vòng đời của một công trình khoa học: từ khâu tìm kiếm khám phá ấn phẩm đa nguồn, phân tích đọc sâu tài liệu chuyên ngành hai cột, tổng hợp mạng lưới tri thức học thuật (GraphRAG), cho đến khâu soạn thảo và biên dịch bản thảo chuẩn quốc tế bằng LaTeX.

Hệ thống VeriScholar tập trung giải quyết bài toán giảm thiểu hiện tượng ảo giác trích dẫn trong nghiên cứu học thuật bằng cách kết hợp kiến trúc Agentic RAG với hai tầng bảo chứng bổ trợ lẫn nhau: (1) *đối chiếu tọa độ trực quan đoạn văn nguồn* (*visual citation grounding*) thông qua việc ánh xạ từng thẻ trích dẫn về đúng hộp tọa độ chuẩn hóa trên trang PDF gốc để học giả trực tiếp thẩm định bằng mắt thường, và (2) *hậu kiểm tính bám sát bằng chứng* (*semantic & lexical claim verification*) nhằm từ chối sinh câu trả lời khi ngữ cảnh thu hồi không đủ độ tin cậy, qua đó hạn chế rủi ro dẫn chiếu sai lệch trong các ấn phẩm khoa học.

1.2 Tính cấp thiết của đề tài

Trong kỷ nguyên số và bùng nổ thông tin hiện nay, số lượng tài liệu khoa học trên toàn cầu tăng trưởng với tốc độ chưa từng có. Mỗi tuần có hàng nghìn bài báo mới xuất hiện trên các cơ sở dữ liệu học thuật quốc tế uy tín như arXiv, Semantic Scholar, IEEE Xplore hay ACM Digital Library. Việc tiếp cận, đọc hiểu, phân loại và tổng hợp tri thức từ kho tài liệu khổng lồ này đặt ra áp lực quá tải thông tin rất lớn đối với các nhà nghiên cứu và sinh viên đại học.

Mặc dù sự xuất hiện của các mô hình ngôn ngữ lớn (LLM) cùng kiến trúc Tạo sinh tăng cường truy xuất (RAG – *Retrieval-Augmented Generation*) mở ra tiềm năng lớn trong việc giảm thiểu hiện tượng suy diễn thiếu căn cứ của mô hình tham số thuần túy [10], việc ứng dụng các công cụ AI phổ quát hiện nay vào quy trình

nghiên cứu khoa học chuyên sâu vẫn đang gặp phải bốn rào cản nghiêm trọng:

(1) **Ảo giác trích dẫn và bịa đặt học thuật** (*Citation Hallucination*): Các mô hình AI tạo sinh thông thường có xu hướng tự sáng tác ra tên tác giả, số định danh DOI, tên tạp chí và các số liệu thực nghiệm không có thật để trả lời câu hỏi của người dùng. Trong nghiên cứu khoa học, việc trích dẫn một nguồn tài liệu không tồn tại hoặc gán ghép sai nội dung nguồn là vi phạm nghiêm trọng chuẩn mực liêm chính học thuật và đạo đức nghiên cứu.

(2) **Thiếu cơ chế kiểm chứng tọa độ trực quan trên tài liệu gốc:** Phần lớn các công cụ hỗ trợ đọc tài liệu hiện nay (như ChatPDF thông thường) sử dụng phương pháp trích xuất văn bản thô, làm mất cấu trúc hai cột đặc trưng của các bài báo khoa học, loại bỏ các công thức toán học và chỉ trích dẫn số trang chung chung. Người nghiên cứu không thể xác minh được khẳng định của AI nằm chính xác tại dòng chữ nào, đoạn văn nào trên trang PDF gốc.

(3) **Sự phân mảnh của hệ sinh thái công cụ hỗ trợ nghiên cứu:** Quy trình nghiên cứu hiện nay bị chia cắt giữa nhiều phần mềm độc lập: học giả phải dùng Zotero / Mendeley để quản lý danh mục tài liệu, dùng Connected Papers / Elicit để khám phá mạng lưới bài báo, dùng ChatPDF / NotebookLM để tra cứu nhanh, và chuyển sang Overleaf để soạn thảo bản thảo LaTeX. Sự thiếu liên kết này làm đứt gãy mạch tư duy và tiêu tốn đáng kể thời gian chuyển đổi ngữ cảnh của nhà nghiên cứu.

(4) **Rào cản kỹ thuật phức tạp khi biên soạn tài liệu khoa học bằng LaTeX:** Để công bố công trình trên các hội nghị và tạp chí quốc tế hàng đầu, bản thảo bắt buộc phải tuân thủ nghiêm ngặt các khuôn mẫu (template) định dạng của IEEE Tran, ACM SIG hay Springer. Khi xảy ra lỗi biên dịch cú pháp trong LaTeX, học giả thường mất nhiều thời gian để đọc các tệp nhật ký (.log) dài hàng trăm dòng nhằm dò tìm lỗi thủ công, làm giảm hiệu suất hoàn thiện bài báo.

Để làm rõ khoảng trống công nghệ trên thực tế, Bảng 1-1 tổng hợp và đối chiếu năng lực kỹ thuật giữa các nền tảng hỗ trợ nghiên cứu học thuật tiêu biểu hiện nay với hệ thống VeriScholar theo năm tiêu chí cốt lõi.

Xuất phát từ các khoảng trống công nghệ được chỉ ra tại Bảng 1-1, việc nghiên cứu và xây dựng một nền tảng trợ lý AI chuyên biệt, có khả năng đối chiếu bằng chứng tọa độ trực quan trên bản PDF gốc, xây dựng đồ thị tri thức 4 thực thể và tự

Bảng 1-1: *Đối chiếu năng lực kỹ thuật giữa các công cụ hỗ trợ nghiên cứu học thuật hiện có và hệ thống VeriScholar*

Nền tảng	Định vị tọa độ PDF 2 cột	Truy xuất lai Hybrid RRF & Cross-Encoder	Đồ thị tri thức 4 thực thể & Wiki	Tự sửa lỗi biên dịch LaTeX (ReAct)	Quản lý khóa BYOK AES-256-GCM
ChatPDF	Chỉ dẫn chiếu mức số trang, mất cấu trúc 2 cột	Chỉ dùng tìm kiếm vector đơn luồng	Không hỗ trợ	Không hỗ trợ soạn thảo LaTeX	Không hỗ trợ khóa riêng
NotebookLM	Trích dẫn đoạn văn bản phẳng, không tô sáng hộp tọa độ PDF	Truy xuất ngữ cảnh dài độc quyền (đóng)	Bản đồ khái niệm tóm tắt cơ bản	Không hỗ trợ biên dịch LaTeX	Không hỗ trợ khóa riêng
Elicit	Trích xuất bảng tóm tắt, thiếu khung tọa độ hình học trên trang	Tìm kiếm ngữ nghĩa kết hợp lọc siêu dữ liệu	Ma trận khái niệm theo cột, không có đồ thị liên kết	Không hỗ trợ soạn thảo LaTeX	Không hỗ trợ khóa riêng
SciSpace	Hỗ trợ quét vùng chọn thủ công trên trang PDF	Tìm kiếm lai cơ bản trên kho công khai	Liên kết trích dẫn bài báo, chưa tách 4 thực thể	Có mẫu văn bản, không tự sửa lỗi biên dịch	Không hỗ trợ khóa riêng
Overleaf	Không tích hợp trình đọc hỏi đáp tài liệu RAG	Không có phân hệ truy xuất tri thức học thuật	Không hỗ trợ đồ thị tri thức	Báo lỗi dòng nhật ký thô, gợi ý thủ công	Không hỗ trợ BYOK
VeriScholar (Đề tài)	Ánh xạ tọa độ chuẩn hóa trên PDF 2 cột (IoU $=$ 0,9266)	Hybrid RRF ($k = 60$) kết hợp Cross-Encoder	Đồ thị 4 thực thể trên PostgreSQL & Wiki cộng đồng	Agentic ReAct tự sửa lỗi trong E2B Sandbox (96%)	Mã hóa AES-256-GCM, giải mã tạm thời trên RAM

động sửa lỗi biên dịch bản thảo LaTeX là một đòi hỏi cấp thiết nhằm nâng cao năng suất và chất lượng công bố khoa học quốc tế, đáp ứng các tiêu chuẩn học thuật khắt khe và định hướng ứng dụng các mô hình AI Agent tự kiểm chứng hiện đại [2, 18].

1.3 Mục tiêu nghiên cứu

1.3.1 Mục tiêu tổng quát

Xây dựng thành công nền tảng phần mềm VeriScholar đóng vai trò trợ lý AI hỗ trợ nghiên cứu khoa học và biên soạn bài báo quốc tế theo chuẩn mực đại học nghiên cứu như VinUniversity, tích hợp khép kín 4 không gian làm việc học thuật trên nền kiến trúc phần mềm lục giác hiện đại, đảm bảo tính toàn vẹn dữ liệu và khả năng đối chiếu trích dẫn trực quan.

1.3.2 Mục tiêu kỹ thuật cụ thể

Đề tài hướng tới hoàn thành 4 mục tiêu kỹ thuật định lượng cụ thể:

(1) Hiện thực hóa cơ chế nạp bài báo hai tầng kết hợp kho lưu trữ định danh theo nội dung CAS (*Content-Addressable Storage*) dựa trên mã băm SHA-256 nhằm loại bỏ các bản lưu vật lý dư thừa khi nhiều học giả cùng nạp một ấn phẩm (tiết kiệm tới 98% dung lượng lưu trữ tệp nhị phân trên máy chủ trong kịch bản 50 học giả cùng nạp một bài báo kinh điển), đồng thời hỗ trợ thu hồi an toàn tài liệu theo cơ chế đếm tham chiếu (ref_count).

(2) Phát triển công cụ tìm kiếm lai Agentic Hybrid RAG kết hợp thuật toán hợp nhất xếp hạng đảo ngược RRF (*Reciprocal Rank Fusion*, $k = 60$) giữa tìm kiếm vector ngữ nghĩa (HNSW Cosine 1536 chiều) và tìm kiếm từ vựng (chỉ mục GIN tsvector kết hợp tái chấm điểm Okapi BM25), đi kèm cơ chế đối chiếu khung viền tọa độ chuẩn hóa trên bản PDF gốc đạt độ chính xác trung bình IoU $\geq 0{,}85$ (thực nghiệm đạt $0{,}9266 \pm 0{,}0126$).

(3) Xây dựng đồ thị tri thức học thuật 4 thực thể (GraphRAG: *Paper*, *Method*, *Dataset*, *MetricResult*) và hệ thống trang tổng hợp tri thức cộng đồng (Graph Wiki) trực tiếp trên cơ sở dữ liệu quan hệ PostgreSQL 16 với tiện ích mở rộng vector hóa pgvector, bảo toàn tính toàn vẹn và nhất quán giao dịch ACID mà không cần duy trì các hệ quản trị cơ sở dữ liệu đồ thị rời rạc.

(4) Tích hợp tác nhân AI dựa trên máy trạng thái LangGraph ReAct hoạt động trong môi trường đám mây cô lập E2B Sandbox MicroVM, có khả năng tự

động phân tích nhật ký lỗi và vá lỗi biên dịch LaTeX đạt tỷ lệ thành công trên 90% (thực nghiệm đạt 96% trên tập 25 ca kiểm thử lỗi chuẩn).

1.4 Nội dung nghiên cứu và phương pháp tiếp cận

1.4.1 Nội dung nghiên cứu

Đề tài tập trung nghiên cứu và giải quyết các nội dung khoa học trọng tâm:

- **Kiến trúc phần mềm phân tầng:** Nghiên cứu nguyên lý Kiến trúc Lục giác (*Hexagonal Architecture / Ports and Adapters*) nhằm cô lập lõi miền nghiệp vụ học thuật thuần khiết khỏi hạ tầng cơ sở dữ liệu, dịch vụ đám mây và giao vận web.
- **Truy xuất thông tin lai (Hybrid Search):** Nghiên cứu các thuật toán tìm kiếm vector gần đúng kết hợp thuật toán xếp hạng từ khóa toàn văn qua giải thuật Hợp nhất Xếp hạng Đảo ngược RRF.
- **Xử lý tài liệu và tọa độ trực quan:** Nghiên cứu kỹ thuật trích xuất tài liệu PDF khoa học hai cột và thuật toán chuẩn hóa hệ tọa độ hộp chữ Bounding Box $[x_0, y_0, x_1, y_1]$ trên dải giá trị chuẩn $[0..1000]$.
- **Agent thông minh tự phục hồi:** Nghiên cứu chu trình Agent ReAct (Suy luận – Hành động – Quan sát) trong việc tự động phân tích và khắc phục lỗi cú pháp biên dịch tài liệu học thuật trong môi trường Sandbox an toàn.
- **Bảo mật và quyền riêng tư:** Nghiên cứu cơ chế bảo mật kho khóa cá nhân (BYOK – *Bring Your Own Key*) mã hóa khóa API bằng thuật toán AES-256-GCM với nguyên tắc Zero-Retention trên bộ nhớ RAM.

1.4.2 Phương pháp tiếp cận

Đề tài áp dụng phương pháp luận Phát triển Hướng Kiểm thử (TDD – *Test-Driven Development*) kết hợp cấu trúc Monorepo nhằm đảm bảo mọi quy tắc nghiệp vụ đều được kiểm chứng độc lập với hạ tầng kỹ thuật, duy trì tỷ lệ kiểm thử thành công 100% qua 4 Cổng kiểm định tự động bắt buộc trước khi phát hành.

1.5 Phạm vi và đối tượng nghiên cứu

Đề tài xác định rõ đối tượng và phạm vi nghiên cứu:

- **Đối tượng nghiên cứu:** Quy trình nghiên cứu khoa học, phân tích bố cục tài liệu chuyên sâu, trích xuất mạng lưới tri thức học thuật và kỹ thuật biên dịch bản thảo khoa học chuẩn quốc tế.
- **Khách thể thụ hưởng và bối cảnh đối chuẩn:** Đề tài lấy môi trường học thuật xuất bản quốc tế tại Trường Đại học VinUni (VinUniversity) làm bối cảnh đại học nghiên cứu đối chuẩn điển hình để xây dựng các yêu cầu nghiệp vụ; hệ thống đồng thời phục vụ rộng rãi giảng viên, nghiên cứu sinh, học viên cao học và sinh viên tại các cơ sở giáo dục đại học định hướng nghiên cứu.
- **Định dạng tài liệu và ngôn ngữ đánh giá:** Phân hệ phân tích bố cục, truy xuất lai (Hybrid RAG) và đồ thị tri thức (GraphRAG) được thiết kế và đánh giá thực nghiệm trọng tâm trên các ấn phẩm khoa học chuẩn PDF bố cục 1–2 cột bằng tiếng Anh (ngôn ngữ chuẩn của các hội nghị IEEE, ACM, NeurIPS, ICLR); trong khi phân hệ soạn thảo bản thảo LaTeX và giao diện hệ thống hỗ trợ đầy đủ bảng mã Unicode tiếng Việt và tiếng Anh cùng định dạng trích dẫn BibTeX.
- **Hạ tầng kỹ thuật triển khai:** Máy chủ đám mây Linux (Ubuntu 24.04 LTS trên Microsoft Azure), hệ quản trị PostgreSQL 16 tích hợp pgvector, bộ đệm Redis 7, nền tảng container Docker, môi trường Sandbox E2B và nền tảng giám sát LLM Langfuse.

1.6 Bố cục của báo cáo đồ án tốt nghiệp

Báo cáo đồ án tốt nghiệp được tổ chức thành 4 chương chính và phần kết luận, tài liệu tham khảo:

- **Chương 1: Tổng quan về đề tài** – Trình bày bối cảnh ra đời, tính cấp thiết của đề tài kèm bảng đối chiếu các công cụ hiện hành, các mục tiêu nghiên cứu tổng quát và cụ thể, nội dung nghiên cứu, phương pháp tiếp cận, phạm vi nghiên cứu và bố cục của báo cáo.
- **Chương 2: Cơ sở lý thuyết và công nghệ** – Trình bày có hệ thống các công nghệ nền tảng và cơ sở lý thuyết khoa học được ứng dụng trong đề tài, bao gồm: Hệ quản trị cơ sở dữ liệu PostgreSQL và Redis; Kiến trúc Lục giác (Hexagonal Architecture); Quy trình tạo sinh tăng cường truy xuất nâng cao (Advanced RAG), phân tích bố cục PDF hai cột, thuật toán RRF và tái xếp hạng Cross-Encoder; Đồ thị tri thức (Knowledge Graph), GraphRAG phân

cụm cộng đồng Louvain và hệ tri thức biên dịch Graph Wiki; Mô hình Agent thông minh Agentic AI, mẫu chu trình ReAct và cơ chế tự đánh giá; Thư viện điều phối máy trạng thái LangGraph; Dịch vụ suy luận OpenAI API trên hạ tầng đám mây Microsoft Azure, Sandbox an toàn E2B và mô hình bảo mật quản lý khóa BYOK AES-256-GCM; Nền tảng truy vết và giám sát LLM Langfuse; cùng các ngôn ngữ, khung làm việc và công cụ phát triển phần mềm hiện đại.

- **Chương 3: Phân tích và thiết kế hệ thống** – Khảo sát các luồng nghiệp vụ học thuật và thiết kế các giải thuật cốt lõi, đặc tả các Use Case chuẩn BA, thiết kế biểu đồ tuần tự UML, kiến trúc Monorepo phân tách ranh giới và thiết kế chi tiết 20 bảng cơ sở dữ liệu vật lý thuộc 5 phân hệ kèm lược đồ thực thể quan hệ (ERD).
- **Chương 4: Cài đặt và kết quả thực nghiệm** – Trình bày cấu hình môi trường triển khai thực tế, giao diện trực quan của các phân hệ chức năng chính, kết quả kiểm thử tự động 100% qua 4 Cổng xác minh (580 kịch bản kiểm thử), cùng các bảng kết quả thực nghiệm đối chứng (Ablation Study) về truy xuất lai, tự sửa lỗi LaTeX và số liệu giám sát vận hành trên nền tảng Langfuse Cloud.
- **Kết luận và hướng phát triển** – Tổng kết các kết quả đạt được của đề tài đối chiếu với mục tiêu ban đầu, phân tích khách quan các hạn chế kỹ thuật còn tồn tại và đề xuất các hướng phát triển mở rộng hệ thống trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT VÀ CÔNG NGHỆ

2.1 Tổng quan về PostgreSQL và Redis

2.1.1 PostgreSQL là gì?

PostgreSQL là hệ quản trị cơ sở dữ liệu quan hệ đối tượng mã nguồn mở (*Object-Relational Database Management System – ORDBMS*) tiên tiến và đáng tin cậy nhất hiện nay, được phát triển liên tục hơn 35 năm bởi cộng đồng khoa học máy tính quốc tế. Được thiết kế với triết lý tuân thủ nghiêm ngặt chuẩn kiến trúc ANSI/ISO SQL, PostgreSQL nổi bật với độ bền vững dữ liệu, khả năng mở rộng linh hoạt và cơ chế giao dịch đạt chuẩn ACID (*Atomicity, Consistency, Isolation, Durability*) tuyệt đối.



Hình 2-1: *PostgreSQL*

Các đặc điểm kỹ thuật then chốt của PostgreSQL bao gồm:

- **Cơ chế kiểm soát tương tranh đa phiên bản (MVCC):** Cho phép các thao tác đọc không chặn ghi và ghi không chặn đọc thông qua cơ chế cô lập ảnh chụp (*Snapshot Isolation*), duy trì thông lượng truy vấn cao ngay cả khi nhiều học giả cùng truy cập đồng thời (các xung đột ghi chéo nếu phát sinh sẽ được bộ phát hiện khóa tương tranh của hệ quản trị tự động xử lý).
- **Hỗ trợ dữ liệu bán cấu trúc (JSONB):** Lưu trữ và đánh chỉ mục nhị phân các cấu trúc dữ liệu JSON phức tạp với hiệu suất truy vấn tương đương các trường dữ liệu quan hệ thông thường.
- **Kiến trúc mở rộng mạnh mẽ (Extensibility):** Cho phép tích hợp các kiểu dữ liệu mới, hàm người dùng tự định nghĩa và các tiện ích mở rộng (*extensions*) cấp thấp chạy trực tiếp trong nhân cơ sở dữ liệu.

Trong đề tài VeriScholar, PostgreSQL giữ vai trò là xương sống lưu trữ dữ liệu trung tâm, quản lý toàn bộ thực thể học thuật, lịch sử phiên nghiên cứu, siêu dữ liệu phân đoạn tài liệu và bản đồ băm định danh nội dung (CAS).

2.1.2 Extension pgvector và chỉ mục HNSW

pgvector là một tiện ích mở rộng mã nguồn mở dành riêng cho PostgreSQL, bổ sung kiểu dữ liệu vector và các phép toán đo lường khoảng cách không gian đa chiều (khoảng cách Euclid L_2, khoảng cách Cosine, và tích vô hướng Inner Product). Nhờ pgvector, PostgreSQL hoạt động như một hệ quản trị cơ sở dữ liệu tích hợp đa mô hình quan hệ – vector (*Integrated Relational-Vector DBMS*), cho phép lưu trữ và truy vấn trực tiếp các vector Embedding ngữ nghĩa 1536 chiều ngay bên cạnh dữ liệu quan hệ chuẩn ACID.

Điểm đột phá trong việc tối ưu hóa tốc độ tìm kiếm láng giềng gần nhất xấp xỉ (*Approximate Nearest Neighbor – ANN*) của pgvector là việc hiện thực hóa thuật toán chỉ mục HNSW (*Hierarchical Navigable Small World*) [11]:

- **Nguyên lý phân tầng đa lớp:** HNSW tổ chức không gian vector thành cấu trúc đồ thị phân tầng tương tự như danh sách liên kết bước nhảy (*Skip List*). Các tầng trên cùng chứa các liên kết xa với mật độ thưa giúp định vị nhanh vùng lân cận tổng quát; các tầng dưới có mật độ nút dày đặc dần cho phép thu hẹp chính xác không gian tìm kiếm.

- **Độ phức tạp truy vấn kỳ vọng thực nghiệm $O(\log N)$:** Thay vì phải tính toán khoảng cách tuần tự trên toàn bộ kho vector (*Flat Search* với độ phức tạp $O(N)$), thuật toán định tuyến tham lam (*Greedy Routing*) trên đồ thị thế giới nhỏ phân tầng của HNSW đạt độ phức tạp kỳ vọng thực nghiệm tỷ lệ logarit $O(\log N)$ theo số lượng đỉnh N trên các tập dữ liệu thực tế [11].

- **Cập nhật linh hoạt và kiểm soát độ thu hồi theo siêu tham số:** Khác với thuật toán IVFFlat đòi hỏi phải huấn luyện lại tâm cụm khi phân phối dữ liệu biến động lớn, HNSW cho phép chèn mới vector trực tuyến. Độ thu hồi (*Recall*) và tốc độ truy vấn của chỉ mục được điều chỉnh linh hoạt thông qua ba siêu tham số cốt lõi: số liên kết hai chiều tối đa trên mỗi nút (m), kích thước danh sách ứng viên động khi dựng đồ thị (ef_construction) và kích thước danh sách ứng viên khi tìm kiếm (ef_search).

Việc hợp nhất dữ liệu quan hệ và dữ liệu vector Embedding trong cùng một hệ quản trị PostgreSQL giúp loại bỏ hoàn toàn sự cồng kềnh của kiến trúc đa cơ sở

dữ liệu, đảm bảo tính nhất quán giao dịch ACID trong suốt quá trình người dùng lưu trữ hoặc xóa tài liệu.

2.1.3 Redis là gì?

Redis (*Remote Dictionary Server*) là hệ thống lưu trữ cấu trúc dữ liệu trong bộ nhớ RAM (*In-Memory Data Store*) mã nguồn mở với hiệu năng cực cao. Nhờ cơ chế lưu trữ toàn bộ dữ liệu trực tiếp trên bộ nhớ truy cập ngẫu nhiên và mô hình xử lý vào/ra đa hợp bất đồng bộ (*I/O Multiplexing*), Redis có khả năng đáp ứng hàng trăm nghìn thao tác đọc ghi mỗi giây với độ trễ tính bằng micro giây.



Hình 2-2: *Redis*

Các đặc tính cốt lõi của Redis bao gồm:

- **Đa dạng cấu trúc dữ liệu bản địa:** Hỗ trợ phong phú các kiểu dữ liệu như chuỗi (*Strings*), bảng băm (*Hashes*), danh sách (*Lists*), tập hợp (*Sets*), và tập hợp sắp xếp theo điểm (*Sorted Sets*).
- **Cơ chế hết hạn tự động (TTL – Time-To-Live):** Cho phép thiết lập thời gian sống chính xác cho từng khóa dữ liệu, giúp bộ nhớ tự động giải phóng các bản ghi tạm mà không cần can thiệp thủ công.
- **Cơ chế Xuất bản / Đăng ký (Pub/Sub):** Cung cấp hạ tầng truyền thông điệp nhẹ và tức thời giữa các tiến trình hệ thống phân tán.

Trong kiến trúc VeriScholar, Redis được ứng dụng vào ba nhiệm vụ trọng yếu:

(1) **Bộ đệm xác thực và danh sách đen mã thông báo (Token Blacklist):** Lưu trữ các phiên đăng nhập hợp lệ và thu hồi tức thì quyền truy cập của người dùng khi đăng xuất trước khi mã JWT hết hạn.

(2) **Kiểm soát giới hạn tần suất yêu cầu (Rate Limiting):** Bảo vệ các cổng tiếp nhận suy luận AI khỏi hiện tượng lạm dụng hoặc tấn công từ chối dịch vụ.

(3) **Đệm kết quả tra cứu định danh học thuật (DOI Cache):** Lưu tạm kết quả giải mã siêu dữ liệu của các tài liệu khoa học thường xuyên được trích dẫn, giảm tải cho mạng lưới tra cứu bên ngoài.

2.2 Tổng quan về Hexagonal Architecture

2.2.1 Hexagonal Architecture là gì?

Kiến trúc Lục giác (*Hexagonal Architecture*), còn được gọi là mẫu kiến trúc Cổng và Bộ điều hợp (*Ports and Adapters Pattern*), do chuyên gia phần mềm Alistair Cockburn khởi xướng vào năm 2005 [4]. Mục tiêu cốt lõi của kiến trúc này là giải quyết triệt để bài toán phụ thuộc chặt chẽ (*tight coupling*) giữa logic nghiệp vụ trung tâm của phần mềm với các công nghệ giao vận bên ngoài như hệ quản trị cơ sở dữ liệu, giao thức mạng, và giao diện người dùng.

Trong các kiến trúc phân tầng truyền thống (như mô hình 3 tầng Presentation - Business Logic - Data Access), mã nguồn nghiệp vụ thường bị phụ thuộc trực tiếp vào cơ sở dữ liệu hoặc khung giao diện. Ngược lại, Hexagonal Architecture thiết lập một ranh giới bảo vệ nghiêm ngặt: logic nghiệp vụ học thuật nằm tại trung tâm và hoàn toàn cô lập, không mang bất kỳ sự ràng buộc nào với các thư viện ngoại vi, hệ quản trị cơ sở dữ liệu hay giao thức mạng.

Các lợi ích nền tảng của Hexagonal Architecture bao gồm:

- **Độc lập công nghệ:** Cho phép thay thế hệ quản trị cơ sở dữ liệu, chuyển đổi nhà cung cấp mô hình trí tuệ nhân tạo (LLM), hoặc đổi mới giao diện người dùng mà không cần sửa đổi bất kỳ dòng mã logic nghiệp vụ nào.
- **Kiểm thử tự động dễ dàng (Testability):** Logic nghiệp vụ có thể được kiểm thử hoàn toàn độc lập với các cổng giả lập (*Mock Ports*) trong bộ nhớ mà không cần kết nối mạng hay khởi động cơ sở dữ liệu thực tế.
- **Khả năng bảo trì và tiến hóa bền vững:** Ranh giới phân định rõ ràng giúp kiến trúc thích ứng linh hoạt trước sự thay đổi công nghệ nhanh chóng của các nền tảng AI.

2.2.2 Kiến trúc Ports and Adapters

Hệ thống theo Kiến trúc Lục giác được cấu trúc thành ba phân vùng độc lập tuân thủ nghiêm ngặt nguyên lý đảo ngược phụ thuộc (*Dependency Inversion Principle*):

(1) **Tầng lõi miền nghiệp vụ thuần khiết (*Pure Core Domain*):** Nằm tại trung tâm của hệ thống, chỉ chứa các thực thể nghiệp vụ (*Entities*), quy tắc logic và

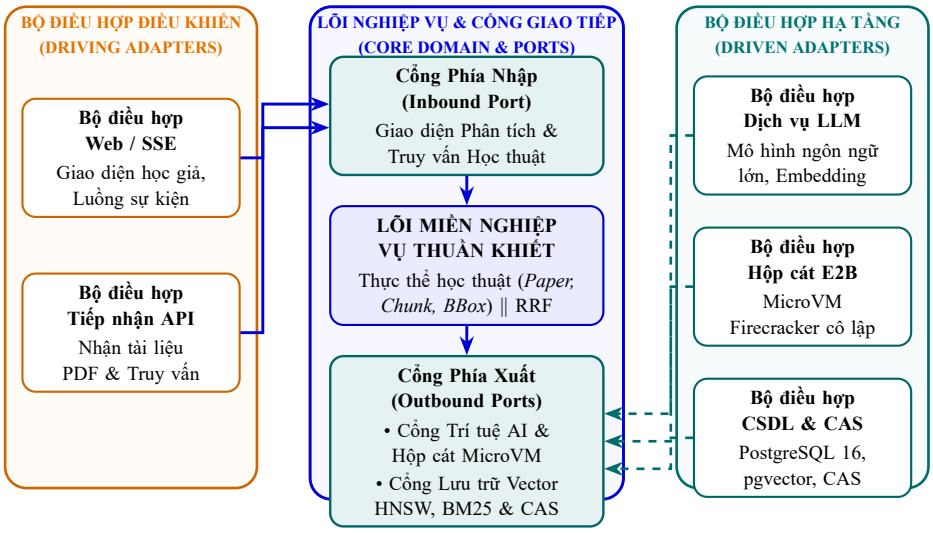
các thuật toán tính toán học thuật. Vùng này tuân thủ nguyên tắc cô lập tuyệt đối: chỉ sử dụng các cấu trúc dữ liệu bản địa của ngôn ngữ lập trình, hoàn toàn không phụ thuộc vào bất kỳ thư viện giao tiếp mạng, web framework hay hệ quản trị cơ sở dữ liệu nào.

(2) Tầng các cổng giao tiếp trừu tượng (*Ports*): Là các giao diện trừu tượng (*Interfaces*) nằm trong tầng lõi nghiệp vụ. Cổng quy định hợp đồng giao tiếp chuẩn mà các thành phần bên ngoài bắt buộc phải tuân thủ để phục vụ nghiệp vụ (chẳng hạn như: cổng phân giải tài liệu học thuật, cổng trích xuất tài liệu PDF, hay cổng lưu trữ dữ liệu). Có hai loại cổng chính:

- *Cổng phía nhập (Inbound/Driving Ports):* Định nghĩa các phương thức cho phép Agent bên ngoài kích hoạt quy trình nghiệp vụ của hệ thống.
- *Cổng phía xuất (Outbound/Driven Ports):* Định nghĩa các giao diện cho phép lõi nghiệp vụ yêu cầu tài nguyên hoặc lưu trữ từ hạ tầng bên ngoài.

(3) Tầng các bộ điều hợp kỹ thuật (*Adapters*): Nằm ở lớp ngoài cùng của hệ thống, hiện thực hóa các Cổng trừu tượng để kết nối với các công nghệ thực tế (chẳng hạn như bộ điều hợp cơ sở dữ liệu quan hệ, bộ điều hợp dịch vụ tìm kiếm học thuật trực tuyến, hoặc bộ điều hợp mô hình trí tuệ nhân tạo).

Mô hình phân tầng kiến trúc và ranh giới phân tách trừu tượng của hệ thống được mô tả theo dạng các khối năng lực module trong Hình 2-3.



Hình 2-3: *Mô hình phân rã module và cấu trúc Cổng – Điều hợp (Ports and Adapters)*

Luồng phụ thuộc trong hệ thống luôn có chiều hướng tâm: các bộ điều hợp kỹ thuật bên ngoài phụ thuộc vào các Cổng trong lõi, trong khi logic nghiệp vụ học

thuật ở trung tâm hoàn toàn không biết gì về cơ sở hạ tầng thực tế bên dưới. Điều này bảo đảm tính liêm chính và sự ổn định cao độ cho toàn bộ nền tảng.

2.3 Tổng quan về Advanced RAG

2.3.1 RAG là gì? (Retrieval-Augmented Generation)

RAG (*Retrieval-Augmented Generation* – Tạo sinh tăng cường truy xuất) là một kỹ thuật kiến trúc đột phá trong xử lý ngôn ngữ tự nhiên, được Patrick Lewis và các cộng sự tại Meta AI giới thiệu lần đầu tiên vào năm 2020 [10]. Ý tưởng cốt lõi của RAG là kết hợp sức mạnh suy luận ngôn ngữ của các mô hình ngôn ngữ lớn (LLM) với khả năng truy xuất thông tin chính xác từ một kho dữ liệu tri thức bên ngoài (*External Knowledge Base*).

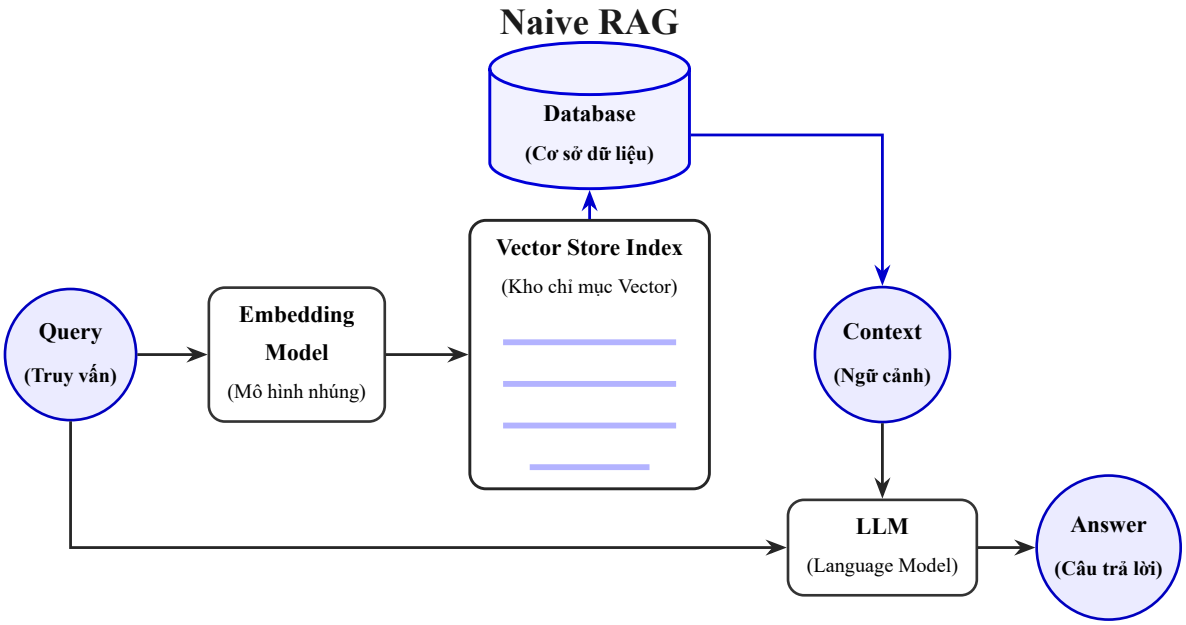
Trước khi có RAG, các mô hình ngôn ngữ lớn thuần túy dựa vào tập tham số tĩnh được học trong quá trình tiền huấn luyện. Điều này bộc lộ ba hạn chế nghiêm trọng:

(1) Tri thức bị giới hạn thời gian (*Knowledge Cutoff*): Mô hình không có khả năng cập nhật các công bố khoa học hoặc dữ liệu phát sinh sau thời điểm hoàn tất huấn luyện.

(2) Hiện tượng ảo giác (*Hallucination*): Khi đối mặt với các câu hỏi chuyên sâu hoặc thiếu thông tin, mô hình có xu hướng tự tạo ra các câu trả lời nghe có vẻ hợp lý nhưng hoàn toàn sai lệch về mặt sự thật học thuật.

(3) Thiếu minh bạch và khả năng kiểm chứng: Mô hình ngôn ngữ truyền thống không cung cấp nguồn dẫn chứng gốc, khiến các nhà nghiên cứu không thể xác minh độ tin cậy của phát biểu.

Kiến trúc RAG truyền thống (*Naive RAG*) vận hành qua ba pha tuần tự: Đánh chỉ mục (*Indexing*), Truy xuất (*Retrieval*) và Tạo sinh (*Generation*). Mô hình kiến trúc chi tiết được thể hiện trong Hình 2-4.



Hình 2-4: *Mô hình kiến trúc tạo sinh tăng cường truy xuất truyền thống (Naive RAG)*

2.3.2 Advanced RAG là gì?

Mặc dù Naive RAG giải quyết được vấn đề cập nhật thông tin, mô hình này bộc lộ nhiều điểm nghẽn khi áp dụng vào tài liệu học thuật: việc cắt đoạn cố định làm đứt gãy ngữ cảnh, tìm kiếm vector đơn lẻ bỏ sót thuật ngữ chuyên ngành, và câu trả lời dễ lẫn tạp thông tin nhiễu.

Theo phân loại chuẩn tắc của Gao và cộng sự [8], Advanced RAG (RAG Nâng cao) ra đời nhằm khắc phục triệt để các hạn chế của Naive RAG thông qua việc tái cấu trúc toàn diện quy trình truy xuất tri thức thành ba giai đoạn tinh chỉnh chuyên sâu [8, 2, 18]:

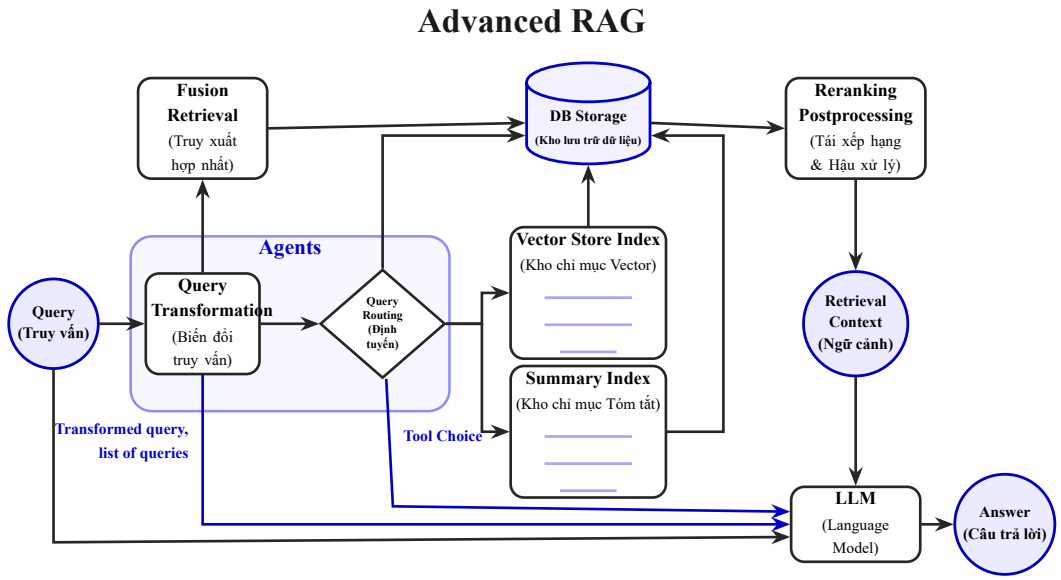
(1) Tiền truy xuất (*Pre-Retrieval*): Viết lại truy vấn (*Query Rewriting*), phân rã câu hỏi phụ (*Sub-queries*), phân tích bố cục tài liệu khoa học hai cột (*Two-Column PDF Layout Analysis*) bằng cách phân tách ranh giới hình học cột trái/phải theo ngưỡng hoành độ tâm khối để tránh trộn dòng ngang, và chuẩn hóa tọa độ khung bao (*Bounding Box Normalization*) về miền tỷ lệ không thứ nguyên $[0, 1]$ thông qua phép biến đổi $(x_0/W, y_0/H, x_1/W, y_1/H)$ theo chiều rộng W và chiều cao H của trang PDF.

(2) Truy xuất (*Retrieval*): Tìm kiếm lai (*Hybrid Search*) kết hợp vector đa chiều và từ khóa thưa để tối ưu hóa độ phủ tri thức.

(3) Hậu truy xuất (*Post-Retrieval*): Tái xếp hạng chéo (*Cross-Encoder Reranking*) chấm điểm tương tác toàn phần giữa cặp truy vấn – đoạn văn nhằm lọc nhiễu ngữ nghĩa, nén chọn lọc ngữ cảnh và kiểm chứng trích dẫn tọa độ Bounding

Box.

Kiến trúc Advanced RAG phối hợp đa Agent điều phối được mô hình hóa trong Hình 2-5.



Hình 2-5: *Mô hình kiến trúc tạo sinh tăng cường truy xuất nâng cao (Advanced RAG)*

2.3.3 Hybrid Search và thuật toán RRF

Trong hệ thống truy xuất tài liệu khoa học, hai phương pháp tìm kiếm truyền thống bộc lộ những ưu và nhược điểm trái ngược nhau:

(1) Truy xuất vector đặc (*Dense Vector Retrieval*): Văn bản được ánh xạ thành các vector Embedding ngữ nghĩa trong không gian nhiều chiều (1536 chiều). Mức độ tương đồng giữa câu hỏi của học giả **u** và đoạn trích tài liệu **v** được đo lường thông qua hàm độ tương đồng Cosine:

$$\text{CosineSimilarity}(\mathbf{u}, \mathbf{v}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} = \frac{\sum_{i=1}^{d} u_i v_i}{\sqrt{\sum_{i=1}^{d} u_i^2} \sqrt{\sum_{i=1}^{d} v_i^2}} \tag{2.1}$$

Ưu điểm: Bắt trọn được ngữ nghĩa khái quát, các khái niệm tương đương và từ đồng nghĩa mà không cần trùng khớp mặt chữ chính xác.
Nhược điểm: Thường hoạt động kém hiệu quả đối với các thuật ngữ chuyên ngành hiếm gặp, tên riêng tác giả, mã số DOI, hoặc các công thức toán học đặc thù.

(2) Truy xuất từ khóa thưa hai pha (*Two-Stage Sparse Keyword Retrieval* –

***GIN ts_rank_cd & Okapi BM25*):** Trong cơ sở dữ liệu PostgreSQL, việc truy xuất từ khóa thưa được thực hiện qua cơ chế hai pha bổ trợ: pha đầu tiên sử dụng chỉ mục đảo tổng quát GIN trên kiểu dữ liệu tsvector kết hợp hàm xếp hạng mật độ bao phủ ts_rank_cd để lọc nhanh tập ứng viên tiềm năng; pha thứ hai tái chấm điểm chính xác tập ứng viên dựa trên tần suất từ khóa (TF) và nghịch đảo tần suất tài liệu (IDF) theo công thức chuẩn hóa Okapi BM25 [13]:

$$\text{Score}_{\text{BM25}}(D,Q) = \sum_{i=1}^{n} \text{IDF}(q_i) \cdot \frac{f(q_i,D)\cdot(k_1+1)}{f(q_i,D)+k_1\cdot\left(1-b+b\cdot\frac{|D|}{\text{avgdl}}\right)} \tag{2.2}$$

trong đó $f(q_i, D)$ là tần suất từ q_i trong tài liệu D, $|D|$ là độ dài tài liệu, avgdl là độ dài trung bình, $k_1 = 1{,}5$ (trong dải tiêu chuẩn $[1{,}2;\ 2{,}0]$) và $b = 0{,}75$.
Ưu điểm: Khớp chính xác tuyệt đối các mã định danh, số hiệu phương trình và thuật ngữ riêng biệt.
Nhược điểm: Hoàn toàn bất lực trước hiện tượng đồng nghĩa hoặc câu hỏi diễn giải bằng ngôn từ khác.

Bảng 2-1 so sánh đối chiếu đặc tính vận hành giữa hai phương pháp.

Tiêu chí so sánh	**Dense Vector (Cosine HNSW)**	**Sparse Keyword (BM25 GIN)**
Không gian biểu diễn	Vector thực liên tục đa chiều (1536 chiều)	Vector từ khóa thưa rời rạc (tsvector)
Khả năng hiểu ngữ nghĩa	Xuất sắc: Bắt được từ đồng nghĩa, ẩn dụ	Kém: Chỉ dựa trên trùng khớp ký tự
Độ chính xác thuật ngữ	Trung bình: Dễ bị nhiễu với từ viết tắt	Xuất sắc: Khớp tuyệt đối DOI, tên riêng
Cấu trúc chỉ mục	Đồ thị láng giềng HNSW	Danh sách nghịch đảo GIN (ts_rank_cd) + tái chấm điểm BM25

Bảng 2-1: *So sánh đặc tính giữa tìm kiếm ngữ nghĩa Dense Vector và từ khóa BM25*

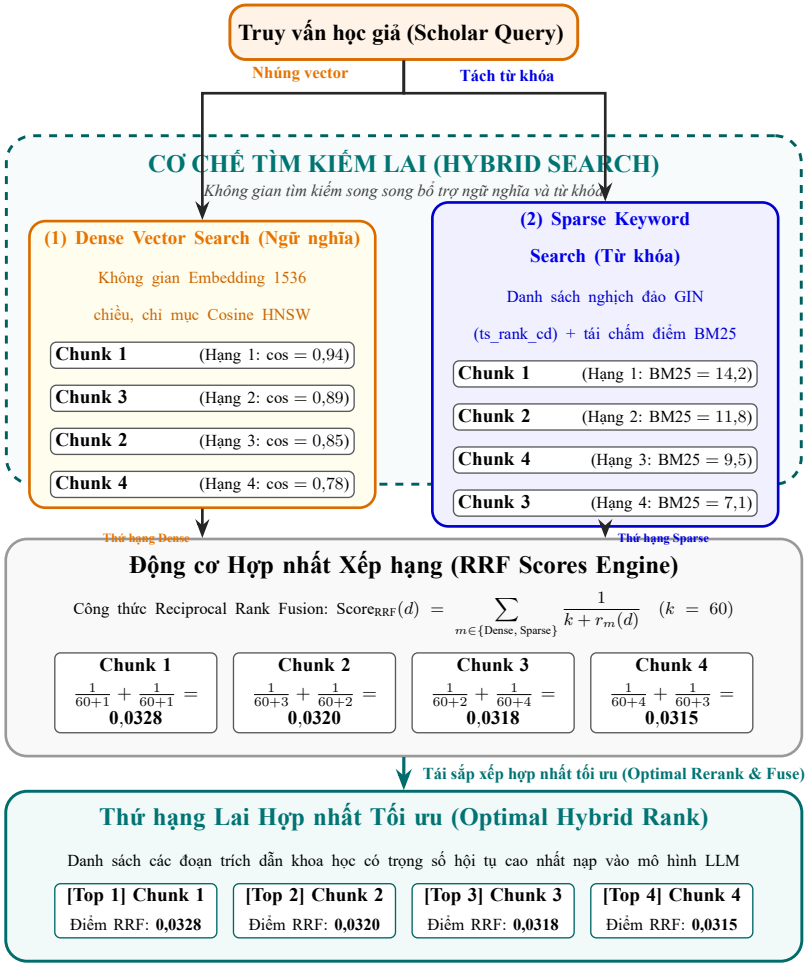
Để dung hòa trọn vẹn sức mạnh của cả hai phương pháp, đề tài ứng dụng thuật toán Hợp nhất Xếp hạng Đảo ngược (*Reciprocal Rank Fusion – RRF*) [5]. Thay vì cộng gộp điểm số vốn không cùng thang đo giữa Cosine và BM25, RRF chỉ tính

toán dựa trên thứ hạng nguyên vị trí của tài liệu trong từng danh sách:

$$\text{RRF_Score}(d) = \sum_{m \in M} \frac{1}{k + r_m(d)} \tag{2.3}$$

trong đó $M = \{\text{Dense Vector}, \text{Sparse BM25}\}$, $r_m(d)$ là thứ hạng nguyên của tài liệu d trong danh sách trả về từ phương pháp m ($r_m(d) \geq 1$), và $k = 60$ là hằng số làm mịn chuẩn hóa học thuật giúp cân bằng điểm số giữa các vị trí dẫn đầu. Sau bước hợp nhất RRF, danh sách ứng viên hội tụ tiếp tục được đưa qua mô hình tái xếp hạng chéo (*Cross-Encoder*) hoặc hàm điều chỉnh mật độ từ khóa để xác lập thứ tự ưu tiên cuối cùng trước khi nạp vào cửa sổ ngữ cảnh của mô hình ngôn ngữ.

Quy trình tìm kiếm lai kết hợp thuật toán RRF và cơ chế tính điểm định lượng phân hạng được thể hiện trực quan trong Hình 2-6.



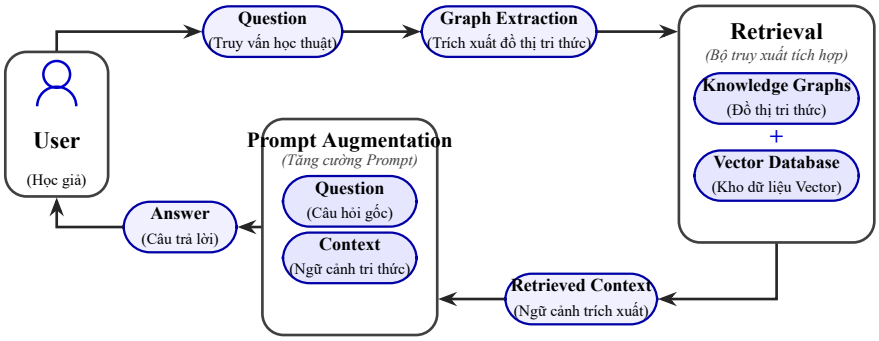
Hình 2-6: *Cơ chế tìm kiếm lai (Hybrid Search) kết hợp thuật toán Hợp nhất Xếp hạng Đảo ngược (RRF)*

2.4 Tổng quan về GraphRAG, Knowledge Graph và Graph Wiki

2.4.1 GraphRAG là gì?

GraphRAG (*Graph-based Retrieval-Augmented Generation*) là bước tiến hóa mang tính đột phá kết hợp giữa Đồ thị tri thức (*Knowledge Graph*) và quy trình tạo sinh tăng cường truy xuất (RAG), do Darren Edge và cộng sự tại Microsoft Research đề xuất năm 2024 [7]. Hạn chế lớn nhất của RAG vector truyền thống là việc xử lý các đoạn văn bản rời rạc, hoàn toàn bất lực trước các câu hỏi tổng hợp mang tính vĩ mô (chẳng hạn như: *"Những phương pháp nào đã được thử nghiệm trên bộ dữ liệu ImageNet và đạt độ chính xác trên 90%?"*). Khi đó, thông tin bị phân mảnh ở nhiều bài báo khác nhau khiến cơ chế tìm kiếm tương đồng vector đơn lẻ bỏ sót phần lớn các mối liên hệ ngữ nghĩa chéo.

GraphRAG giải quyết triệt để bài toán này bằng cách chuyển đổi toàn bộ kho tài liệu khoa học thành đồ thị thực thể, sau đó kết hợp sức mạnh truy xuất cấu trúc đồ thị với tìm kiếm vector đa chiều. Mô hình kiến trúc toàn trình của GraphRAG được mô phỏng chi tiết trong Hình 2-7.



Hình 2-7: *Mô hình kiến trúc tạo sinh tăng cường truy xuất dựa trên đồ thị tri thức (GraphRAG)*

Nhờ sự kết hợp này, hệ thống hỗ trợ cơ chế truy xuất hai cấp độ bổ trợ nhau:

- **Truy xuất mức cục bộ (*Low-Level / Local Retrieval*):** Khai thác các liên kết láng giềng trực tiếp của thực thể trong đồ thị để đối chiếu chính xác chỉ số thực nghiệm, công thức toán học hoặc tọa độ trích dẫn trong từng bài báo.
- **Truy xuất mức toàn cục (*High-Level / Global Retrieval*):** Áp dụng thuật toán phát hiện cộng đồng bằng tối ưu hóa độ mô-đun Louvain (*Louvain Modularity Optimization*) [3] (cùng hướng mở rộng lý thuyết sang thuật toán phân cụm cộng đồng phân tầng Leiden [3]) để gom nhóm các thực thể có mật độ liên kết cao và biên dịch các bản tóm tắt cộng đồng vĩ mô, cho phép trả

lời các câu hỏi so sánh liên ngành, phân tích xu hướng và phát hiện khoảng trống nghiên cứu (*Research Gaps*).

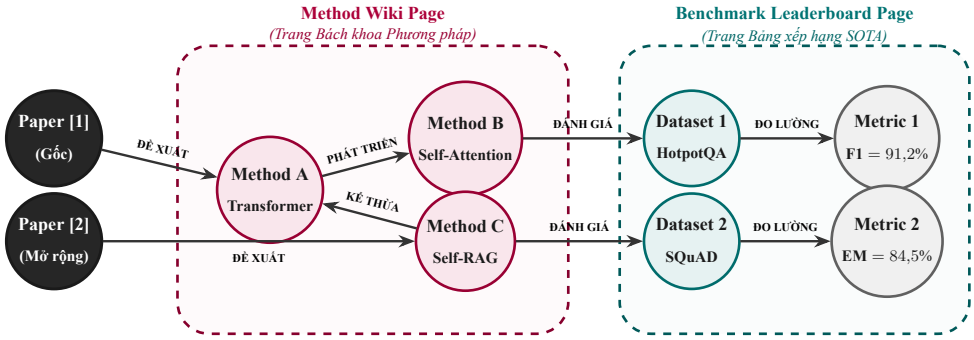
2.4.2 Knowledge Graph là gì? (Đồ thị tri thức)

Đồ thị tri thức (*Knowledge Graph*) là mô hình toán học biểu diễn mạng lưới tri thức dưới dạng một đồ thị có hướng có gán nhãn $G = (V, E)$ [9], trong đó tập đỉnh V đại diện cho các thực thể thế giới thực (*Entities*) và tập cạnh E đại diện cho các mối quan hệ ngữ nghĩa kết nối giữa các thực thể đó (*Semantic Relations*). Tri thức trong đồ thị thường được tổ chức theo cấu trúc bộ ba vị ngữ chuẩn (*Subject – Predicate – Object Triples*), ví dụ: (Paper [1]) $\xrightarrow{\text{ĐỀ XUẤT}}$ (Transformer) [16].

So với các dạng lưu trữ dữ liệu văn bản phi cấu trúc hoặc cơ sở dữ liệu quan hệ bảng phẳng, Knowledge Graph sở hữu ba ưu thế vượt trội:

- **Khả năng suy luận bắc cầu (*Multi-hop Reasoning*):** Đồ thị cho phép lần theo chuỗi các cạnh liên kết để khám phá ra các mối liên hệ gián tiếp giữa các công trình khoa học mà không cần chúng phải xuất hiện cùng nhau trong một văn bản.
- **Định danh và khử nhập nhằng thực thể (*Entity Disambiguation*):** Mỗi bài báo, phương pháp, tập dữ liệu hoặc chỉ số thực nghiệm được định danh duy nhất thông qua một nút mạng chuẩn hóa, hạn chế hiện tượng trùng tên hoặc đồng âm khác nghĩa.
- **Cấu trúc tri thức toàn cục (*Global Structural Context*):** Cung cấp bức tranh toàn cảnh về sự phát triển của các trường phái học thuật, sơ đồ kế thừa giữa các mô hình và chuẩn đánh giá thực nghiệm.

Mạng lưới đồ thị tri thức học thuật gồm bốn loại đỉnh cốt lõi (*Paper Nodes, Method Nodes, Dataset Nodes, Metric Nodes*) và cơ chế biên dịch trang bách khoa cộng đồng được mô hình hóa trong Hình 2-8.



Hình 2-8: *Cấu trúc Đồ thị tri thức học thuật (Academic Knowledge Graph) và cơ chế biên dịch trang bách khoa (Graph Wiki)*

2.4.3 Graph Wiki là gì? (Pre-compiled Community Wiki)

Graph Wiki (*Pre-compiled Community Wiki* – Trang bách khoa cộng đồng biên dịch sẵn) là mô hình tổ chức tri thức kết tinh từ thuật toán phân cụm cộng đồng Louvain [3], chuyển đổi các cụm thực thể và liên kết trích dẫn trong đồ thị tri thức thành các trang tài liệu tổng hợp có cấu trúc liên kết hai chiều.

Thay vì chỉ lưu trữ các đỉnh và cạnh rời rạc trong cơ sở dữ liệu rồi mới tổng hợp lại từ đầu ở mỗi lượt hỏi đáp, cơ chế Graph Wiki biên dịch trước mạng lưới tri thức theo từng cộng đồng chủ đề thành các trang chuyên đề có cấu trúc (như minh họa tại Hình 2-8):

(1) Trang chuyên đề Phương pháp (*Method Pages*): Tổng hợp nguyên lý thuật toán, danh sách các bài báo đề xuất gốc cùng các biến thể kế thừa trong cùng cụm cộng đồng.

(2) Trang chuẩn đánh giá (*Benchmark Pages*): Liệt kê các bộ dữ liệu tiêu chuẩn cùng bảng đối chiếu kết quả thực nghiệm (*Metric Results*) trích xuất từ các công trình trong kho tài liệu.

(3) Liên kết hai chiều (*Bidirectional Hyperlinks*): Cho phép học giả điều hướng trực tiếp giữa các trang khái niệm và các bài báo nguồn có cùng nền tảng lý thuyết, hỗ trợ quá trình tổng quan tài liệu (*Literature Review*) mạch lạc và có dẫn chứng.

Nhờ cơ chế biên dịch trước các trang cộng đồng Graph Wiki, tác tử suy luận có thể truy xuất nhanh ngữ cảnh tổng quan cấp cộng đồng kết hợp với truy vấn đồ thị cục bộ, giảm thiểu độ trễ duyệt đồ thị nhiều bước trong thời gian thực và duy trì tính nhất quán của các dẫn chứng khoa học.

2.5 Tìm hiểu về Agentic AI và ReAct

2.5.1 Khái niệm và kiến trúc tổng quan của Agentic AI

Agentic AI (Trí tuệ nhân tạo có tính Agent) đại diện cho bước chuyển dịch căn bản từ mô hình ngôn ngữ lớn (LLM) bị động (chỉ tạo văn bản theo dạng câu hỏi – câu trả lời một lượt) sang các hệ thống Agent thông minh tự chủ có mục tiêu (*Goal-Oriented Autonomous Agents*). Thay vì cố gắng giải quyết một bài toán phức tạp bằng một lần suy luận duy nhất, Agentic AI có khả năng tự phân tách nhiệm vụ thành các bước nhỏ, sử dụng các công cụ ngoại vi để thu thập dữ liệu thực tế, quan sát môi trường và tự điều chỉnh kế hoạch hành động.

Theo khung lý thuyết chuẩn tắc về Agent thông minh của Lilian Weng [17], một AI Agent hoàn chỉnh được xây dựng dựa trên 4 trụ cột năng lực cốt lõi:

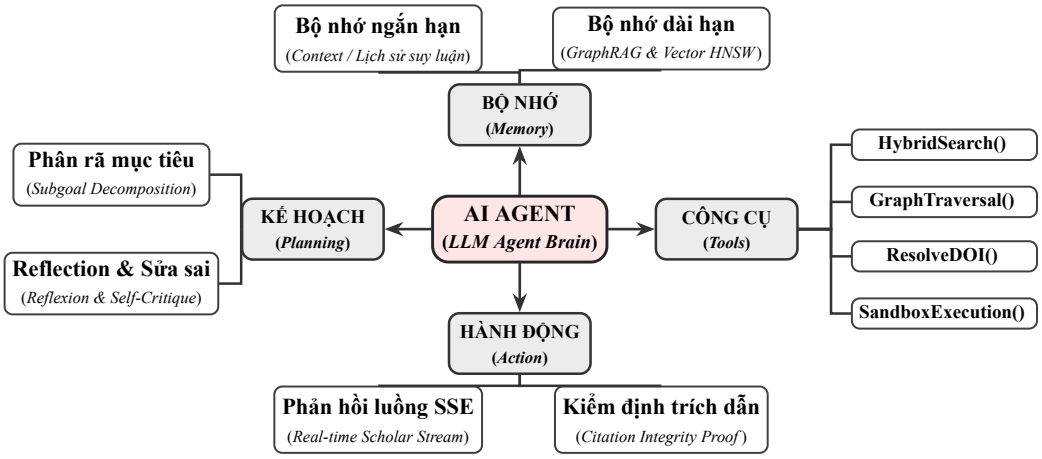
(1) Bộ não trung tâm (*LLM Agent Brain*): Hạt nhân điều phối nhận thức, suy luận logic và ra quyết định hành động.

(2) Bộ nhớ (*Memory*): Gồm bộ nhớ ngắn hạn (*Context* hội thoại) và bộ nhớ dài hạn (*GraphRAG* & Vector).

(3) Lập kế hoạch (*Planning*): Phân rã mục tiêu lớn (*Subgoal Decomposition*) và tự đánh giá sửa sai.

(4) Tập công cụ ngoại vi (*Tools*): Các API tra cứu dữ liệu, tìm kiếm tài liệu và môi trường Sandbox tính toán.

Mô hình kiến trúc tổng thể của AI Agent nghiên cứu học thuật được thể hiện trong Hình 2-9.



Hình 2-9: *Kiến trúc AI Agent Độc lập (LLM-Powered Autonomous Agent)*

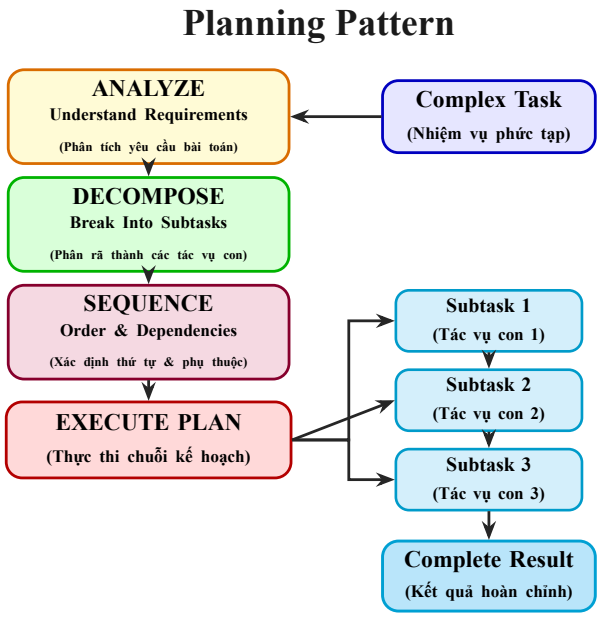
Để hiện thực hóa các năng lực tự chủ trên, cộng đồng khoa học đã chuẩn hóa thành 6 mẫu thiết kế Agent (*Agent Design Patterns*): (1) Lập kế hoạch (*Planning*),

(2) Sử dụng công cụ (*Tool Use*), (3) Suy luận & Hành động (*ReAct*), (4) Tự đánh giá (*Reflection*), (5) Đa Agent (*Multi-Agent*), và (6) Người trong vòng lặp (*Human-in-the-Loop*).

2.5.2 Mẫu kiến trúc Lập kế hoạch (Planning Pattern)

Khi tiếp nhận một yêu cầu nghiên cứu học thuật phức tạp (ví dụ: "Tổng quan các phương pháp GraphRAG trong 3 năm qua và so sánh hiệu năng trên tập dữ liệu chuẩn"), mô hình ngôn ngữ không thể đưa ra lời giải thấu đáo trong một lượt suy luận duy nhất. Mẫu kiến trúc Lập kế hoạch (*Planning Pattern*) giải quyết thách thức này bằng cách mô phỏng tư duy giải quyết vấn đề của các học giả: phân tích yêu cầu, phân rã mục tiêu lớn thành các tác vụ con khả thi, thiết lập trật tự phụ thuộc và điều phối thực thi từng bước.

Quy trình phân rã và điều phối kế hoạch Agent được mô hình hóa trong Hình 2-10.



Hình 2-10: *Mẫu kiến trúc Lập kế hoạch và Phân rã nhiệm vụ (Planning Pattern)*

Quy trình vận hành chi tiết của mẫu thiết kế bao gồm:

- **Phân tích (*Analyze*):** Giải mã yêu cầu, nhận diện phạm vi tri thức, các ràng buộc học thuật và tiêu chí đầu ra mong đợi.
- **Phân rã (*Decompose*):** Tách nhiệm vụ thành các bài toán con nguyên tử (*atomic subtasks*) độc lập hoặc có thể kiểm thử riêng biệt.
- **Sắp đặt thứ tự (*Sequence*):** Xác định đồ thị quan hệ phụ thuộc giữa các bài toán con, bảo đảm dữ liệu đầu ra của tác vụ trước cung cấp chính xác đầu vào

cho tác vụ sau.

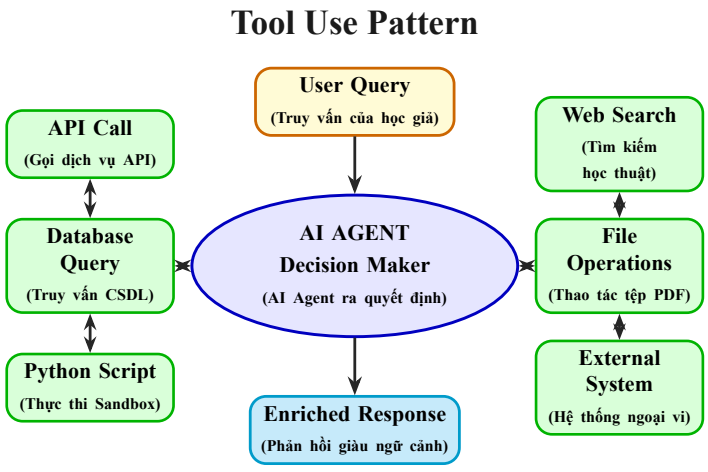
- **Thực thi kế hoạch (*Execute Plan*):** Điều phối thực thi tuần tự qua từng bước (*Subtask 1* → *Subtask 2* → *Subtask 3*) để tổng hợp nên kết quả nghiên cứu toàn diện (*Complete Result*).

Trong khuôn khổ cài đặt thực tế của đề tài VeriScholar, tư duy lập kế hoạch được tích hợp trực tiếp vào pha suy luận (*Thought*) của tác tử đơn lẻ theo chu trình ReAct kết hợp gọi công cụ (*Tool Use*), giúp phân rã truy vấn học thuật phức tạp mà không phát sinh độ trễ của một tầng lập kế hoạch tách rời.

2.5.3 Mẫu kiến trúc Sử dụng Công cụ (Tool Use Pattern)

Mặc dù sở hữu năng lực ngôn ngữ mạnh mẽ, các mô hình ngôn ngữ lớn (LLM) thuần túy hoàn toàn bị giới hạn bởi: (1) tri thức tĩnh không cập nhật dữ liệu mới phát sinh, (2) không có khả năng thực hiện các phép tính số học phức tạp chính xác, và (3) không thể trực tiếp thao tác với hệ thống tệp tin hoặc cơ sở dữ liệu. Mẫu kiến trúc Sử dụng Công cụ (*Tool Use Pattern*) [14] trao cho LLM vai trò của một "Bộ não ra quyết định" (*Decision Maker*) kết nối với hệ sinh thái công cụ thực thi chuyên biệt.

Mô hình tương tác giữa AI Agent và tập công cụ ngoại vi được thể hiện trong Hình 2-11.



Hình 2-11: *Mẫu kiến trúc Agent sử dụng công cụ ngoại vi (Tool Use Pattern)*

Trong mẫu kiến trúc này, AI Agent vận hành như một thực thể điều phối trung tâm:

- Khi nhận truy vấn (*User Query*), Agent phân tích xem câu trả lời có đòi hỏi dữ liệu chính xác hay tính toán đặc thù hay không.
- Agent phát sinh lệnh gọi hàm chuẩn hóa (*Function Calling*) gửi tới các công

cụ phù hợp: nhánh dữ liệu (*Database Query, API Call, Python Script*) hoặc nhánh tệp và tìm kiếm (*File Operations, Web Search, External System*).

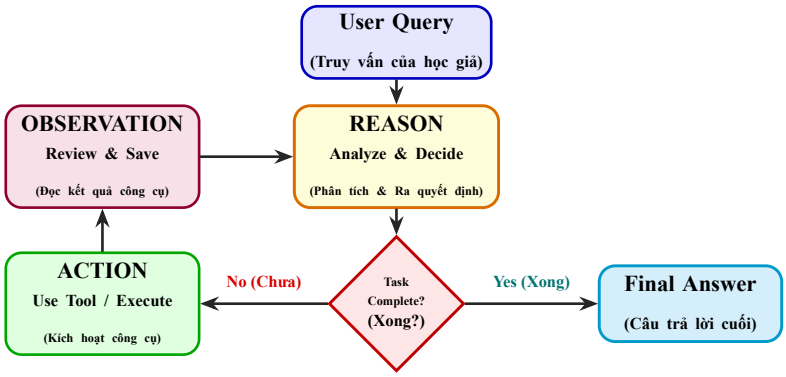
- Dữ liệu thực nghiệm thu được từ môi trường ngoại vi được tích hợp ngược trở lại ngữ cảnh để LLM sinh ra phản hồi chuẩn xác, giàu dẫn chứng (*Enriched Response*).

2.5.4 Mẫu kiến trúc Suy luận và Hành động ReAct (ReAct Pattern)

ReAct (*Synergizing Reasoning and Acting in Language Models*) là mẫu kiến trúc Agent kinh điển do Shunyu Yao và các cộng sự tại Đại học Princeton và Google Research công bố vào tháng 10 năm 2022 [19]. Trong tiến trình phát triển các mô hình ngôn ngữ lớn có khả năng giải quyết bài toán đa bước, các hướng tiếp cận đơn lẻ thường tách biệt giữa suy luận chuỗi tư duy nội tại (*Chain-of-Thought*, công bố đầu năm 2022) [19] và cơ chế tự học gọi hàm API ngoại vi độc lập (tiêu biểu là *Toolformer*, công bố tháng 2 năm 2023) [14]. ReAct đặt nền móng hợp nhất đồng thời cả hai năng lực suy luận và hành động theo một vòng lặp đan xen nhịp nhàng:

$$\text{Thought} \longrightarrow \text{Action} \longrightarrow \text{Observation} \longrightarrow \text{Thought} \ldots \tag{2.4}$$

Mô hình chu trình suy luận và hành động ReAct được thể hiện trong Hình 2-12.



Hình 2-12: *Mẫu kiến trúc suy luận và hành động ReAct (Reasoning and Acting Pattern)*

Mỗi chu trình ReAct bao gồm ba pha nối tiếp nhau:

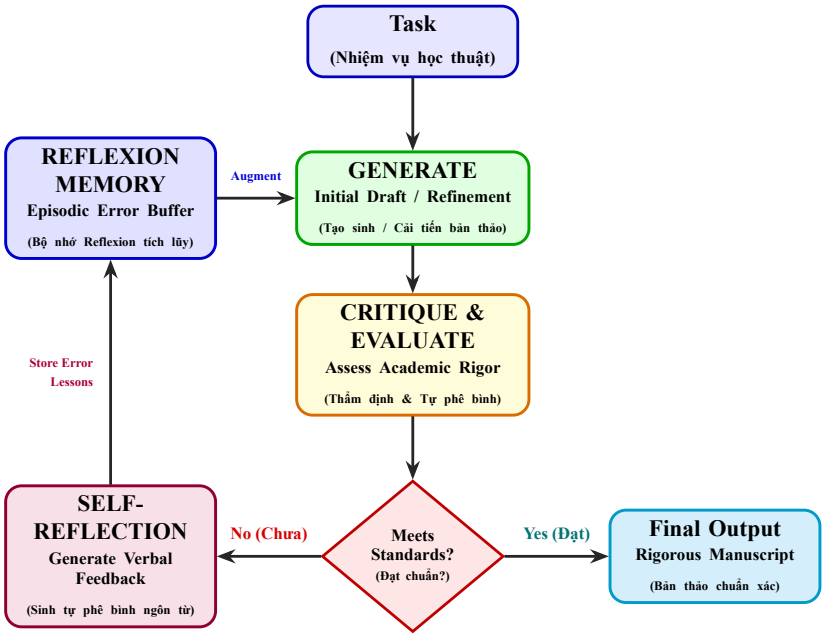
- **Suy nghĩ (*Thought / Reason*):** Agent tự sinh ra lập luận bằng văn bản để phân tích tình hình hiện tại, nhận diện khoảng trống dữ liệu và lựa chọn công cụ cần gọi tiếp theo.
- **Hành động (*Action*):** Agent phát ra lệnh có cấu trúc cụ thể (gọi hàm tra cứu tài liệu, tính toán dữ liệu hoặc truy vấn đồ thị).

- **Quan sát (*Observation*):** Hệ thống thực thi công cụ và trả về dữ liệu thực tế. Agent đọc kết quả quan sát này để tiếp tục chu trình tư duy kế tiếp cho đến khi hoàn thành mục tiêu.

2.5.5 Cơ chế Reflection và Tự sửa sai Reflexion (Reflection Pattern)

Reflexion (*Language Agents with Verbal Reinforcement Learning*) là một khuôn khổ mở rộng được phát triển bởi Noah Shinn và các đồng nghiệp [15], cung cấp cho AI Agent khả năng tự đánh giá học hỏi từ những sai lầm mà không đòi hỏi phải cập nhật trọng số mô hình.

Trong môi trường nghiên cứu học thuật, rủi ro trích dẫn sai số trang, nhầm lẫn tác giả hoặc ảo giác dữ liệu thực nghiệm là tối kỵ. Khung chu trình tự đánh giá và hiệu chỉnh của Reflexion được thể hiện trong Hình 2-13.



Hình 2-13: *Cơ chế Reflection và tự sửa sai Reflexion (Reflection Pattern)*

Cơ chế Reflexion vận hành thông qua ba thành phần phối hợp chặt chẽ:

(1) Bộ đánh giá (*Evaluator*): Chấm điểm kết quả đầu ra của Agent dựa trên tiêu chí liêm chính học thuật (ví dụ: độ phủ trích dẫn có trùng khớp 100% với văn bản gốc hay không).

(2) Bộ phản hồi ngôn từ (*Self-Reflection*): Nếu bộ đánh giá phát hiện lỗi sai, mô hình sẽ tự sinh ra một đoạn tự phê bình bằng lời văn (*verbal critique*), phân tích nguyên nhân vì sao bước lập luận trước bị lỗi và đề xuất hướng sửa đổi cụ thể.

(3) Bộ nhớ Reflexion (*Memory Buffer*): Lưu trữ bản tự phê bình vào ngữ cảnh

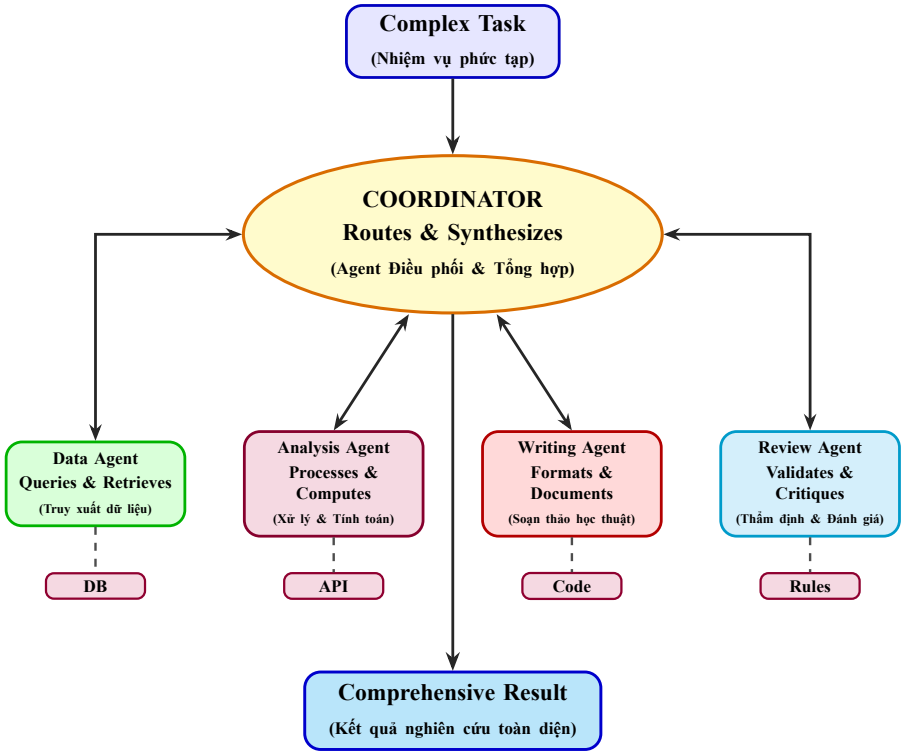
hoạt động. Ở lượt thực thi tiếp theo, Agent đọc lại kinh nghiệm sai sót này để không lặp lại sai lầm cũ, bảo đảm bản thảo học thuật đạt độ chính xác tối đa.

Trong kiến trúc hiện tại của VeriScholar, thay vì kích hoạt một vòng lặp LLM tự phê bình đa lượt tốn kém chi phí mã thông báo, nguyên lý kiểm định đầu ra được hiện thực hóa bằng các rào chắn xác định (*Deterministic Guardrails*) kết hợp kiểm chứng độ trùng khớp trích dẫn (*Citation Grounding Verification*) và cơ chế tự phục hồi truy vấn ngay trong vòng lặp ReAct đơn tác tử.

2.5.6 Mẫu kiến trúc Phối hợp Đa Agent (Multi-Agent Pattern)

Khi độ phức tạp của bài toán vượt quá năng lực xử lý của một Agent đơn lẻ trong một cửa sổ ngữ cảnh, mô hình Phối hợp Đa Agent (*Multi-Agent Collaboration Pattern*) trở thành giải pháp kiến trúc tất yếu. Nguyên lý thiết kế của mẫu này dựa trên sự chuyên môn hóa vai trò (*Role Specialization*) và phân chia trách nhiệm rõ ràng (*Separation of Concerns*), tương tự như một hội đồng chuyên gia khoa học đảm nhiệm từng khâu riêng biệt.

Mô hình kiến trúc đa Agent điều phối phân cấp được thể hiện trong Hình 2-14.



Hình 2-14: *Mẫu kiến trúc phối hợp đa Agent chuyên trách (Multi-Agent Collaboration Pattern)*

Cấu trúc phối hợp đa Agent gồm hai tầng nhận thức:

- **Agent Điều phối (*Coordinator*):** Tiếp nhận nhiệm vụ lớn, lập kế hoạch phân

bổ, định tuyến yêu cầu tới các Agent chuyên trách và chịu trách nhiệm tổng hợp các kết quả riêng lẻ thành báo cáo khoa học hoàn chỉnh.

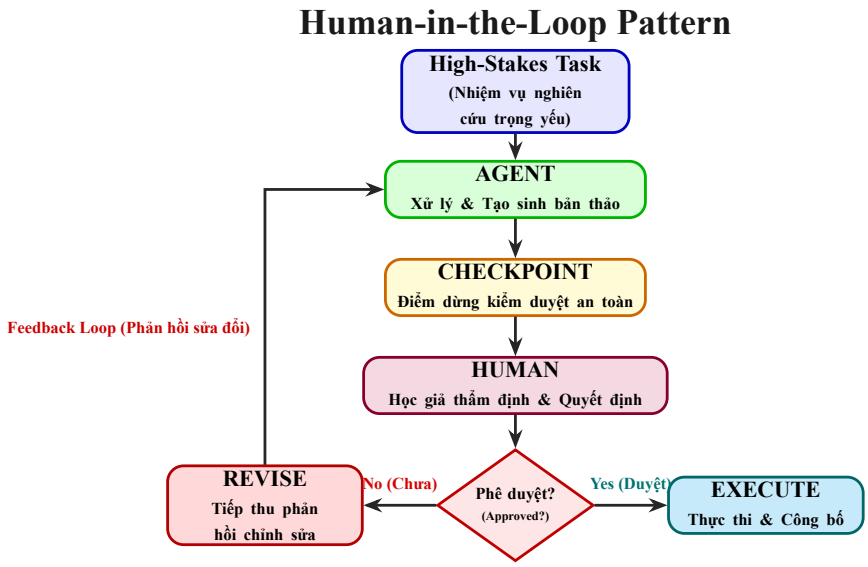
- **Các Agent chuyên trách (*Specialized Agents*):** Mỗi Agent được tinh chỉnh hệ thống chỉ dẫn (*system prompt*) và cấp quyền truy cập tập công cụ riêng biệt: *Data Agent* kết nối CSDL tri thức, *Analysis Agent* xử lý dữ liệu và thuật toán, *Writing Agent* định dạng văn bản học thuật theo chuẩn quốc tế, và *Review Agent* đối chiếu quy chế và tiêu chí liêm chính.

Trong phạm vi thực nghiệm hiện tại, VeriScholar triển khai kiến trúc tác tử đơn điều phối trung tâm (*Single-Agent ReAct Orchestrator*) kết hợp tập công cụ chuyên biệt (*Tool Use*) nhằm tối ưu hóa độ trễ truyền phát thời gian thực; mô hình phối hợp đa tác tử phân cấp đóng vai trò là nền tảng lý thuyết định hướng cho giai đoạn mở rộng tiếp theo của hệ thống.

2.5.7 Mẫu tương tác Người trong vòng lặp (Human-in-the-Loop Pattern)

Trong các hệ sinh thái nghiên cứu khoa học chuyên sâu, việc trao toàn quyền tự quyết không kiểm soát cho các AI Agent tiềm ẩn nhiều rủi ro học thuật (như trích dẫn sai sự thật, diễn giải sai lệch số liệu thực nghiệm hoặc kích hoạt các mã lệnh không an toàn). Mẫu tương tác Người trong vòng lặp (*Human-in-the-Loop – HITL Pattern*) thiết lập các điểm kiểm soát an toàn (*Checkpoints*) nhằm bảo đảm các quyết định trọng yếu luôn có sự giám sát và phê duyệt của học giả con người.

Cơ chế kiểm soát và phản hồi tương tác HITL được mô hình hóa trong Hình 2-15.



Hình 2-15: *Mẫu tương tác người trong vòng lặp (Human-in-the-Loop Pattern)*

Cơ chế vận hành của mô hình HITL bao gồm:

- **Nhận diện tác vụ trọng yếu (*High-Stakes Task*):** Hệ thống tự động xác định các hành vi có nguy cơ ảnh hưởng lớn (thay đổi trạng thái đồ thị tri thức, xuất bản bài tổng quan, hoặc thực thi mã lệnh Python trong Sandbox).
- **Điểm dừng an toàn (*Checkpoint*):** Agent tạm dừng quy trình tự động, khóa trạng thái luồng làm việc và gửi thông báo kèm ngữ cảnh tóm tắt tới người dùng.
- **Đánh giá và Quyết định (*Review & Decide*):** Học giả xem xét bản thảo hoặc mã lệnh. Nếu được chấp thuận (*Approved = Yes*), hệ thống tiến hành thực thi (*Execute*). Nếu bị từ chối (*Approved = No*), học giả cung cấp chỉ dẫn sửa đổi; Agent tiếp thu phản hồi (*Revise*) để tái tạo sinh kết quả cho đến khi đạt chuẩn.

Trên hệ thống VeriScholar hiện hành, triết lý kiểm chứng bởi con người được thực hiện thông qua cơ chế đối chiếu trực quan tọa độ trích dẫn (*Visual Bounding-Box Verification*) trên trình đọc PDF hai khung nhìn và kiểm soát giới hạn thực thi tất định; cơ chế điểm dừng tạm khóa luồng thực thi (*Interrupt Checkpoints*) là hướng mở rộng tương tác nâng cao trong tương lai.

2.6 Tổng quan về LangGraph

2.6.1 LangGraph là gì?

LangGraph là một thư viện điều phối Agent trí tuệ nhân tạo thế hệ mới được phát triển bởi tổ chức LangChain, chuyên biệt cho việc thiết kế các hệ thống Agent phức tạp, có trạng thái (*stateful*) và hỗ trợ cấu trúc đồ thị có chu trình lặp (*cyclic graphs*).



Hình 2-16: *LangGraph*

Trước khi LangGraph ra đời, phần lớn các khung phát triển ứng dụng LLM chỉ hỗ trợ các đường ống xử lý theo đồ thị có hướng không chu trình tuyến tính (*Directed Acyclic Graph – DAG*), trong đó dữ liệu chỉ chảy một chiều từ nút bắt đầu đến nút kết thúc ($A \rightarrow B \rightarrow C$). Tuy nhiên, các bài toán học thuật phức tạp đòi hỏi Agent phải có khả năng:

- Thực hiện các vòng lặp Reflexion: Nếu bước kiểm tra trích dẫn phát hiện dữ liệu không chính xác, hệ thống phải quay lui lại bước truy xuất để tìm kiếm tài liệu mới.
- Hợp tác đa Agent (*Multi-Agent Collaboration*): Phân chia trách nhiệm cho nhiều Agent chuyên môn hóa (Agent lập kế hoạch, Agent tra cứu, Agent phản biện) cùng trao đổi và hội tụ về một kết quả tối ưu.
- Lưu trữ trạng thái bền vững (*Persistence & Checkpointing*): Ghi nhớ toàn bộ cây trạng thái sau mỗi bước thực thi, cho phép tạm dừng, khôi phục hoặc phát lại luồng suy luận tại bất kỳ thời điểm nào.

LangGraph cung cấp nền tảng chuẩn mực giải quyết hoàn hảo các yêu cầu trên, biến quy trình suy luận của AI thành một máy trạng thái hữu hạn (*Finite State Machine*) có thể kiểm soát và đo lường chặt chẽ.

2.6.2 Kiến trúc StateGraph điều phối Agent

Cốt lõi kiến trúc của LangGraph được xây dựng xung quanh khái niệm StateGraph – một đồ thị toán học trong đó mọi nút đều đọc và ghi dữ liệu vào một lược đồ trạng thái chia sẻ chung (*Shared State Schema*).

Một đồ thị StateGraph bao gồm ba thành phần cấu thành cơ bản:

(1) Tập các nút (*Nodes*): Đại diện cho các đơn vị tính toán độc lập. Mỗi nút nhận trạng thái hiện tại của hệ thống, thực hiện một tác vụ cụ thể (chẳng hạn như phân tích câu hỏi của học giả, gọi công cụ tìm kiếm lai, hoặc thẩm định trích dẫn) và trả về dữ liệu cập nhật.

(2) Lược đồ trạng thái và hàm thu gọn (*State Schema & Reducers*): Định nghĩa cấu trúc dữ liệu tổng thể của phiên làm việc. Hàm thu gọn quy định cách thức tích hợp dữ liệu mới từ các nút vào trạng thái chung (ví dụ: nối thêm thông điệp mới vào lịch sử hội thoại thay vì ghi đè).

(3) Tập các cạnh điều hướng (*Edges*): Xác định quy tắc chuyển dịch giữa các nút:

- *Cạnh cố định (Normal Edges):* Luôn chuyển tiếp vô điều kiện từ nút này sang nút khác sau khi tác vụ hoàn tất.
- *Cạnh rẽ nhánh có điều kiện (Conditional Edges):* Vận hành dựa trên một hàm đánh giá logic để quyết định nút tiếp theo. Ví dụ: sau nút Thẩm

định trích dẫn, nếu độ trùng khớp đạt yêu cầu thì chuyển đến nút Kết thúc (END), nếu chưa đạt thì rẽ nhánh quay lại nút Mở rộng truy vấn.

Bên cạnh đó, kiến trúc StateGraph tích hợp sẵn cơ chế can thiệp của con người vào quy trình (*Human-in-the-Loop*), cho phép Agent dừng lại xin ý kiến của học giả trước khi thực thi các quyết định quan trọng (như xác nhận nội dung bản thảo hoặc tải tài liệu lớn), tạo nên sự phối hợp tin cậy giữa con người và trí tuệ nhân tạo.

2.7 Tìm hiểu về Azure OpenAI và E2B Sandbox

2.7.1 Dịch vụ suy luận OpenAI API và Hạ tầng đám mây Microsoft Azure

Dịch vụ suy luận OpenAI API và nền tảng đám mây Microsoft Azure (*Azure OpenAI Service*) cung cấp giao diện lập trình tiêu chuẩn công nghiệp để truy cập các mô hình ngôn ngữ lớn và mô hình biểu diễn vector ngữ nghĩa hiện đại (điển hình là mô hình suy luận gpt-4o-mini và mô hình nhúng text-embedding-3-small).



Hình 2-17: *Microsoft Azure*

Khác biệt căn bản giữa dịch vụ API cấp doanh nghiệp trên hạ tầng đám mây và các giao diện trò chuyện tiêu dùng công cộng nằm ở các tiêu chuẩn bảo mật và vận hành khắt khe:

- **Cam kết bảo vệ dữ liệu học thuật tuyệt đối:** Dữ liệu văn bản, tài liệu nghiên cứu và câu hỏi của học giả truyền qua cổng API không được sử dụng để tái huấn luyện hoặc cải tiến bất kỳ mô hình cơ sở nào của nhà cung cấp.
- **Hạ tầng mạng cô lập cấp doanh nghiệp:** Hỗ trợ triển khai trên mạng riêng ảo (*Virtual Network – VNet*), các điểm cuối riêng tư (*Private Endpoints*) và mã hóa đường truyền nghiêm ngặt theo chuẩn TLS 1.3, đáp ứng các tiêu chuẩn chứng nhận quốc tế như ISO/IEC 27001, SOC 2 Type II và GDPR.
- **Độ sẵn sàng và độ ổn định cao (High Availability SLA):** Cung cấp cơ chế điều phối hạn ngạch thông lượng ổn định (*Provisioned Throughput Units –*

PTU), bảo đảm thời gian phản hồi đồng đều và giảm thiểu nguy cơ nghẽn dịch vụ vào giờ cao điểm.

2.7.2 E2B Sandbox là gì?

E2B Sandbox là nền tảng môi trường thực thi ảo hóa cô lập chuyên biệt cho các AI Agent, cho phép mô hình ngôn ngữ thực thi mã nguồn tự sinh trong một không gian an toàn.



Hình 2-18: *E2B Sandbox*

Công nghệ nền tảng của E2B dựa trên kiến trúc máy ảo siêu nhỏ (*MicroVM*) được phát triển từ công nghệ ảo hóa AWS Firecracker [1]:

(1) Khởi động siêu tốc: Mỗi môi trường MicroVM có thể khởi tạo hoàn tất trong thời gian chưa đầy 200 mili-giây, cung cấp đầy đủ một nhân Linux hoàn chỉnh với hệ điều hành và tài nguyên tính toán độc lập.

(2) Cô lập ranh giới hạt nhân (Kernel-level Isolation): Mọi đoạn mã lập trình (chẳng hạn như kịch bản phân tích số liệu Python, trích xuất biểu đồ hoặc lệnh biên dịch tài liệu LaTeX) đều chạy bên trong ranh giới máy ảo riêng. Nếu kịch bản chứa mã độc hoặc gặp sự cố tràn bộ nhớ, sự cố hoàn toàn bị giới hạn bên trong MicroVM mà không thể lan truyền hay gây hại đến máy chủ chính của hệ thống, ngăn chặn nguy cơ tấn công thực thi mã độc từ xa (*Remote Code Execution – RCE*).

(3) Môi trường trạng thái linh hoạt: Hỗ trợ tương tác hai chiều, truyền tệp nhị phân và thu nhận kết quả tính toán tức thời để đưa ngược lại vào chu trình suy luận của Agent.

2.7.3 Mô hình bảo mật BYOK (Bring Your Own Key)

Mô hình Mang theo khóa cá nhân (*Bring Your Own Key – BYOK*) là giải pháp kiến trúc bảo đảm quyền tự chủ tài nguyên và an toàn riêng tư cho các nhà khoa học. Trong mô hình này, học giả cung cấp khóa API cá nhân từ nhà cung cấp dịch vụ trí

tuệ nhân tạo để trực tiếp thanh toán và kiểm soát hạn ngạch sử dụng; máy chủ của hệ thống không đứng ra thu phí trung gian hay lưu trữ khóa dưới dạng văn bản thô trên đĩa.

Để bảo vệ khóa API của học giả trước nguy cơ lộ lọt dữ liệu từ tầng lưu trữ, đề tài triển khai chuẩn mã hóa đối xứng xác thực AEAD (*Authenticated Encryption with Associated Data*) với thuật toán AES-256 theo chế độ GCM (*Galois/Counter Mode*) [6]:

Thuật toán sử dụng khóa bí mật 256 bits (K), khóa con băm trường hữu hạn $H = E_K(0^{128})$ và vectơ khởi tạo ngẫu nhiên 96 bits (IV) riêng biệt cho mỗi lần mã hóa. Từ IV 96 bits, khối bộ đếm khởi đầu 128 bits J_0 được xác lập theo đặc tả NIST SP 800-38D là $J_0 = \text{IV} \parallel 0^{31} \parallel 1$. Bản rõ khóa API được mã hóa bằng chế độ bộ đếm GCTR bắt đầu từ $\text{inc}_{32}(J_0)$, đồng thời hàm băm vạn năng GHASH_H kết hợp dữ liệu xác thực bổ sung (AAD) và bản mã (C) rồi thực hiện phép XOR với khối mã hóa $E_K(J_0)$ để tạo ra thẻ xác thực toàn vẹn 128 bits (T):

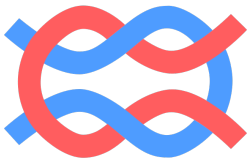
$$\begin{aligned} C &= \text{GCTR}_K\big(\text{inc}_{32}(J_0), \text{Plaintext}\big), \quad J_0 = \text{IV} \parallel 0^{31} \parallel 1 \\ T &= \text{GHASH}_H(\text{AAD}, C) \oplus E_K(J_0) \end{aligned} \tag{2.5}$$

Quy trình giải mã tuân thủ nguyên tắc giải mã tạm thời trong bộ nhớ khả biến (*In-Memory Ephemeral Decryption*): Khóa API bản rõ chỉ được khôi phục cục bộ trong không gian bộ nhớ RAM của tiến trình xử lý ngay tại thời điểm khởi tạo yêu cầu gọi mô hình ngôn ngữ, bị giải phóng ngay sau khi kết thúc phiên suy luận và tuyệt đối không bao giờ được ghi vào nhật ký hệ thống hay các bản ghi giám sát truy vết quan sát (*Observability Traces*).

2.8 Tổng quan về Langfuse

2.8.1 Langfuse là gì? (LLM Tracing & Observability)

Langfuse là nền tảng kỹ nghệ mã nguồn mở hàng đầu thế giới chuyên phục vụ việc giám sát, quan sát vận hành (*LLM Observability*) và truy vết chi tiết (*Tracing*) dành riêng cho các ứng dụng sử dụng mô hình ngôn ngữ lớn và hệ thống AI Agent đa bước.



Hình 2-19: *Langfuse*

Trong các phần mềm truyền thống, việc giám sát thường chỉ dừng lại ở nhật ký ghi lỗi (*Error Logs*) hoặc các chỉ số máy chủ cơ bản (CPU, RAM). Tuy nhiên, đối với hệ thống AI Agent phức tạp thực hiện hàng loạt bước suy luận, tìm kiếm và tự sửa sai, lập trình viên không thể nắm bắt được nguyên nhân dẫn đến một câu trả lời không chính xác nếu không có công cụ giám sát chuyên dụng.

Langfuse giải quyết thách thức này thông qua mô hình cây truy vết phân tầng (*Hierarchical Tracing*):

- **Dấu vết tổng thể (*Trace*):** Đại diện cho toàn bộ một chu trình xử lý của Agent từ thời điểm học giả đặt câu hỏi đến khi bản thảo câu trả lời được hoàn tất.
- **Khoảng đoạn thực thi (*Spans*):** Đại diện cho từng hành động nghiệp vụ cụ thể bên trong chu trình, chẳng hạn như thời gian trích xuất văn bản từ tệp PDF, độ trễ truy vấn vector HNSW trên PostgreSQL, hoặc bước tra cứu đồ thị tri thức.
- **Lượt tạo sinh LLM (*Generations*):** Ghi lại siêu dữ liệu tương tác với mô hình AI, bao gồm tên mô hình, nội dung câu nhắc (*Prompt*), kết quả phản hồi, số lượng mã thông báo (*Tokens*) tiêu thụ và độ trễ mạng. Đặc biệt, cơ chế truy vết của Langfuse được tách biệt hoàn toàn với luồng giải mã khóa cá nhân BYOK tạm thời trong bộ nhớ RAM: các thông tin nhạy cảm như khóa API của học giả hay chuỗi xác thực đều bị loại bỏ hoàn toàn trước khi tạo bản ghi truy vết, bảo đảm tính minh bạch vận hành mà không làm lộ lọt bí mật xác thực.

Nhờ cấu trúc truy vết này, mọi quyết định suy luận của Agent đều trở nên minh bạch và có thể kiểm toán toàn diện.

2.8.2 Cơ chế theo dõi chi phí và độ trễ LLM

Việc kiểm soát ngân sách và tối ưu hóa tốc độ phản hồi là yếu tố sống còn khi triển khai các hệ thống AI học thuật vào thực tế. Langfuse cung cấp cơ chế giám sát thời

gian thực với độ chính xác cao:

(1) **Phân rã mã thông báo và hạch toán chi phí tài chính:** Hệ thống tự động phân loại chính xác số lượng token đầu vào (*Input/Prompt Tokens*), token đầu ra (*Output/Completion Tokens*), và token tận dụng từ bộ nhớ đệm (*Cached Tokens*). Bằng cách kết nối với bảng giá chính thức của nhà cung cấp dịch vụ đám mây, Langfuse tính toán chi phí tài chính tức thì cho từng truy vấn, giúp học giả và quản trị viên theo dõi ngân sách minh bạch.

(2) **Phân tích độ trễ và thời gian sinh ký tự đầu tiên (TTFT):** Langfuse phân rã chi tiết thời gian phản hồi thành hai chỉ số chính:

- *Thời gian đến ký tự đầu tiên (Time-to-First-Token – TTFT):* Đo lường khoảng thời gian từ lúc người dùng gửi yêu cầu đến khi ký tự đầu tiên được truyền phát về giao diện qua luồng SSE. TTFT thấp mang lại trải nghiệm tương tác tức thì và mượt mà cho học giả.
- *Thời gian thực thi của từng công cụ ngoại vi:* Giúp phát hiện chính xác điểm nghẽn hiệu năng trong chuỗi Agent (ví dụ: bước tìm kiếm từ khóa BM25 bị chậm hay do mô hình ngôn ngữ phản hồi trễ).

(3) **Bộ chỉ số đánh giá độ tin cậy học thuật:** Tích hợp các bộ tiêu chí chấm điểm tự động đo lường độ trung thực của dẫn chứng (*Faithfulness*) và độ bám sát câu hỏi (*Answer Relevance*), cung cấp bằng chứng định lượng để liên tục cải tiến hệ thống.

2.9 Các công cụ và ngôn ngữ phát triển hệ thống

2.9.1 Python và FastAPI

Để hiện thực hóa tầng logic nghiệp vụ và các thuật toán AI Agent, đề tài lựa chọn ngôn ngữ lập trình Python kết hợp với web framework FastAPI:



Hình 2-20: *Python*



Hình 2-21: *FastAPI*

- **Ngôn ngữ lập trình Python:** Python là ngôn ngữ lập trình tiêu chuẩn thực tế (*de facto standard*) của cộng đồng nghiên cứu trí tuệ nhân tạo và khoa học

dữ liệu toàn cầu. Python cung cấp hệ thống thư viện tính toán khoa học vượt trội, khả năng tích hợp bản địa với các mô hình học sâu, cùng cú pháp gợi ý kiểu dữ liệu tĩnh chặt chẽ (*Type Hinting*) và cơ chế lập trình bất đồng bộ (*async/await*) giúp tối ưu hóa hiệu năng xử lý tác vụ I/O.

- **FastAPI Framework:** FastAPI là một trong những khung phát triển web bằng Python có tốc độ thực thi nhanh nhất hiện nay [12], được xây dựng trên nền tảng máy chủ bất đồng bộ ASGI (*Starlette*) và thư viện xác thực dữ liệu pydantic. FastAPI tự động thẩm định dữ liệu đầu vào, tự sinh tài liệu giao diện lập trình OpenAPI chuẩn quốc tế, và hỗ trợ xuất sắc giao thức truyền phát luồng sự kiện máy chủ (*Server-Sent Events – SSE*) phục vụ việc hiển thị câu trả lời AI theo thời gian thực cho học giả.

2.9.2 TypeScript và React

Ở tầng giao diện người dùng, hệ thống sử dụng ngôn ngữ TypeScript kết hợp cùng thư viện React để tạo nên trải nghiệm nghiên cứu tương tác trực quan và hiện đại:



Hình 2-22: *React*

- **Ngôn ngữ TypeScript:** TypeScript là phần mở rộng có kiểu tĩnh chặt chẽ của JavaScript do Microsoft phát triển. Việc áp dụng TypeScript giúp hệ thống phát hiện sớm các xung đột kiểu dữ liệu ngay từ giai đoạn biên dịch, tăng cường độ tin cậy và khả năng tái cấu trúc của các mô hình dữ liệu phức tạp (như tọa độ vùng bao chữ nhật trên trang PDF, siêu dữ liệu bài báo và trạng thái luồng SSE).

- **Thư viện React:** React là thư viện phát triển giao diện người dùng khai báo (*Declarative UI*) dựa trên kiến trúc thành phần tái sử dụng (*Component-Based Architecture*). Cơ chế Document Object Model ảo (*Virtual DOM*) của React giúp tối ưu hóa việc kết xuất đồ họa, duy trì tốc độ khung hình mượt mà ngay cả khi người dùng đồng thời cuộn văn bản PDF nhiều trang, hiển thị khung viền dẫn chứng và trò chuyện trực tiếp cùng AI Agent.

2.9.3 Docker, Pytest và Vitest

Quy trình phát triển và kiểm thử chất lượng phần mềm được chuẩn hóa thông qua bộ công cụ hiện đại:



Hình 2-23: *Docker*

- **Docker:** Nền tảng ảo hóa cấp hệ điều hành (*Containerization*) cho phép đóng gói toàn bộ ứng dụng, thư viện phụ thuộc và môi trường chạy vào các vùng chứa biệt lập (*Containers*). Nhờ Docker, hệ thống bảo đảm tính nhất quán hoạt động tuyệt đối giữa máy phát triển của sinh viên và máy chủ triển khai thực tế của nhà trường.
- **Khung kiểm thử Pytest:** Công cụ kiểm định chất lượng mã nguồn backend tiêu chuẩn của Python, hỗ trợ cơ chế tải bối cảnh dùng chung (*Fixtures*), tự động dọn dẹp dữ liệu kiểm thử và kiểm tra tính toàn vẹn của các ranh giới kiến trúc Lục giác.
- **Khung kiểm thử Vitest:** Bộ công cụ kiểm thử đơn vị siêu tốc dành cho giao diện TypeScript/React, cho phép mô phỏng hành vi tương tác của người dùng, xác nhận tính chính xác của các thành phần đồ họa trước khi xuất xưởng phần mềm.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1 Mô tả bài toán

Trong hoạt động nghiên cứu khoa học tại các trường đại học và viện nghiên cứu hàng đầu nói chung, cũng như tại VinUniversity nói riêng, quá trình sản xuất các công bố học thuật quốc tế chất lượng cao (các hội nghị chuyên ngành danh giá như NeurIPS, ICML, CVPR và các tạp chí uy tín thuộc danh mục Q1/Q2) đòi hỏi học giả phải xử lý khối lượng tri thức khổng lồ với độ chính xác và tính liêm chính học thuật (*Academic Integrity*) tuyệt đối.

Trước đây, khi tiến hành một công trình nghiên cứu mới, học giả, giảng viên và sinh viên thường phải thực hiện thủ công toàn bộ các công đoạn nghiên cứu thông qua nhiều công cụ đơn lẻ, rời rạc:

(1) Tìm kiếm và thu thập tài liệu học thuật: Học giả phải chủ động truy cập độc lập vào từng cơ sở dữ liệu hoặc cổng lưu trữ như Google Scholar, arXiv, DBLP, ACM Digital Library hay IEEE Xplore. Việc tải các tệp PDF tài liệu về máy tính cá nhân diễn ra phân tán, dễ dẫn đến tình trạng lưu trữ trùng lặp các tệp dung lượng lớn, gây lãng phí tài nguyên và khó khăn trong việc quản trị phiên bản hay đồng bộ thư viện nghiên cứu.

(2) Đọc hiểu và trích xuất dữ liệu thực nghiệm: Khi mở tệp PDF trên các trình đọc tài liệu truyền thống hoặc sử dụng các công cụ trò chuyện với tài liệu phổ biến, các công cụ này thường chỉ trích xuất văn bản thô theo dòng quét một cột, làm xáo trộn hoàn toàn cấu trúc hai cột (*two-column format*) đặc thù của các bài báo khoa học. Đáng lo ngại hơn, các mô hình ngôn ngữ lớn phổ quát hiện nay thường xuyên mắc phải hiện tượng ảo giác trích dẫn (*Citation Hallucination*): tự bịa đặt các số liệu đo lường không có thật, hoặc chỉ trích dẫn số trang chung chung. Học giả phải mất hàng giờ đồng hồ rà soát thủ công từng dòng chữ, bảng biểu trên tài liệu gốc để kiểm chứng tính xác thực của thông tin.

(3) Tổng hợp mạng lưới phương pháp và bộ dữ liệu: Để có cái nhìn toàn cảnh về một lĩnh vực (State-of-the-Art), nhà nghiên cứu phải tự lập các bảng tính Excel hoặc ghi chú Notion rời rạc nhằm theo dõi xem phương pháp nào đã được thử nghiệm trên bộ dữ liệu nào và đạt kết quả ra sao. Cách làm thủ công

này rất dễ bỏ sót các mối liên hệ bắc cầu giữa các công trình nghiên cứu và không hỗ trợ truy vấn so sánh liên tài liệu.

(4) Soạn thảo bản thảo khoa học bằng LaTeX: Học giả phải chuyển ngữ cảnh sang các trình soạn thảo như TeXstudio hay Overleaf. Khi biên dịch thất bại, học giả phải tự lần mò qua các tệp nhật ký lỗi (.log) dài hàng nghìn dòng để phát hiện và sửa các lỗi cú pháp vi mô (thiếu thẻ đóng môi trường toán học, ký tự đặc biệt chưa thoát mã, hoặc nhãn trích dẫn bị xung đột).

Sự chia cắt giữa các công cụ đơn lẻ khiến người nghiên cứu phải tiêu tốn một tỷ trọng thời gian đáng kể (ước tính khoảng 30% đến 40% quỹ thời gian thao tác kỹ thuật trong quy trình tổng quan tài liệu và chuẩn bị bản thảo) cho việc chuyển đổi ngữ cảnh, định dạng lại văn bản và đối chiếu nguồn trích dẫn thủ công, thay vì tập trung vào tư duy sáng tạo khoa học cốt lõi.

Do đó, việc nghiên cứu và xây dựng nền tảng trợ lý AI nghiên cứu khoa học toàn trình VeriScholar là hết sức cấp thiết. Nền tảng được thiết kế nhằm giải quyết triệt để bài toán ảo giác trích dẫn thông qua kiến trúc Agentic RAG lai (Hybrid Search RRF), tự động định vị khung viền tọa độ Bounding Box minh chứng trực tiếp trên bản PDF gốc, kết nối mạng tri thức học thuật 4 thực thể (GraphRAG), và tích hợp AI Agent tự phục hồi lỗi biên dịch LaTeX trong môi trường máy ảo cách ly an toàn.

3.2 Khảo sát nghiệp vụ và quy trình xử lý tài liệu nghiên cứu

Để xây dựng nền tảng trợ lý AI hỗ trợ nghiên cứu khoa học sát với nhu cầu thực tiễn của học giả, đề tài tiến hành phân tích quy trình làm việc học thuật quốc tế (*Academic Research Workflow*) cùng thực trạng vận hành của các công cụ quản lý tài liệu và trợ lý AI hiện hành. Việc phân tích tập trung vào chu trình nghiên cứu tiêu chuẩn của các nhà khoa học, giảng viên, nghiên cứu sinh và sinh viên làm đồ án tốt nghiệp tại các đại học định hướng nghiên cứu (như VinUniversity), qua đó làm rõ các điểm nghẽn kỹ thuật trong quy trình truyền thống và xác lập các yêu cầu nghiệp vụ cũng như thiết kế thuật toán cốt lõi cho hệ thống.

3.2.1 Khảo sát quy trình tìm kiếm và thu thập tài liệu nghiên cứu

Hiện nay, việc tìm kiếm bài báo khoa học chủ yếu dựa trên thao tác thủ công của học giả trên các công cụ tìm kiếm phổ quát và các cơ sở dữ liệu học thuật độc lập:

- Học giả nhập từ khóa trên Google Scholar, arXiv hoặc Semantic Scholar, sau

đó duyệt từng trang kết quả để tải tệp PDF về máy tính.

- Việc tải tài liệu diễn ra phân tán ở nhiều thư mục máy tính cá nhân. Khi làm việc theo nhóm, các thành viên thường gửi tệp qua email hoặc ứng dụng nhắn tin, dẫn đến tình trạng nhiều người tải trùng một tệp PDF dung lượng từ 5 MB đến 20 MB, gây lãng phí dung lượng lưu trữ và khó kiểm soát bản mới nhất.
- Khó khăn trong việc phân loại sơ bộ: Nhiều bài báo trên arXiv chỉ là bản thảo chưa qua bình duyệt (*Preprint*), trong khi các bài trên tạp chí đã trải qua phản biện kín (*Peer-Reviewed*). Học giả mất nhiều thời gian tra cứu thủ công xem bài báo đã được chấp nhận xuất bản hay chưa.

3.2.2 Khảo sát quy trình đọc sâu và thẩm định nguồn trích dẫn

Trong giai đoạn đọc hiểu tài liệu chuyên ngành:

- Hầu hết các bài báo quốc tế (định dạng IEEE, ACM, Springer) đều được trình bày dạng hai cột (*two-column*). Khi sử dụng các công cụ trích xuất PDF thông thường, văn bản bị đọc nhảy ngang giữa hai cột, làm đảo lộn hoàn toàn thứ tự ngữ nghĩa của các công thức toán và đoạn văn.
- Khi sử dụng các công cụ AI trò chuyện với tài liệu, câu trả lời do mô hình sinh ra thường chỉ dẫn chứng số trang chung chung (ví dụ: "theo trang 4 của tài liệu"). Trong một trang chứa hàng nghìn chữ cùng nhiều bảng biểu và biểu đồ, học giả vẫn phải tự mắt tìm kiếm từng câu chữ để xác nhận.
- Nguy cơ ảo giác: Mô hình ngôn ngữ lớn có xu hướng tự điền khuyết các số liệu đo lường khi không tìm thấy thông tin chính xác, dẫn đến nguy cơ sai lệch nghiêm trọng nếu học giả đưa số liệu này vào bài báo của mình. Do đó, yêu cầu hiển thị khung viền tọa độ Bounding Box định vị chính xác vị trí văn bản gốc là tiêu chí bắt buộc để bảo đảm liêm chính học thuật.

3.2.3 Khảo sát quy trình quản lý và liên kết đồ thị tri thức học thuật

Khi tiến hành tổng quan tài liệu (*Literature Review*):

- Nhà nghiên cứu cần đối chiếu xem các công trình tiền nhiệm đã sử dụng kiến trúc mô hình nào, thử nghiệm trên bộ dữ liệu chuẩn nào và đạt kết quả đo lường cụ thể ra sao.
- Phương pháp truyền thống là tạo bảng tính Excel thủ công. Khi số lượng bài báo tăng lên hàng chục hoặc hàng trăm bài, bảng tính trở nên cồng kềnh, khó

cập nhật và hoàn toàn mất đi khả năng truy vấn quan hệ bắc cầu giữa các thực thể tri thức.

- Học giả có nhu cầu xây dựng tự động một mạng đồ thị tri thức liên kết 4 thực thể cốt lõi: *Bài báo (Paper) → Phương pháp (Method) → Bộ dữ liệu (Dataset) → Kết quả đo lường (Metric Result)*, cho phép đặt câu hỏi so sánh liên tài liệu trong cùng một không gian nghiên cứu.

3.2.4 Khảo sát quy trình biên soạn bản thảo và sửa lỗi biên dịch LaTeX

Biên soạn bản thảo là công đoạn tốn nhiều thời gian và công sức nhất trong toàn bộ chu trình nghiên cứu:

- Ngôn ngữ LaTeX là tiêu chuẩn công bố bắt buộc của các hội nghị khoa học uy tín, nhưng có cú pháp rất chặt chẽ. Chỉ một lỗi nhỏ như quên đóng ngoặc môi trường toán học, sử dụng ký tự đặc biệt chưa thoát mã (như _, %, &), hoặc sai nhãn trích dẫn BibTeX cũng khiến toàn bộ quá trình biên dịch bị đình trệ.
- Tệp nhật ký lỗi (.log) của bộ biên dịch XeLaTeX thường dài hàng nghìn dòng với nhiều thông điệp kỹ thuật khó hiểu. Học giả, đặc biệt là sinh viên mới nghiên cứu, thường mất nhiều thời gian tra cứu Google để tìm cách khắc phục.
- Nhu cầu thực tế là một trợ lý AI có khả năng tự động đọc tệp nhật ký lỗi, phát hiện nguyên nhân gốc rễ và tự tạo bản vá sửa lỗi chính xác trên mã nguồn LaTeX mà không làm biến dạng các phần nội dung khác.

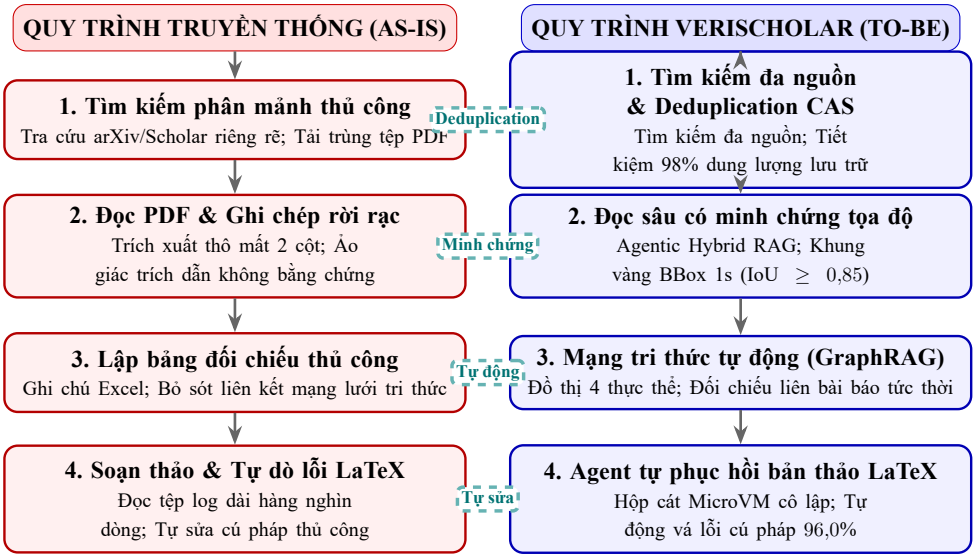
3.2.5 Khảo sát quy trình quản lý hồ sơ học giả và bảo mật khóa cá nhân

Đối với công tác quản trị và bảo mật thông tin nghiên cứu:

- Học giả cần một định danh duy nhất toàn cầu để phân biệt với các tác giả trùng tên khác. Chuẩn quốc tế ORCID iD gồm 16 chữ số là chuẩn mực được cộng đồng học thuật toàn cầu công nhận.
- Về tính bảo mật và quyền sở hữu trí tuệ: Các công trình nghiên cứu đang trong quá trình thực hiện chứa các phát kiến độc quyền chưa công bố. Học giả rất e ngại việc dữ liệu nghiên cứu bị các nền tảng AI thương mại sử dụng để huấn luyện mô hình.
- Do đó, hệ thống cần trang bị cơ chế "Tự mang khóa bí mật" (*Bring Your Own Key – BYOK*), cho phép học giả sử dụng khóa API riêng của mình. Khóa phải

được mã hóa bảo vệ nghiêm ngặt và tuân thủ quy chuẩn không lưu vết trên đĩa cứng, bảo đảm sự an tâm tuyệt đối cho nhà khoa học.

Từ kết quả khảo sát thực tế trên, sự chuyển dịch từ quy trình nghiên cứu truyền thống (AS-IS) sang quy trình nghiên cứu mới (TO-BE) trên nền tảng VeriScholar được mô hình hóa trực quan trong Hình 3-1.



Hình 3-1: *Mô hình đối chiếu chuyển đổi quy trình nghiên cứu khoa học AS-IS và TO-BE*

3.2.6 Thiết kế chi tiết các thuật toán xử lý cốt lõi của hệ thống

Để hiện thực hóa quá trình chuyển đổi từ quy trình AS-IS sang TO-BE nêu trên, hệ thống VeriScholar thiết kế và cài đặt bốn đường ống thuật toán cốt lõi xử lý xuyên suốt từ bước bóc tách hình học tài liệu, truy xuất lai tự kiểm định, tự phục hồi mã nguồn LaTeX đến tổng hợp đồ thị tri thức học thuật:

(a) Thuật toán sắp xếp khối văn bản PDF hai cột và phân đoạn cửa sổ trượt bảo toàn tọa độ: Với mỗi trang tài liệu có kích thước vật lý (W, H), bộ giải mã bố cục trích xuất tập các khối văn bản thô kèm hình chữ nhật bao quanh (x_0, y_0, x_1, y_1). Thuật toán trước tiên loại bỏ các phần tử nhiễu đầu trang và chân trang thỏa mãn điều kiện $y_0 < 0{,}06 \times H$ hoặc $y_1 > 0{,}93 \times H$ có độ dài dưới 80 ký tự. Tiếp theo, đường trung tuyến dọc $\text{mid_x} = 0{,}5 \times W$ được thiết lập để phân loại từng khối văn bản theo ba vùng bố cục: khối trải rộng toàn trang (khi $x_0 < 0{,}35 \times W$ và $x_1 > 0{,}65 \times W$, như tiêu đề hoặc tóm tắt), khối thuộc cột trái ($x_{\text{center}} < \text{mid_x}$) và khối thuộc cột phải ($x_{\text{center}} \geq \text{mid_x}$). Các khối được sắp xếp tất định theo khóa bộ bốn $(\text{band}, \text{col}, y_0, x_0)$, bảo đảm luồng văn bản đọc hoàn chỉnh hết cột trái trước khi chuyển sang cột phải. Trên chuỗi từ đã

sắp xếp, thuật toán áp dụng cửa sổ trượt kích thước tối đa max_words $= 250$ từ với độ gối lấp ngữ cảnh overlap_words $= 50$ từ, đồng thời chiếu chuẩn hóa mọi khung bao hình học về không gian đơn vị bất biến độ phân giải $[0, 1]^4$:

$$\hat{B} = \left(\frac{x_0}{W}, \frac{y_0}{H}, \frac{x_1}{W}, \frac{y_1}{H} \right) \in [0, 1]^4 \tag{3.1}$$

(b) Thuật toán truy xuất lai SQL CTE Hybrid Retrieval và cổng kiểm định độ tự tin CRAG: Khi tiếp nhận truy vấn của học giả, hệ thống thực thi truy vấn hợp nhất ngay trong PostgreSQL thông qua biểu thức bảng chung (CTE) gồm hai nhánh song song: nhánh ngữ nghĩa dense_search quét chỉ mục đồ thị phân cấp HNSW với toán tử khoảng cách Cosine (embedding <=> query_vec), và nhánh từ vựng sparse_search quét chỉ mục đảo GIN trên cột content_tsv bằng hàm mật độ bao phủ ts_rank_cd kết hợp tái chấm điểm Okapi BM25 ($k_1 = 1{,}5,\ b = 0{,}75$). Hai danh sách ứng viên được hợp nhất thứ hạng bằng công thức Reciprocal Rank Fusion (RRF) với hằng số làm trơn $k = 60$, sau đó đi qua tầng tái xếp hạng chéo (Cross-Encoder Reranking) để tính điểm hội tụ $\tau \in [0, 1]$. Trước khi gọi mô hình sinh câu trả lời, cổng kiểm định tiền sinh văn bản (*CRAG Confidence Gating*) đối chiếu τ với hai ngưỡng quyết định:

- Khi $\tau \geq 0{,}35$ (CORRECT): Ngữ cảnh đạt độ tin cậy cao, cho phép tổng hợp câu trả lời trực tiếp kèm thẻ dẫn nguồn.
- Khi $0{,}20 \leq \tau < 0{,}35$ (AMBIGUOUS_EXPANDED): Hệ thống tự động kích hoạt hàm mở rộng truy vấn học thuật đa chiều (expand_academic_query) và thực hiện truy xuất bổ sung vòng hai trước khi tổng hợp.
- Khi $\tau < 0{,}20$ (ACTIVE_ABSTENTION): Hệ thống chủ động từ chối suy diễn nhằm triệt tiêu hoàn toàn nguy cơ ảo giác ngoại suy.

(c) Thuật toán tự phục hồi mã nguồn LaTeX qua đồ thị LangGraph ReAct: Quá trình biên dịch và tự sửa lỗi bản thảo được điều phối bởi đồ thị trạng thái hữu hạn gồm 4 nút chức năng: reason_and_plan (phân tích trạng thái và lập kế hoạch), tool_execution (thực thi biên dịch trong hộp cát cô lập hoặc trích xuất ngữ cảnh), healing_router (đánh giá mã thoát và điều phối chu trình tự vá lỗi với giới hạn tối đa $N_{\text{max}} = 3$ vòng lặp), và generate_final_answer (kết xuất phản hồi). Khi phát hiện lỗi biên dịch (ExitCode $\neq 0$), hệ thống áp dụng cơ chế sửa lỗi hai tầng: (i) bộ vá lỗi tất định tốc độ cao (_auto_patch và apply_heuristic_repair) tự động nạp các gói lệnh bị thiếu (amsmath, graphicx,

booktabs), thoát mã các ký tự đặc biệt (_, %, &) ngoài môi trường toán và đóng các khối \begin{...} bị khuyết; (ii) bộ sinh bản vá vi phẫu bằng mô hình ngôn ngữ (generate_patch) phân tích tệp nhật ký .log để sửa các xung đột cú pháp phức tạp. Đặc biệt, chốt chặn chống dao động tuần hoàn (*SHA-256 Anti-Oscillation Guard*) tính mã băm SHA-256 của mã nguồn sau mỗi lần vá và đối chiếu với tập lịch sử seen_source_hashes; nếu phát hiện mã băm lặp lại hoặc mức độ lỗi không giảm, healing_router lập tức dừng vòng lặp và khôi phục phiên bản tốt nhất (best_latex_source).

(d) Thuật toán trích xuất đồ thị tri thức 4 thực thể, hợp nhất ngữ nghĩa và phân cụm Louvain (GraphRAG): Đối với mỗi tài liệu trong thư viện nghiên cứu, đường ống GraphRAG sử dụng cơ chế ép kiểu lược đồ đầu ra OpenAI Structured Outputs (kết hợp bộ quy tắc biểu thức chính quy dự phòng Rule Fallback) để bóc tách 4 loại nút thực thể (Paper, Method, Dataset, Metric Result) cùng 4 loại cạnh quan hệ có hướng (PROPOSES, EVALUATED_ON, ACHIEVES, CITES). Để ngăn chặn sự phân mảnh đỉnh giữa các bài báo khác nhau, thuật toán chuẩn hóa tên thực thể (canonical_name) và tính độ tương đồng Cosine giữa các vector nhúng thực thể; các nút cùng loại có Cosine $\geq 0{,}90$ được tự động hợp nhất về một thực thể chuẩn tắc duy nhất trong bảng graph_nodes. Cuối cùng, thuật toán phân cụm cộng đồng Louvain được thực thi trên đồ thị quy đổi vô hướng nhằm tối ưu hóa độ mô-đun hóa Q, từ đó tổng hợp bản tóm tắt cộng đồng theo thư mục (folder_community_summaries) và biên dịch hệ thống trang tri thức liên kết hai chiều (wiki_pages).

3.3 Giới thiệu các Actor và các Use Case tương ứng trong hệ thống

Từ việc khảo sát nghiệp vụ và quy trình nghiên cứu khoa học thực tế (Mục 3.2), có thể nhận thấy nhu cầu cấp thiết trong việc xây dựng một nền tảng trợ lý AI chuyên sâu, phục vụ đa dạng các thao tác nghiên cứu từ khám phá tài liệu, đọc sâu kiểm chứng đến biên soạn bản thảo.

Dựa trên yêu cầu bài toán và quy trình vận hành hệ thống, đề tài đề xuất hệ thống được thiết kế với hai tác nhân chính (Actor):

(1) Học giả (Scholar / Researcher): Là Actor trung tâm của hệ thống (bao gồm các giảng viên, nhà nghiên cứu, nghiên cứu sinh và sinh viên làm đồ án tốt nghiệp). Học giả tương tác trực tiếp với các phân hệ của nền tảng: tra cứu và tải tài liệu nghiên cứu từ các kho học thuật mở, đọc sâu tài liệu với sự trợ giúp của AI kèm định vị khung viền Bounding Box, khai phá đồ thị tri thức

đa thực thể, quản lý kho lưu trữ tài liệu cá nhân và biên soạn bản thảo bài báo LaTeX.

(2) **Quản trị viên hệ thống (Administrator / DevOps):** Chịu trách nhiệm vận hành kỹ thuật, thực thi kiểm định thực nghiệm qua các điểm cuối REST vận hành chuyên biệt (như /api/v1/evaluations/run-benchmark), giám sát cơ chế khóa bảo vệ tài khoản tự động và bộ đếm tham chiếu cơ sở dữ liệu, theo dõi các luồng sự kiện suy luận mô hình ngôn ngữ lớn (*Trace Logs*) trên bảng điều khiển Langfuse Cloud, cũng như giám sát hạ tầng đám mây Azure Portal và hộp cát MicroVM cách ly.

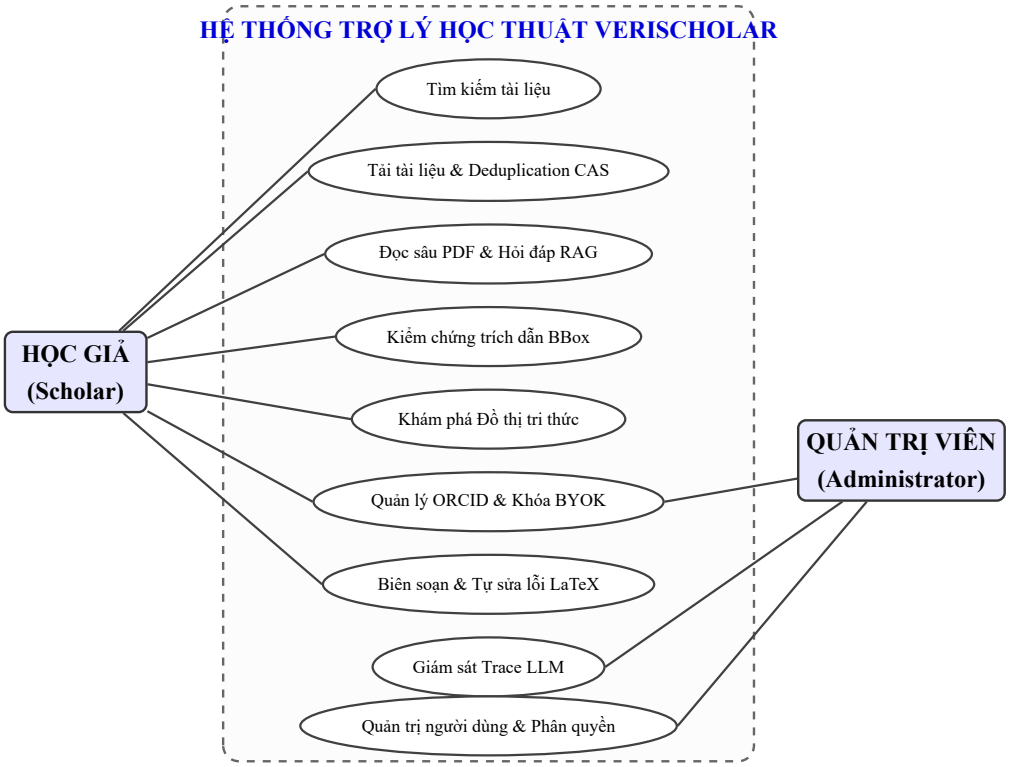
Các Use Case chính đối với mỗi Actor được tổng hợp và liệt kê trong Bảng 3-1.

Bảng 3-1: *Các Use Case chính đối với mỗi Actor trong hệ thống VeriScholar*

STT	Tên Actor	Use Case
1	**Học giả (Scholar)**	Đăng ký / Đăng nhập tài khoản
2		Tìm kiếm tài liệu học thuật
3		Tải tài liệu cá nhân và Deduplication lưu trữ CAS
4		Đọc sâu tài liệu PDF hai cột và Hỏi đáp với AI
5		Kiểm chứng trích dẫn tọa độ Bounding Box trực quan
6		Khám phá và truy vấn Đồ thị tri thức (GraphRAG)
7		Quản lý hồ sơ định danh ORCID và Kho khóa API BYOK
8		Biên soạn bản thảo LaTeX và Tự phục hồi lỗi biên dịch
9	**Quản trị viên (Admin)**	Quản lý tài khoản người dùng và phân quyền
10		Giám sát sức khỏe hạ tầng MicroVM (Sandbox)
11		Theo dõi Trace mô hình ngôn ngữ và Chi phí Token
12		Cấu hình chính sách lưu trữ và an toàn dữ liệu

3.4 Phân tích các Use Case của hệ thống

Từ bảng các Use Case đối với từng Actor được xác định ở Mục 3.3, biểu đồ Use Case tổng quan (Use Case Diagram) minh họa mối quan hệ tương tác giữa các Actor và các phân hệ của VeriScholar được trình bày trong Hình 3-2.



Hình 3-2: *Biểu đồ Use Case tổng quan của hệ thống VeriScholar*

3.4.1 Phân tích các Use Case phía Học giả

Nhằm làm rõ bản chất vận hành của các chức năng trọng tâm, tiểu mục này tiến hành đặc tả chi tiết 3 Use Case tiêu biểu của học giả: *Tra cứu và tải tài liệu nghiên cứu*, *Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ*, và *Biên soạn bản thảo và tự phục hồi lỗi LaTeX*.

3.4.1.1 Phân tích Use Case "Tra cứu và tải tài liệu nghiên cứu"

Use Case này cho phép học giả tìm kiếm các tài liệu nghiên cứu khoa học từ các kho học thuật mở toàn cầu (arXiv, CrossRef, Semantic Scholar), lọc các tài liệu đã bình duyệt, và tải trực tiếp tài liệu vào kho lưu trữ cá nhân kèm cơ chế Deduplication CAS SHA-256. Đặc tả chi tiết được trình bày trong Bảng 3-2.

Bảng 3-2: *Đặc tả Use Case "Tra cứu và tải tài liệu nghiên cứu"*

Tên Use Case	**Tra cứu và tải tài liệu nghiên cứu**
Actor	Học giả (Scholar)
Mục tiêu	Tìm kiếm bài báo khoa học theo từ khóa hoặc mã DOI/arXiv, lọc bài báo đã bình duyệt và lưu trữ vào thư viện nghiên cứu cá nhân.
Điều kiện tiên quyết	Học giả đã đăng nhập vào hệ thống VeriScholar.
Điều kiện kết thúc	Tài liệu được tải vào không gian làm việc; bản ghi thư mục và khối tệp CAS được tạo lập thành công.
Luồng sự kiện chính	1. Học giả nhập từ khóa tìm kiếm hoặc mã định danh DOI/arXiv vào thanh tìm kiếm. 2. Giao diện người dùng phản hồi tức thì với khung xương tải trang (*Skeleton Loading*); đối với truy vấn từ khóa từ 3 từ trở lên, hệ thống áp dụng bộ lọc độ tương đồng tiêu đề bằng chỉ số Jaccard (*Jaccard Token Overlap* ngưỡng $\geq 0{,}2$) nhằm ngăn chặn bài báo lạc đề (trong khi truy vấn trực tiếp theo mã định danh DOI/arXiv ID và nhánh tìm kiếm ngữ nghĩa Exa bỏ qua lọc từ vựng này), đồng thời truy vấn qua các cổng học thuật mở và hiển thị danh sách kết quả. 3. Học giả có thể kích hoạt nút gạt "Đã bình duyệt" (*Peer-reviewed*) để hệ thống tái sắp xếp thứ tự ưu tiên các bài báo uy tín lên đầu. 4. Học giả chọn bài báo mong muốn và nhấp nút "Tải tài liệu". 5. Hệ thống tải tệp PDF toàn văn, tính mã Hash SHA-256 để kiểm tra Deduplication trong kho lưu trữ CAS, và liên kết tài liệu vào không gian làm việc của học giả.
Luồng thay thế / Ngoại lệ	• Nếu tệp PDF đã tồn tại trong kho lưu trữ chung: Hệ thống tái sử dụng khối tệp vật lý sẵn có và tăng biến đếm tham chiếu (ref_count), rút ngắn thời gian nạp xuống mức tức thời. • Nếu bài báo thuộc diện thu phí bản quyền: Hệ thống thông báo chỉ hỗ trợ hiển thị tóm tắt (*Abstract*) và cho phép học giả tự tải lên tệp PDF cá nhân từ máy tính.

Đối với các truy vấn từ khóa thông thường có độ dài từ 3 từ khóa trở lên ($|Q| \geq 3$), độ tương đồng giữa truy vấn của học giả và tiêu đề bài báo từ các kho mở (OpenAlex, Semantic Scholar, CrossRef, arXiv) được chuẩn hóa thông qua hệ số tương đồng Jaccard giữa tập từ vựng truy vấn Q và tập từ vựng tiêu đề ứng viên

T:

$$J(Q,T) = \frac{|Q \cap T|}{|Q \cup T|} \tag{3.2}$$

Hệ thống thiết lập ngưỡng $J(Q,T) \geq 0{,}2$ đối với các truy vấn từ khóa ($|Q| \geq 3$) để tự động lọc bỏ các tài liệu trùng lặp ngẫu nhiên hoặc lạc đề, trong khi các truy vấn tra cứu chính xác theo mã định danh DOI/arXiv ID hoặc nhánh tìm kiếm ngữ nghĩa chuyên sâu (Exa Neural Search) được miễn trừ bước lọc từ vựng này nhằm bảo toàn độ phủ ngữ nghĩa tối đa.

3.4.1.2 Phân tích Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"

Use Case này hỗ trợ học giả phân tích chuyên sâu nội dung bài báo, đặt câu hỏi cho trợ lý AI và nhận câu trả lời có kèm các thẻ trích dẫn tọa độ Bounding Box hiển thị viền vàng trực quan trên bản PDF gốc (Bảng 3-3).

Bảng 3-3: *Đặc tả Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"*

Tên Use Case	**Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ**
Actor	Học giả (Scholar)
Mục tiêu	Học giả đặt câu hỏi nghiên cứu về tài liệu đang đọc; trợ lý AI phản hồi kèm định vị chính xác vị trí câu văn bằng chứng trên trang PDF gốc.
Điều kiện tiên quyết	Học giả đã mở một tài liệu PDF trong phân hệ Đọc sâu.
Điều kiện kết thúc	Câu trả lời được truyền phát qua luồng SSE kèm theo các thẻ minh chứng số $[1]$, $[2]$ liên kết với tọa độ BBox.
Luồng sự kiện chính	1. Học giả nhập câu hỏi nghiên cứu vào khung hội thoại (ví dụ: "Độ chính xác phân loại của mô hình trên ImageNet là bao nhiêu?"). 2. AI Agent kích hoạt công cụ tìm kiếm lai Hybrid Search (kết hợp vector HNSW và từ khóa BM25 qua thuật toán RRF) cùng tầng tái xếp hạng chéo để trích xuất các phân đoạn văn bản liên quan nhất. 3. Cổng kiểm định tiền sinh văn bản CRAG đánh giá ngưỡng tự tin ($\tau \geq 0{,}35$: tổng hợp trực tiếp; $0{,}20 \leq \tau < 0{,}35$: mở rộng truy vấn vòng hai; $\tau < 0{,}20$: từ chối suy diễn), sau đó mô hình tổng hợp câu trả lời và thực thi hàm kiểm chứng hậu sinh tất định (verify_output_grounding) nhằm xác thực mọi thẻ trích dẫn $[i]$ khớp với phân đoạn nguồn. 4. Trợ lý AI truyền luồng phản hồi theo thời gian thực cùng siêu dữ liệu tọa độ chuẩn hóa $[0, 1]^4$ về giao diện người dùng. 5. Học giả nhấp chuột vào thẻ trích dẫn $[1]$ trong câu trả lời; trình xem PDF lập tức tự động cuộn trang và vẽ khung viền vàng nhấp nháy 1 giây bao quanh đúng đoạn văn bản gốc (đạt ngưỡng kiểm định thực nghiệm IoU $\geq 0{,}85$).
Luồng thay thế / Ngoại lệ	• Nếu điểm tự tin CRAG $\tau < 0{,}20$ (trạng thái ACTIVE_ABSTENTION): Hệ thống chủ động từ chối suy diễn và thông báo rõ tài liệu không cung cấp đủ bằng chứng, tuyệt đối không tự bịa đặt số liệu. • Nếu học giả chưa nhập khóa API cá nhân (BYOK): Hệ thống điều hướng mở hộp thoại cài đặt khóa.

Trong thời gian chạy (*Runtime*), bộ giải mã tài liệu và trình kết xuất giao diện kiểm soát bất biến hình học của mọi khung tọa độ chuẩn hóa $\hat{B} = (\hat{x}_0, \hat{y}_0, \hat{x}_1, \hat{y}_1)$ trong miền đơn vị $0 \leq \hat{x}_0 \leq \hat{x}_1 \leq 1$ và $0 \leq \hat{y}_0 \leq \hat{y}_1 \leq 1$. Trong quy trình đánh giá chất lượng ngoại tuyến (*Offline Benchmark Evaluation*), độ chính xác định vị

tọa độ trực quan được đo lường thông qua chỉ số diện tích giao trên diện tích hợp (*Intersection over Union – IoU*) giữa hộp bao được hệ thống trích xuất B_{pred} và hộp bao vùng văn bản chuẩn B_{src} trên trang PDF gốc:

$$\text{IoU}(B_{\text{pred}}, B_{\text{src}}) = \frac{\text{Area}(B_{\text{pred}} \cap B_{\text{src}})}{\text{Area}(B_{\text{pred}} \cup B_{\text{src}})} \tag{3.3}$$

Bộ tiêu chuẩn kiểm thử của đề tài thiết lập ngưỡng chấp nhận thực nghiệm $\text{IoU} \geq 0{,}85$ nhằm bảo đảm khung viền vàng chỉ tô sáng chính xác câu văn chứa dẫn chứng khoa học mà không bị lệch dòng hay cắt ngang công thức toán học.

3.4.1.3 Phân tích Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"

Use Case này hỗ trợ học giả soạn thảo trực tuyến bài báo khoa học, biên dịch tài liệu sang định dạng PDF trong môi trường máy ảo cách ly, và tự động sửa các lỗi cú pháp khi biên dịch thất bại (Bảng 3-4).

Bảng 3-4: *Đặc tả Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"*

Tên Use Case	**Biên soạn bản thảo và tự phục hồi lỗi LaTeX**
Actor	Học giả (Scholar)
Mục tiêu	Biên soạn mã nguồn bài báo khoa học LaTeX, biên dịch sang PDF và tự động sửa lỗi cú pháp thông qua AI Agent ReAct.
Điều kiện tiên quyết	Học giả đang mở dự án bản thảo trong phân hệ Soạn thảo.
Điều kiện kết thúc	Bản thảo được biên dịch thành công; tệp PDF xem trước hiển thị trực tiếp trên giao diện.
Luồng sự kiện chính	1. Học giả chỉnh sửa mã nguồn LaTeX trên trình biên tập Monaco Editor. 2. Học giả nhấp nút "Biên dịch bản thảo". 3. Hệ thống chuyển mã nguồn vào E2B MicroVM cách ly an toàn và kích hoạt lệnh xelatex. 4. Nếu biên dịch thành công (ExitCode $== 0$): Tệp PDF được trả về và hiển thị trên khung xem trước thời gian thực.
Luồng thay thế / Ngoại lệ	• Nếu quá trình biên dịch thất bại (ExitCode $\neq 0$): a. Hệ thống trích xuất tệp nhật ký lỗi main.log. b. AI Agent phân tích lỗi và tạo bản vá sửa đổi vi phẫu (*Surgical Diff*). c. Hệ thống tự động áp dụng bản vá và kích hoạt tái biên dịch ($N < 3$). d. Khi thành công, thông báo lỗi được xóa bỏ và giao diện cập nhật PDF mới.

3.4.2 Phân tích các Use Case phía Quản trị viên

Theo quy chuẩn tối giản hóa sản phẩm 80/20, các tác vụ quản trị vận hành (UC-09 đến UC-12) không tách thành một trang giao diện quản trị độc lập cồng kềnh mà được thực thi trực tiếp thông qua hệ thống điểm cuối REST vận hành chuyên biệt (/api/v1/evaluations/run-benchmark), cơ chế khóa bảo vệ tự động cùng bộ đếm tham chiếu (ref_count) ngay trong tầng cơ sở dữ liệu PostgreSQL, kết hợp với bảng điều khiển giám sát từ xa trên Langfuse Cloud và Azure Portal.

3.4.2.1 Phân tích Use Case "Quản trị người dùng và chính sách bảo mật BYOK"

Use Case cho phép Quản trị viên kiểm soát vòng đời tài khoản học giả, thực thi cơ chế tự động tạm khóa bảo vệ khi đăng nhập sai liên tiếp, quản lý bộ đếm tham chiếu lưu trữ CAS và đảm bảo các chính sách khóa API mã hóa tuân thủ chuẩn an toàn thông tin (Bảng 3-5).

Bảng 3-5: *Đặc tả Use Case "Quản trị người dùng và chính sách bảo mật BYOK"*

Tên Use Case	**Quản trị người dùng và chính sách bảo mật BYOK**
Actor	Quản trị viên (Administrator)
Mục tiêu	Giám sát tài khoản học giả, quản lý hạn ngạch tài nguyên và đảm bảo tính an toàn của kho khóa API cá nhân qua cơ chế bảo vệ tự động.
Điều kiện tiên quyết	Quản trị viên có quyền truy cập hệ thống vận hành và cơ sở dữ liệu quản trị.
Điều kiện kết thúc	Chính sách bảo mật tài khoản, hạn ngạch và bộ đếm tham chiếu CAS được đồng bộ chính xác.
Luồng sự kiện chính	1. Quản trị viên kiểm tra trạng thái tài khoản, định danh ORCID và cấu hình bảo mật qua giao diện lập trình vận hành hoặc trình quản trị cơ sở dữ liệu. 2. Hệ thống tự động kiểm tra ngưỡng thử mật khẩu sai (tạm khóa 15 phút sau 5 lần thất bại liên tiếp) và duy trì tính toàn vẹn của kho khóa BYOK mã hóa AES-256-GCM. 3. Khi tài liệu được thêm mới hoặc gỡ bỏ, cơ chế đếm tham chiếu (ref_count) trong bảng document_storage_blobs tự động cập nhật để thu hồi khối tệp mồ côi. 4. Hệ thống lưu cấu hình và ghi nhật ký kiểm toán vận hành.
Luồng ngoại lệ	• Nếu phát hiện phiên làm việc bất thường hoặc vượt ngưỡng xác thực: Hệ thống tự động thu hồi phiên và kích hoạt khóa tạm thời để bảo vệ tài khoản.

3.4.2.2 Phân tích Use Case "Giám sát hệ thống và Trace mô hình"

Use Case cho phép Quản trị viên kích hoạt bộ kiểm định thực nghiệm tự động qua điểm cuối /api/v1/evaluations/run-benchmark, đồng thời theo dõi các chỉ số hoạt động hạ tầng trên Azure Portal, thời gian phản hồi API, chi phí Token sử dụng và các *Trace Logs* suy luận trên nền tảng Langfuse Cloud (Bảng 3-6).

Bảng 3-6: *Đặc tả Use Case "Giám sát hệ thống và Trace mô hình"*

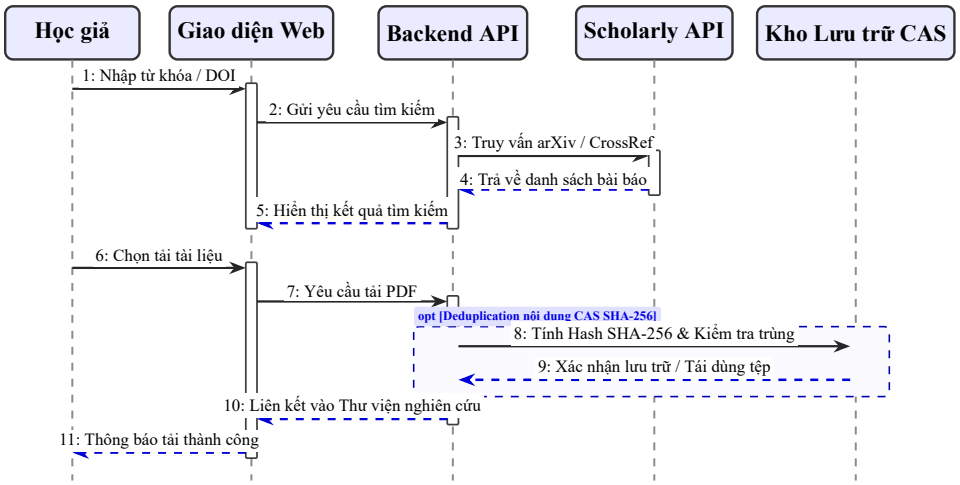
Tên Use Case	**Giám sát hệ thống và Trace mô hình**
Actor	Quản trị viên (Administrator)
Mục tiêu	Giám sát tình trạng vận hành hạ tầng, đánh giá định lượng hiệu năng RAG/Agent và theo dõi chi tiết Trace suy luận trên Langfuse Cloud.
Điều kiện tiên quyết	Quản trị viên đã xác thực quyền vận hành; kết nối telemetry tới Langfuse Cloud và Azure Portal hoạt động bình thường.
Điều kiện kết thúc	Báo cáo đánh giá chuẩn hóa (JSON/Markdown) và chuỗi Trace suy luận được ghi nhận đầy đủ.
Luồng sự kiện chính	1. Quản trị viên gọi điểm cuối vận hành /api/v1/evaluations/run-benchmark hoặc truy cập bảng điều khiển Langfuse Cloud / Azure Portal. 2. Hệ thống thu thập dữ liệu đo lường từ tập kiểm thử chuẩn cùng nhật ký telemetry thời gian thực. 3. Hiển thị các chỉ số định lượng: độ trung thực trích dẫn, độ trễ truy xuất Hybrid RAG, tỷ lệ tự sửa lỗi LaTeX và lượng Token tiêu thụ theo từng bước suy luận. 4. Quản trị viên phân tích các phiên có độ trễ cao để tinh chỉnh tham số chỉ mục HNSW và ngưỡng tự tin CRAG.
Luồng ngoại lệ	• Nếu kết nối tới dịch vụ Langfuse bên ngoài bị gián đoạn: Hệ thống tự động bỏ qua bước đẩy telemetry bất đồng bộ và ghi nhận vào nhật ký cục bộ mà không làm gián đoạn luồng phản hồi của học giả.

3.5 Phân tích một số biểu đồ tuần tự chính của hệ thống

Biểu đồ tuần tự (Sequence Diagram) được sử dụng để mô hình hóa sự tương tác theo dòng thời gian giữa các đối tượng và thành phần trong hệ thống khi thực thi một Use Case cụ thể. Nhằm làm rõ cơ chế phối hợp giữa giao diện người dùng, lõi nghiệp vụ và các dịch vụ hạ tầng, tiểu mục này trình bày ba biểu đồ tuần tự cho ba nghiệp vụ cốt lõi của VeriScholar.

3.5.1 Biểu đồ tuần tự Use Case "Tra cứu và tải tài liệu nghiên cứu"

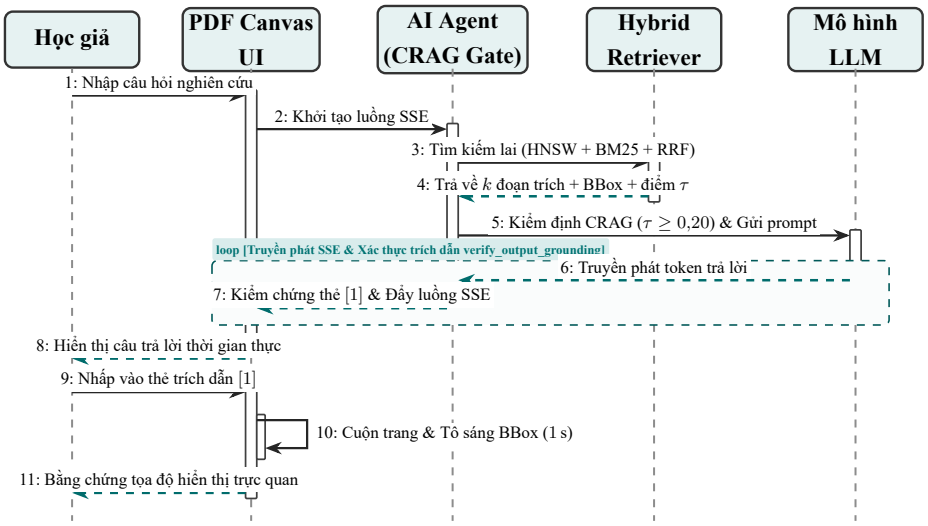
Quy trình tra cứu đa nguồn, kiểm tra Deduplication nội dung theo mã Hash SHA-256 và lưu trữ vào kho CAS được mô tả trong Hình 3-3.



Hình 3-3: *Biểu đồ tuần tự Use Case Tra cứu và tải tài liệu nghiên cứu*

3.5.2 Biểu đồ tuần tự Use Case "Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ"

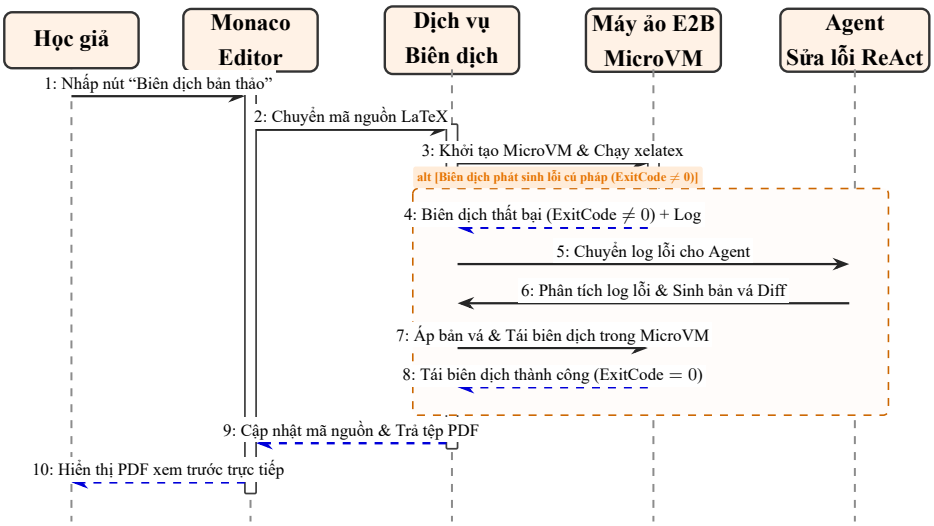
Chu trình hỏi đáp tăng cường truy xuất Hybrid RAG (kết hợp vector HNSW và từ khóa BM25 qua thuật toán RRF), cổng kiểm định độ tự tin tiền sinh văn bản CRAG ($\tau \in \{0{,}20,\ 0{,}35\}$), xác thực trích dẫn hậu sinh (verify_output_grounding), truyền luồng sự kiện SSE và tô sáng khung viền tọa độ Bounding Box được mô hình hóa trong Hình 3-4.



Hình 3-4: *Biểu đồ tuần tự Use Case Hỏi đáp đọc sâu và kiểm chứng trích dẫn tọa độ*

3.5.3 Biểu đồ tuần tự Use Case "Biên soạn bản thảo và tự phục hồi lỗi LaTeX"

Cơ chế phối hợp khép kín giữa trình soạn thảo Monaco, E2B MicroVM cách ly an toàn và chu trình tự sửa lỗi ReAct Reflexion khi phát hiện lỗi cú pháp được minh họa trong Hình 3-5.



Hình 3-5: *Biểu đồ tuần tự Use Case Biên soạn bản thảo và tự phục hồi lỗi LaTeX*

3.6 Thiết kế cơ sở dữ liệu cho hệ thống

Cơ sở dữ liệu là thành phần nền tảng quyết định độ tin cậy, tính toàn vẹn và hiệu năng vận hành của toàn bộ hệ thống VeriScholar. Căn cứ vào các yêu cầu nghiệp vụ và các Use Case đã được phân tích ở các mục trước, hệ thống sử dụng hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL 16 mạnh mẽ nhằm bảo đảm tính toàn vẹn giao dịch (ACID), khả năng mở rộng lưu trữ lớn và bảo mật cao.

3.6.1 Thiết kế cơ sở dữ liệu của hệ thống

Cơ sở dữ liệu của hệ thống VeriScholar được thiết kế bám sát đặc tả kỹ thuật và lược đồ quan hệ thực tế gồm đúng 20 bảng vật lý trên PostgreSQL 16 tích hợp pgvector, phục vụ 5 phân hệ nghiệp vụ chuyên sâu:

(1) **Phân hệ Định danh & Bảo mật Khóa (Module 1 – Auth & BYOK):** Quản lý tài khoản học giả, mật khẩu băm, mã định danh ORCID iD, hồ sơ chuyên môn và kho khóa API cá nhân (BYOK) mã hóa đối xứng phân tầng AES-256-GCM.

(2) **Phân hệ Lưu trữ CAS, Thư viện & Trích xuất Tài liệu (Module 2 – CAS Storage, Library & Chunks):** Quản lý khối tệp nhị phân theo mã băm SHA-

256 Deduplication, tổ chức cây thư mục cá nhân, hồ sơ siêu dữ liệu bài báo, bản tóm tắt cấu trúc, phân đoạn văn bản kèm tọa độ Bounding Box, vector nhúng và danh mục tài liệu tham khảo đối chiếu chéo.

(3) **Phân hệ Hội thoại Nghiên cứu & Đọc sâu (Module 3 – Research Sessions & Notes):** Quản lý phiên làm việc đa tài liệu, bảng liên kết tài liệu trong phiên, lịch sử hỏi đáp tăng cường truy xuất Hybrid RAG và ghi chú học thuật có neo giữ tọa độ.

(4) **Phân hệ Đồ thị Tri thức & Wiki Liên kết (GraphRAG):** Lưu trữ mạng lưới đồ thị 4 thực thể (graph_nodes, graph_edges), phân cụm Louvain (folder_ community_summaries) và các trang Wiki học thuật liên kết (wiki_pages).

(5) **Phân hệ Soạn thảo Học thuật & Đồng tác giả AI (Module 5 – Agentic Writing):** Lưu trữ bản thảo LaTeX chuẩn hội nghị IEEE/ACM (drafts), tài nguyên hình ảnh đính kèm (draft_assets) và nhật ký tương tác với Agent tự động sửa lỗi biên dịch (draft_messages).

Toàn bộ 20 bảng cơ sở dữ liệu thực tế của hệ thống VeriScholar được tổng hợp chi tiết theo 5 phân hệ trong Bảng 3-7.

3.6.2 Đặc tả các bảng cơ sở dữ liệu của hệ thống

Nhằm làm rõ cấu trúc vật lý, kiểu dữ liệu và các ràng buộc toàn vẹn, tiểu mục này trình bày đặc tả chi tiết 16 bảng cơ sở dữ liệu cốt lõi đại diện cho cả 5 phân hệ nghiệp vụ của hệ thống.

Bảng 3-7: *Các bảng cơ sở dữ liệu của hệ thống VeriScholar*

STT	Tên bảng	Ý nghĩa nghiệp vụ
I. Phân hệ Định danh & Bảo mật Khóa (Auth & BYOK)		
1	users	Thông tin tài khoản học giả, mật khẩu băm, vai trò và định danh ORCID
2	scholar_profiles	Hồ sơ học thuật mở rộng, cơ quan công tác và hướng nghiên cứu
3	user_api_keys	Kho mã hóa lưu trữ khóa API cá nhân (BYOK) của học giả
II. Phân hệ Lưu trữ CAS, Thư viện & Trích xuất Tài liệu (CAS, Library & Chunks)		
4	document_storage_blobs	Khối tệp nhị phân PDF lưu trữ theo mã Hash SHA-256 (Deduplication CAS)
5	documents	Hồ sơ bài báo khoa học, tiêu đề chuẩn hóa và siêu dữ liệu tài liệu
6	document_chunks	Phân đoạn văn bản kèm tọa độ BBox, vector nhúng VECTOR(1536) và TSVECTOR
7	document_summaries	Bản tóm tắt bài báo cấu trúc (Tổng quan, Đóng góp chính, Phương pháp)
8	reference_items	Danh mục tài liệu tham khảo trích xuất từ bài báo và đối chiếu chéo
9	folders	Cây thư mục phân cấp tổ chức tài liệu nghiên cứu cá nhân
III. Phân hệ Hội thoại Nghiên cứu & Đọc sâu (Research Sessions & Notes)		
10	research_sessions	Phiên nghiên cứu đa tài liệu, quản lý trạng thái hiển thị Reader
11	session_documents	Bảng liên kết nhiều–nhiều (N–N) giữa phiên nghiên cứu và bài báo
12	session_turns	Lịch sử các lượt hỏi đáp nghiên cứu kèm trích dẫn chứng minh
13	research_notes	Ghi chú học thuật có neo giữ tọa độ trên trang PDF (Anchor Memory)
IV. Phân hệ Đồ thị Tri thức Học thuật & Wiki Liên kết (GraphRAG & Academic Wiki)		
14	graph_nodes	Các nút thực thể đồ thị tri thức 4 loại (Paper, Method, Dataset, MetricResult)
15	graph_edges	Các cạnh quan hệ học thuật có hướng (PROPOSES, EVALUATED_ON, ACHIEVES, CITES)
16	folder_community_summaries	Bản tóm tắt cộng đồng tri thức phân cụm Louvain cấp thư mục phục vụ truy vấn toàn cục
17	wiki_pages	Các trang Wiki học thuật tổng hợp tự động kèm siêu dữ liệu YAML và liên kết hai chiều
V. Phân hệ Soạn thảo Học thuật & Đồng tác giả AI (Agentic Writing)		
18	drafts	Bản thảo bài báo khoa học định dạng LaTeX chuẩn hội nghị
19	draft_assets	Các tệp tài nguyên, hình ảnh và biểu đồ chèn vào bản thảo LaTeX
20	draft_messages	Nhật ký hội thoại giữa học giả và AI Agent đồng tác giả kèm trạng thái tự sửa lỗi

3.6.2.1 Đặc tả bảng users

Bảng users lưu trữ định danh cá nhân của học giả, chuỗi băm mật khẩu xác thực và liên kết với mã định danh quốc tế ORCID iD (Bảng 3-8).

Bảng 3-8: *Đặc tả bảng users*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh duy nhất người dùng
2	email	VARCHAR(255)	Unique, Not Null	Địa chỉ email đăng nhập hệ thống
3	full_name	VARCHAR(100)	Not Null	Họ và tên của học giả
4	hashed_password	VARCHAR(255)	Not Null	Chuỗi băm mật khẩu xác thực an toàn
5	orcid_id	VARCHAR(50)	Null	Mã định danh học giả quốc tế ORCID iD
6	role	VARCHAR(30)	Default ’researcher’	Vai trò tài khoản trong hệ thống
7	is_active	BOOLEAN	Default TRUE	Trạng thái hoạt động của tài khoản
8	created_at	TIMESTAMPTZ	Default NOW()	Thời điểm khởi tạo tài khoản

3.6.2.2 Đặc tả bảng user_api_keys

Bảng user_api_keys bảo vệ khóa API cá nhân của học giả theo cơ chế BYOK, sử dụng thuật toán mã hóa đối xứng phân tầng (Bảng 3-9).

Bảng 3-9: *Đặc tả bảng user_api_keys*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh bản ghi khóa
2	user_id	UUID	FK → users(id)	Khóa ngoại liên kết với người dùng sở hữu
3	provider	VARCHAR(32)	Not Null	Nhà cung cấp dịch vụ (OpenAI, Gemini)
4	encrypted_key	TEXT	Not Null	Bản mã khóa API được mã hóa an toàn
5	key_hint	VARCHAR(8)	Not Null	4 ký tự cuối nhận diện khóa trên giao diện
6	is_active	BOOLEAN	Default TRUE	Trạng thái kích hoạt sử dụng

3.6.2.3 Đặc tả bảng scholar_profiles

Bảng scholar_profiles lưu trữ thông tin hồ sơ học thuật chuyên sâu, cơ quan công tác và tiểu sử nghiên cứu của học giả (Bảng 3-10).

Bảng 3-10: *Đặc tả bảng scholar_profiles*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh hồ sơ học thuật
2	user_id	UUID	FK → users(id), 1:1	Khóa ngoại tương ứng duy nhất với tài khoản
3	institution	VARCHAR(255)	Null	Cơ quan công tác, trường đại học
4	department	VARCHAR(128)	Null	Khoa, phòng ban hoặc viện nghiên cứu
5	academic_rank	VARCHAR(64)	Null	Chức danh học thuật (GS, PGS, TS)
6	bio	TEXT	Null	Tóm tắt tiểu sử học thuật và thành tựu

3.6.2.4 Đặc tả bảng folders

Bảng folders tổ chức cây thư mục phân cấp phục vụ sắp xếp tài liệu khoa học của từng học giả (Bảng 3-11).

Bảng 3-11: *Đặc tả bảng folders*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh thư mục tài liệu
2	user_id	UUID	FK → users(id)	Khóa ngoại liên kết học giả sở hữu
3	parent_id	UUID	FK → folders(id), Null	Khóa ngoại tự quy chiếu thư mục cha
4	name	VARCHAR(128)	Not Null	Tên thư mục nghiên cứu
5	color	VARCHAR(16)	Default ’#3B82F6’	Mã màu hiển thị thư mục trên giao diện

3.6.2.5 Đặc tả bảng documents

Bảng documents quản lý thông tin thư mục, tác giả, năm xuất bản và liên kết với khối tệp nhị phân CAS (Bảng 3-12).

Bảng 3-12: *Đặc tả bảng documents*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh tài liệu học thuật
2	user_id	UUID	FK → users(id)	Khóa ngoại liên kết với học giả sở hữu
3	blob_id	UUID	FK → document_ storage_ blobs(id)	Khóa ngoại liên kết khối tệp nhị phân CAS
4	folder_id	UUID	FK → folders(id), Null	Khóa ngoại liên kết thư mục chứa
5	title	TEXT	Not Null	Tiêu đề bài báo khoa học đã chuẩn hóa
6	doi	VARCHAR(128)	Index, Null	Mã định danh đối tượng số (DOI) quốc tế
7	page_count	INTEGER	Default 0	Tổng số trang giấy của tài liệu PDF
8	status	VARCHAR(32)	Default ’ready’	Trạng thái trích xuất nội dung tài liệu

3.6.2.6 Đặc tả bảng document_storage_blobs

Bảng document_storage_blobs quản lý các khối tệp nhị phân trên hệ thống tệp đĩa cứng, áp dụng cơ chế định danh theo nội dung SHA-256 (Bảng 3-13).

Bảng 3-13: *Đặc tả bảng document_storage_blobs*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh khối tệp vật lý
2	content_hash	VARCHAR(64)	Unique, Not Null	Mã băm SHA-256 nội dung nhị phân tệp PDF
3	storage_path	TEXT	Not Null	Đường dẫn lưu trữ tệp phân cấp trên đĩa
4	file_size_bytes	BIGINT	Not Null	Dung lượng vật lý của tệp tin (bytes)
5	ref_count	INTEGER	Default 1	Biến đếm số tài liệu đang dùng chung tệp
6	status	VARCHAR(32)	Default ’ready’	Trạng thái khả dụng của khối dữ liệu

3.6.2.7 Đặc tả bảng document_chunks

Bảng document_chunks lưu trữ các phân đoạn văn bản trích xuất từ tài liệu PDF, số trang, tọa độ Bounding Box chuẩn hóa $[0, 1]^4$, vector nhúng 1536 chiều và chỉ mục từ vựng toàn văn (Bảng 3-14).

Bảng 3-14: *Đặc tả bảng document_chunks*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh phân đoạn văn bản
2	document_id	UUID	FK → documents(id)	Khóa ngoại liên kết tới tài liệu gốc
3	user_id	UUID	FK → users(id)	Khóa ngoại phân vùng bảo mật học giả
4	chunk_index	INTEGER	Not Null	Vị trí thứ tự phân đoạn trong tài liệu
5	page_number	INTEGER	Not Null, Index	Số thứ tự trang giấy trong bản PDF gốc
6	content	TEXT	Not Null	Nội dung văn bản của phân đoạn
7	bounding_boxes	JSONB	Not Null	Danh sách tọa độ hình chữ nhật chuẩn hóa $[0, 1]^4$
8	token_count	INTEGER	Default 0	Số lượng token của phân đoạn văn bản
9	embedding	VECTOR(1536)	HNSW Index, Null	Vector nhúng ngữ nghĩa 1536 chiều
10	content_tsv	TSVECTOR	GIN Index, Null	Vector từ vựng phục vụ tìm kiếm toàn văn BM25

3.6.2.8 Đặc tả bảng reference_items

Bảng reference_items lưu trữ danh mục tài liệu tham khảo trích xuất từ bài báo và kết quả đối chiếu tự động (Bảng 3-15).

Bảng 3-15: *Đặc tả bảng reference_items*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh mục tài liệu trích dẫn
2	document_id	UUID	FK → documents(id)	Khóa ngoại tới bài báo trích dẫn
3	resolved_ document_id	UUID	FK → documents(id), Null	Khóa ngoại tới bài báo trong kho đối chiếu trúng
4	ref_index	INTEGER	Not Null	Chỉ số thứ tự tham khảo trong bài $[i]$
5	parsed_title	TEXT	Not Null	Tiêu đề bài báo trích dẫn đã trích xuất
6	doi	VARCHAR(128)	Index, Null	Mã số định danh DOI trích dẫn

3.6.2.9 Đặc tả bảng research_sessions

Bảng research_sessions quản lý không gian làm việc nghiên cứu, tài liệu đang mở và thẻ chức năng đang thao tác (Bảng 3-16).

Bảng 3-16: *Đặc tả bảng research_sessions*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh phiên nghiên cứu
2	user_id	UUID	FK → users(id)	Khóa ngoại liên kết học giả sở hữu
3	active_doc_id	UUID	FK → documents(id), Null	Khóa ngoại tới bài báo đang đọc trên Reader
4	title	VARCHAR(255)	Not Null	Tên chủ đề phiên nghiên cứu
5	active_tab	VARCHAR(32)	Default 'chat'	Thẻ giao diện đang mở (chat / wiki / draft)
6	is_archived	BOOLEAN	Default FALSE	Cờ đánh dấu phiên đã lưu trữ

3.6.2.10 Đặc tả bảng session_turns

Bảng session_turns lưu trữ toàn bộ lịch sử các lượt hỏi đáp, câu trả lời sinh ra từ Hybrid RAG và mảng trích dẫn có neo giữ tọa độ (Bảng 3-17).

Bảng 3-17: *Đặc tả bảng session_turns*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh lượt hỏi đáp
2	session_id	UUID	FK → research_ sessions(id)	Khóa ngoại liên kết phiên nghiên cứu
3	user_id	UUID	FK → users(id)	Khóa ngoại phân quyền học giả
4	question	TEXT	Not Null	Câu hỏi nghiên cứu do học giả nhập
5	answer	TEXT	Not Null	Câu trả lời tổng hợp kèm thẻ dẫn nguồn
6	citations	JSONB	Default ’[]’	Mảng đối tượng trích dẫn kèm tọa độ BBox

3.6.2.11 Đặc tả bảng graph_nodes

Bảng graph_nodes lưu trữ các nút thực thể chuẩn tắc thuộc 4 nhóm học thuật (Paper, Method, Dataset, MetricResult) phục vụ truy vấn đồ thị tri thức liên bài báo (Bảng 3-18).

Bảng 3-18: *Đặc tả bảng graph_nodes*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh nút thực thể
2	user_id	UUID	FK → users(id)	Khóa ngoại phân vùng không gian học giả
3	document_id	UUID	FK → documents(id), Null	Khóa ngoại tới bài báo nguồn khởi tạo nút
4	node_type	VARCHAR(30)	Check, Not Null	Loại nút (Paper, Method, Dataset, MetricResult)
5	canonical_name	VARCHAR(255)	Not Null, Index	Tên thực thể chuẩn hóa dùng để hợp nhất đỉnh
6	display_name	VARCHAR(255)	Not Null	Tên hiển thị đầy đủ của thực thể học thuật
7	description	TEXT	Null	Mô tả tóm tắt ngữ nghĩa của thực thể
8	metadata_json	JSONB	Null	Siêu dữ liệu bổ sung của nút thực thể
9	embedding	VECTOR(1536)	Null	Vector nhúng phục vụ khử trùng lặp Cosine $\geq 0{,}90$

3.6.2.12 Đặc tả bảng graph_edges

Bảng graph_edges lưu trữ các cạnh quan hệ có hướng giữa các thực thể học thuật kèm liên kết minh chứng tới phân đoạn văn bản gốc (Bảng 3-19).

Bảng 3-19: *Đặc tả bảng graph_edges*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh cạnh quan hệ
2	user_id	UUID	FK → users(id)	Khóa ngoại phân vùng học giả sở hữu
3	document_id	UUID	FK → documents(id), Null	Khóa ngoại tới bài báo chứa quan hệ
4	source_node_id	UUID	FK → graph_ nodes(id)	Khóa ngoại trỏ tới nút thực thể nguồn
5	target_node_id	UUID	FK → graph_ nodes(id)	Khóa ngoại trỏ tới nút thực thể đích
6	edge_type	VARCHAR(30)	Check, Not Null	Loại quan hệ (PROPOSES, EVALUATED_ON, ACHIEVES, CITES)
7	weight	FLOAT	Default 1.0	Trọng số tin cậy của liên kết học thuật
8	evidence_ chunk_id	UUID	FK → document_ chunks(id), Null	Khóa ngoại tới phân đoạn văn bản bằng chứng
9	page_number	INTEGER	Null	Số thứ tự trang PDF xuất hiện quan hệ

3.6.2.13 Đặc tả bảng folder_community_summaries

Bảng folder_community_summaries lưu trữ bản tóm tắt cộng đồng tri thức được tổng hợp từ thuật toán phân cụm Louvain cấp thư mục nhằm phục vụ truy vấn toàn cục (Bảng 3-20).

Bảng 3-20: *Đặc tả bảng folder_community_summaries*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh bản tóm tắt cộng đồng
2	user_id	UUID	FK → users(id)	Khóa ngoại liên kết học giả sở hữu
3	folder_id	UUID	FK → folders(id), Null	Khóa ngoại liên kết thư mục chuyên đề
4	summary_text	TEXT	Not Null	Văn bản tổng hợp bức tranh toàn cảnh cụm
5	key_findings	JSONB	Null	Danh sách các phát hiện và xu hướng cốt lõi
6	embedding	VECTOR(1536)	Null	Vector nhúng phục vụ tìm kiếm toàn cục

3.6.2.14 Đặc tả bảng wiki_pages

Bảng wiki_pages lưu trữ các trang tri thức học thuật biên dịch tự động theo định dạng Markdown kèm siêu dữ liệu YAML và mạng liên kết hai chiều [[wikilinks]] (Bảng 3-21).

Bảng 3-21: *Đặc tả bảng wiki_pages*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh trang Wiki học thuật
2	user_id	UUID	FK → users(id)	Khóa ngoại liên kết học giả sở hữu
3	folder_id	UUID	FK → folders(id), Null	Khóa ngoại liên kết thư mục nghiên cứu
4	slug	VARCHAR(255)	Not Null	Định danh đường dẫn tĩnh duy nhất theo người dùng
5	title	VARCHAR(255)	Not Null	Tiêu đề trang tri thức học thuật
6	page_type	VARCHAR(50)	Default 'topic'	Phân loại trang (method, dataset, topic, comparison)
7	markdown_ content	TEXT	Not Null	Nội dung bài tổng hợp định dạng Markdown
8	yaml_metadata	JSONB	Not Null	Siêu dữ liệu cấu trúc YAML đầu trang
9	wikilinks	JSONB	Not Null	Danh sách liên kết chéo hai chiều [[wikilinks]]
10	embedding	VECTOR(1536)	Null	Vector nhúng ngữ nghĩa của trang Wiki

3.6.2.15 Đặc tả bảng drafts

Bảng drafts lưu trữ các bản thảo bài báo khoa học định dạng LaTeX, phiên bản tài liệu và liên kết phiên nghiên cứu nguồn (Bảng 3-22).

Bảng 3-22: *Đặc tả bảng drafts*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh bản thảo khoa học
2	user_id	UUID	FK → users(id)	Khóa ngoại liên kết tác giả chính
3	source_ session_id	UUID	FK → research_ sessions(id), Null	Khóa ngoại tới phiên nghiên cứu khởi tạo
4	title	VARCHAR(255)	Not Null	Tiêu đề bản thảo bài báo
5	format	VARCHAR(32)	Default 'latex'	Định dạng xuất bản (latex / markdown)
6	version	INTEGER	Default 1	Phiên bản cập nhật của bản thảo

3.6.2.16 Đặc tả bảng draft_messages

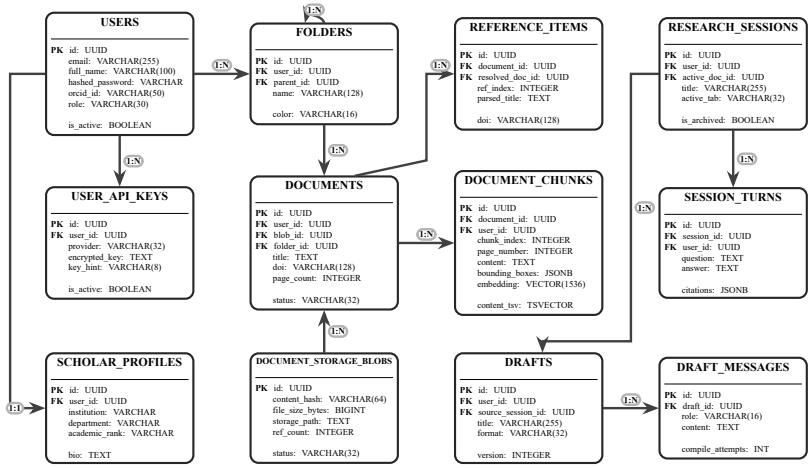
Bảng draft_messages lưu trữ nhật ký trao đổi đồng tác giả giữa học giả và AI Agent trong quá trình biên soạn và tự sửa lỗi biên dịch (Bảng 3-23).

Bảng 3-23: *Đặc tả bảng draft_messages*

STT	Tên cột	Kiểu dữ liệu	Ràng buộc	Mô tả
1	id	UUID	PK, Not Null	Khóa chính định danh tin nhắn đồng tác giả
2	draft_id	UUID	FK → drafts(id)	Khóa ngoại liên kết bản thảo bài báo
3	role	VARCHAR(16)	Not Null	Vai trò gửi (user / assistant / system)
4	content	TEXT	Not Null	Nội dung trao đổi hoặc chỉ dẫn viết bài
5	compile_ attempts	INTEGER	Default 0	Số lượt Agent ReAct tự động sửa lỗi

3.6.3 Lược đồ thực thể quan hệ (ERD) của hệ thống

Từ các bảng cơ sở dữ liệu và các ràng buộc toàn vẹn khóa chính (PK) – khóa ngoại (FK) đã được đặc tả chi tiết ở trên, Lược đồ thực thể quan hệ (ERD) mô tả mối quan hệ liên kết giữa các thực thể trọng tâm trong hệ quản trị cơ sở dữ liệu quan hệ PostgreSQL được trình bày trong Hình 3-6.



Hình 3-6: *Mối quan hệ giữa các bảng trong hệ quản trị cơ sở dữ liệu quan hệ (ERD)*

3.6.4 Thiết kế hạ tầng lưu trữ vector và đồ thị tri thức phụ trợ

Bên cạnh lược đồ thực thể quan hệ RDBMS chuẩn mực nêu trên, để hỗ trợ năng lực suy luận ngữ nghĩa của AI Agent và khám phá mạng lưới học thuật ngay trong cùng một không gian giao dịch ACID, hệ thống tích hợp trực tiếp hai cơ chế chỉ mục và truy vấn đồ thị chuyên biệt trên PostgreSQL 16:

(1) **Chỉ mục Không gian Vector và Từ vựng Toàn văn (Hybrid Vector & Lexical Store):** Nhằm phục vụ truy xuất tương đồng ngữ nghĩa kết hợp từ khóa trong kiến trúc Hybrid Search, cột embedding (1536 chiều) của bảng document_chunks, graph_nodes, folder_community_summaries và wiki_pages được quản lý bằng tiện ích mở rộng pgvector với chỉ mục phân cấp HNSW (vector_cosine_ops), song hành cùng chỉ mục đảo GIN trên cột content_tsv (TSVECTOR).

(2) **Mô hình Đồ thị Tri thức Đa thực thể (Graph Knowledge Store):** Phục vụ cho tính năng GraphRAG, các thực thể thuộc 4 nhóm (Paper, Method, Dataset, MetricResult) và 4 loại cạnh quan hệ (PROPOSES, EVALUATED_ON, ACHIEVES, CITES) được trích xuất tự động bằng cơ chế ép kiểu lược đồ đầu ra OpenAI Structured Outputs (kết hợp bộ quy tắc dự phòng Rule Fallback) và lưu trữ trực tiếp vào hai bảng graph_nodes, graph_edges. Trên cấu trúc lưu trữ này, các truy vấn lan truyền đa bước nhảy (2-hop Graph Traversal) được thực thi qua biểu thức bảng chung SQL CTE (*Common Table Expressions*), kết hợp phân cụm Louvain lưu vào folder_community_summaries và wiki_pages, cho phép học giả đối chiếu liên tài liệu tức thời.

CHƯƠNG 4: CÀI ĐẶT VÀ KẾT QUẢ THỰC NGHIỆM

4.1 Môi trường thực nghiệm

Để chứng minh tính khả thi, độ tin cậy và năng lực giải quyết các thách thức học thuật của hệ thống VeriScholar, chương này trình bày chi tiết về môi trường cài đặt, quy trình triển khai thực tế trên hạ tầng đám mây và kết quả đánh giá thực nghiệm toàn diện.

4.1.1 Cài đặt Visual Studio Code và môi trường phát triển

Visual Studio Code (kết hợp môi trường lập trình trợ lực trí tuệ nhân tạo Cursor IDE) được lựa chọn làm môi trường phát triển tích hợp (IDE) chủ đạo cho toàn bộ dự án VeriScholar. Với cấu trúc kho mã nguồn đơn nhất (Monorepo) kết hợp giữa tầng dịch vụ Backend (Python 3.12) và tầng ứng dụng Frontend (React 19, TypeScript), VS Code đem lại các lợi thế vượt trội:

- **Khả năng quản trị Monorepo linh hoạt:** Cho phép làm việc đồng thời trên nhiều gói mã nguồn (packages/core, apps/api, apps/web) trong cùng một không gian làm việc (*Workspace*), hỗ trợ tự động nhận diện môi trường ảo hóa độc lập cho từng phân vùng.
- **Hệ sinh thái tiện ích lập trình phong phú:** Tích hợp chặt chẽ với tiện ích mở rộng Ruff (linter và formatter siêu tốc cho Python 3.12), Tailwind CSS IntelliSense, ESLint, Python Test Explorer (cho pytest) và LaTeX Workshop hỗ trợ biên dịch và kiểm tra tài liệu học thuật theo thời gian thực.
- **Quản lý mã nguồn phân tán với Git/GitHub:** Toàn bộ lịch sử phát triển, các nhánh tính năng và mã nguồn hoàn chỉnh của dự án được quản trị và công khai tại kho lưu trữ GitHub chính thức https://github.com/dammanhdungvn/VeriScholar, hỗ trợ trực quan hóa lịch sử cam kết (*Git Commits*) và đối chiếu xung đột mã nguồn thuận tiện, đảm bảo tuân thủ nghiêm ngặt quy trình kiểm thử trước khi bàn giao.
- **Công bố minh bạch về công cụ trợ lực trí tuệ nhân tạo:** Trong quá trình triển khai kỹ thuật và biên tập tài liệu học thuật, các công cụ trợ lý lập trình AI (Cursor IDE, GitHub Copilot) được sử dụng dưới sự kiểm soát và xác minh trực tiếp của con người để tăng tốc sinh mã khung kiểm thử đơn vị, kiểm

tra ranh giới kiến trúc AST và chuẩn hóa cú pháp chế bản LaTeX; toàn bộ thiết kế kiến trúc lục giác, thuật toán lõi, tập dữ liệu đối chuẩn và số liệu thực nghiệm đều được kiểm định độc lập qua bộ 4 Cổng xác minh tự động.

Quy trình thiết lập môi trường phát triển cục bộ được thực hiện qua các bước:

1. Tải và cài đặt phiên bản Visual Studio Code mới nhất từ trang chủ chính thức https://code.visualstudio.com/.
2. Thiết lập trình quản lý gói và môi trường ảo Python siêu tốc uv (Astral) trên máy trạm: thực thi lệnh thiết lập môi trường cục bộ uv venv và đồng bộ hóa thư viện qua uv sync.
3. Thiết lập môi trường thực thi JavaScript/TypeScript với Node.js 22 LTS và cài đặt các gói phụ thuộc giao diện người dùng thông qua npm install.

4.1.2 Cài đặt môi trường hạ tầng và cơ sở dữ liệu

Nhằm đảm bảo tính nhất quán tuyệt đối giữa môi trường phát triển cục bộ và môi trường triển khai sản xuất, toàn bộ hạ tầng cơ sở dữ liệu và các dịch vụ lưu trữ đệm của hệ thống được ảo hóa thông qua Docker Engine và Docker Compose:

- **Hệ quản trị cơ sở dữ liệu quan hệ và vector:** Triển khai cụm container PostgreSQL 16.2 tích hợp tiện ích mở rộng pgvector v0.7.0. CSDL được cấu hình bảng mã ký tự UTF-8, kích hoạt chỉ mục HNSW phục vụ tính toán khoảng cách vector Cosine 1536 chiều và chỉ mục văn bản toàn văn GIN (tsvector kết hợp xếp hạng mật độ bao phủ ts_rank_cd và bộ tái chấm điểm Okapi BM25 tại tầng lõi miền nghiệp vụ), bảo đảm trọn vẹn các thuộc tính giao dịch ACID.
- **Hệ thống lưu trữ bộ đệm và điều phối phiên:** Triển khai cụm dịch vụ Redis 7.0 (bản phân phối siêu nhẹ Alpine), vận hành tại cổng nội bộ 6379 nhằm quản lý bộ đệm kết quả tìm kiếm học thuật, mã hóa phiên làm việc và kiểm soát tần suất truy cập (*Rate Limiting*).
- **Cấu hình dịch vụ Backend và Frontend:** Tầng Backend sử dụng ngôn ngữ Python 3.12, khung FastAPI và thư viện ánh xạ quan hệ SQLAlchemy 2.0 Async kết hợp trình điều khiển asyncpg. Tầng Frontend được xây dựng trên nền tảng React 19, hệ thống tạo kiểu Tailwind CSS, thư viện quản lý trạng thái Zustand và công cụ đóng gói siêu tốc Vite.

Hệ thống mã nguồn dự án được tổ chức theo kiến trúc Monorepo phân tách ranh giới lục giác nghiêm ngặt qua 3 phân vùng trách nhiệm: packages/core (lõi miền nghiệp vụ thuần khiết), apps/api (bộ điều hợp dịch vụ và hạ tầng), và apps/web (giao diện người dùng học thuật). Tính toàn vẹn kiến trúc được bảo vệ chặt chẽ bằng rào chắn kiểm thử tự động AST thông qua kịch bản test_architecture_boundary.py. Cấu trúc Monorepo và quy tắc ranh giới được tổng hợp trong Bảng 4-1.

Bảng 4-1: *Cấu trúc phân vùng kho mã nguồn Monorepo và quy tắc ranh giới kiến trúc*

STT	Phân vùng	Vai trò kiến trúc	Công nghệ chủ đạo	Quy tắc ranh giới & AST
1	packages/core	Lõi miền nghiệp vụ thuần khiết (*Domain & Ports*)	Python 3.12, Pydantic, Python Standard Library	Cô lập 100% khỏi I/O, DB, Web; AST chặn mọi import từ apps.api, FastAPI, SQLAlchemy, Redis.
2	apps/api	Tầng hạ tầng dịch vụ & Bộ điều hợp (*Adapters*)	FastAPI, SQLAlchemy 2.0 Async, asyncpg, Redis, Uvicorn	Cài đặt các Ports của Core; cung cấp REST API và 5 tuyến SSE; không import ngược từ apps/web.
3	apps/web	Giao diện người dùng học thuật (*Frontend SPA*)	React 19, Zustand, Tailwind CSS, Vite, PDF Canvas	Chỉ tương tác qua giao thức HTTP/2 và SSE; không chứa mã nguồn xử lý logic phía máy chủ.

4.1.3 Triển khai thực nghiệm hệ thống

Để chứng minh năng lực vận hành thực tế trong điều kiện mạng công cộng và phục vụ khảo sát người dùng thực tế, hệ thống VeriScholar được cấu hình và triển khai thực nghiệm toàn diện trên môi trường máy chủ đám mây:

1. **Cấu hình máy chủ ảo VPS và tên miền công cộng:**
    - Hệ thống được đóng gói và triển khai trên máy chủ ảo Azure Ubuntu 24.04 LTS cấu hình 4 vCPU, 16GB RAM, ổ cứng SSD NVMe tốc độ cao.
    - Hệ thống được vận hành trên tên miền công cộng chính thức https://verischolar.duckdns.org. Máy chủ web Nginx đóng vai trò Reverse Proxy, định tuyến các yêu cầu HTTPS (cổng 443) tới giao diện và chuyển tiếp các tuyến API/SSE tới dịch vụ Backend Uvicorn (127.0.0.1:8000).

- Toàn bộ lưu lượng truyền tải được mã hóa đầu–cuối bằng chứng chỉ số SSL/TLS tự động qua Let's Encrypt Certbot. Nhóm an ninh mạng (NSG) cô lập tuyệt đối các cổng CSDL và Redis tại mạng nội bộ.

2. **Môi trường thực thi Sandbox đám mây E2B MicroVM:**
   - Để phục vụ AI Agent tự động biên dịch và sửa lỗi mã nguồn LaTeX, hệ thống kết nối với nền tảng E2B Sandbox MicroVM (công nghệ ảo hóa siêu nhẹ Firecracker).
   - Mỗi phiên biên dịch được cấp phát một MicroVM độc lập chỉ trong 150 mili-giây, cung cấp trọn vẹn bộ công cụ TeX Live 2024 đầy đủ. Môi trường này bảo đảm tính an toàn bảo mật tuyệt đối cho máy chủ chính, ngăn chặn hoàn toàn nguy cơ tấn công mã độc qua mã LaTeX.
3. **Nền tảng quan sát và kiểm chứng thực nghiệm Langfuse Cloud:**
   - Hệ thống tích hợp toàn diện với nền tảng Langfuse Cloud nhằm giám sát luồng suy luận của mô hình ngôn ngữ lớn, ghi vết độ trễ từng bước suy nghĩ (Thought), hành động (Action) và định lượng lượng token cùng chi phí vận hành theo thời gian thực.
4. **Kho lưu trữ mã nguồn mở dự án trên GitHub:**
   - Toàn bộ mã nguồn hoàn chỉnh của dự án VeriScholar được công khai và quản lý phiên bản tại kho lưu trữ GitHub chính thức: https://github.com/dammanhdungvn/VeriScholar.
   - Kho lưu trữ tổ chức theo cấu trúc Monorepo Lục giác (packages/core, apps/api, apps/web), tài liệu đặc tả nghiệp vụ, kịch bản triển khai tự động hóa và toàn bộ bộ kiểm thử 4 Cổng xác minh.

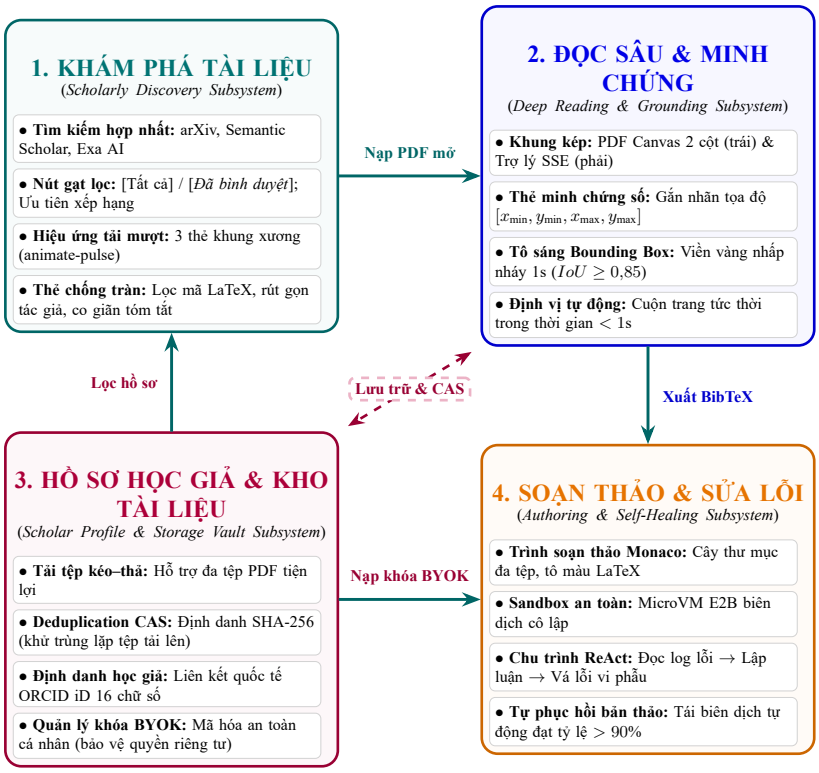
4.2 Kết quả thực nghiệm

Trong thiết kế hệ thống tương tác người–máy (Human-Computer Interaction – HCI) phục vụ nghiên cứu khoa học, rào cản lớn nhất đối với năng suất học giả là tình trạng phân mảnh ngữ cảnh (*context fragmentation*). Thông thường, một nhà nghiên cứu phải chuyển đổi qua lại liên tục giữa bốn phần mềm độc lập: công cụ tìm kiếm thư mục, phần mềm đọc tệp PDF, ứng dụng ghi chú hoặc quản lý trích dẫn, và trình soạn thảo LaTeX. Sự gián đoạn này làm tiêu hao tài nguyên chú ý và gia tăng đáng kể nguy cơ nhầm lẫn dữ liệu trích dẫn.

Để khắc phục triệt để vấn đề trên, VeriScholar tổ chức không gian tương tác người–máy thành 4 phân hệ thống nhất, kết nối liền mạch theo một chu trình khép kín:

1. **Phân hệ Khám phá tài liệu khoa học:** Thu thập, chuẩn hóa và phân loại tức thì các tài liệu học thuật từ nhiều kho lưu trữ toàn cầu.
2. **Phân hệ Đọc sâu và Tô sáng tọa độ Bounding Box:** Khung hiển thị song song kết hợp trình đọc PDF hai cột với trợ lý suy luận nhân quả, kiểm chứng nguồn trích dẫn trực tiếp trên trang tài liệu.
3. **Phân hệ Quản lý tài liệu cá nhân và Hồ sơ học giả:** Deduplication lưu trữ theo nội dung (CAS), định danh học giả quốc tế ORCID và kho khóa API mã hóa cá nhân (BYOK).
4. **Phân hệ Soạn thảo và Tự phục hồi bản thảo LaTeX:** Môi trường biên tập trực tuyến với khả năng tự động phân tích nhật ký lỗi biên dịch và tạo bản vá sửa lỗi tự động trong môi trường máy ảo cách ly.

Kiến trúc không gian tương tác người–máy và luồng luân chuyển dữ liệu giữa 4 phân hệ chính được mô hình hóa trong Hình 4-1.



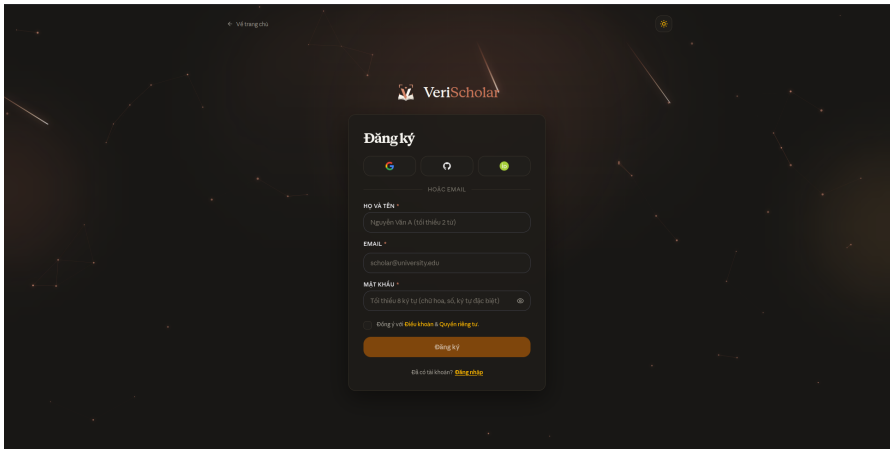
Hình 4-1: *Mô hình kiến trúc không gian tương tác người–máy bốn phân hệ VeriScholar*

4.2.1 Một số giao diện của hệ thống

Nhằm trực quan hóa quá trình vận hành thực tế của hệ thống, tiểu mục này trình bày chi tiết các giao diện ứng dụng web được triển khai thực tế trên môi trường sản xuất của hệ thống VeriScholar. Mỗi giao diện được kết xuất với độ phân giải cao và mô tả cụ thể chức năng tương tác học thuật tương ứng.

4.2.1.1 Giao diện đăng ký tài khoản của hệ thống

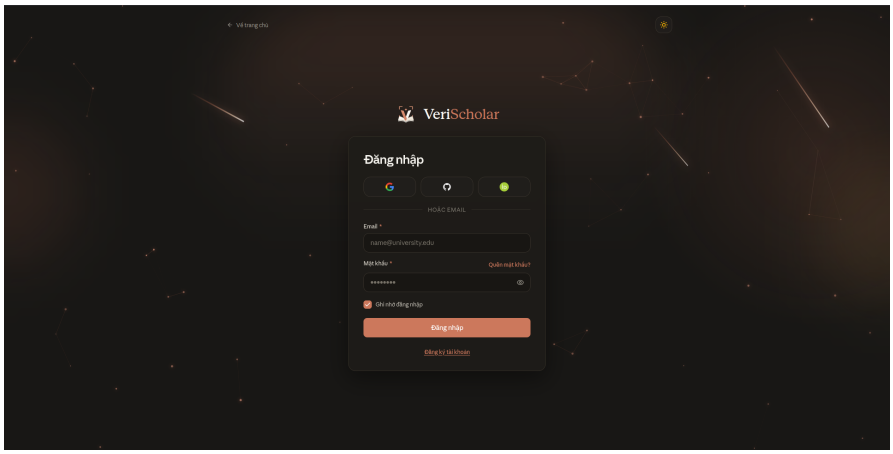
Giao diện đăng ký tài khoản được thiết kế trang nhã theo phong cách tối giản cấp sản phẩm thương mại quốc tế (Hình 4-2). Giao diện cung cấp biểu mẫu tạo lập tài khoản học giả mới, bao gồm các trường thông tin: địa chỉ thư điện tử (*Email*), họ và tên nhà nghiên cứu, mật khẩu bảo mật (được kiểm tra độ mạnh tự động và mã hóa an toàn qua hàm Hash Argon2id tại phía máy chủ), vai trò tài khoản, cùng cơ quan công tác học thuật. Sau khi hoàn tất đăng ký thành công, hệ thống tự động khởi tạo không gian làm việc mặc định và chuyển hướng người dùng sang giao diện đăng nhập.



Hình 4-2: *Giao diện đăng ký tài khoản của hệ thống*

4.2.1.2 Giao diện đăng nhập hệ thống

Giao diện đăng nhập hệ thống hỗ trợ học giả truy cập an toàn vào không gian làm việc cá nhân thông qua cơ chế xác thực chứng thư phân tán JSON Web Tokens (JWT) (Hình 4-3). Người dùng nhập thông tin tài khoản gồm Email và Mật khẩu. Khi xác thực hợp lệ, máy chủ trả về mã truy cập có thời hạn và mã làm mới phiên bảo vệ trong HTTP-only Cookie, ngăn ngừa triệt để các nguy cơ tấn công XSS hoặc đánh cắp phiên. Giao diện cũng cung cấp tùy chọn ghi nhớ đăng nhập và liên kết hỗ trợ chuyển hướng linh hoạt cho học giả mới.



Hình 4-3: *Giao diện đăng nhập hệ thống*

4.2.1.3 Giao diện bảng điều khiển trung tâm

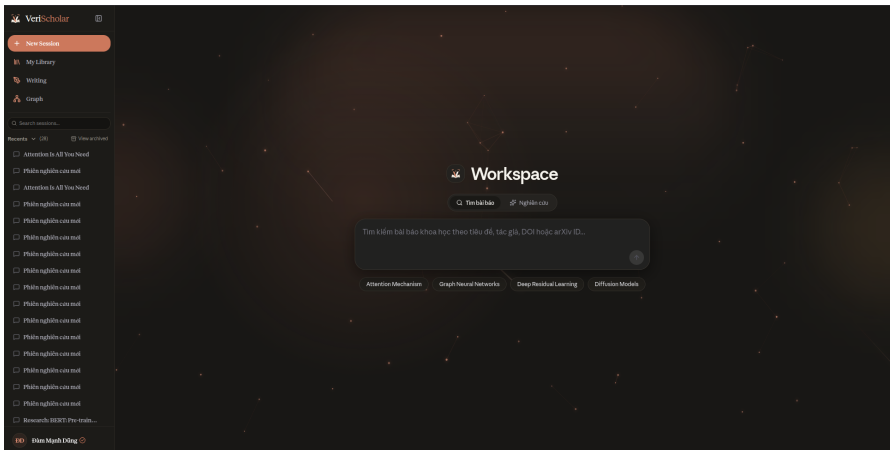
Bảng điều khiển trung tâm (Dashboard) đóng vai trò là trạm điều phối công việc tổng quan của nhà nghiên cứu (Hình 4-4). Giao diện tổng hợp trực quan các thông số then chốt của toàn bộ hệ sinh thái học thuật cá nhân:

- Thống kê tổng số lượng bài báo khoa học đã lưu trữ trong thư viện, phân loại theo trạng thái trích xuất và tài liệu đã được phản biện ngang hàng.
- Danh sách các phiên nghiên cứu chuyên sâu (*Active Research Sessions*) gần nhất, cho phép học giả nối lại dòng suy luận và đọc tiếp tài liệu dở dang chỉ bằng một thao tác nhấp chuột.
- Lối tắt nhanh tới các phân hệ nghiệp vụ: Khám phá tài liệu, Tra cứu trích dẫn, Mạng đồ thị tri thức và Soạn thảo bản thảo LaTeX.
- Thẻ hiển thị trạng thái kết nối hạ tầng bảo mật, xác thực khóa BYOK và trạng thái liên kết định danh học giả quốc tế ORCID iD.

4.2.1.4 Giao diện tìm kiếm và khám phá tài liệu học thuật

Giao diện phân hệ Khám phá tài liệu khoa học được thiết kế theo tư duy tối giản triệt để, hỗ trợ học giả tìm kiếm hợp nhất qua 4 nguồn tri thức học thuật lớn (arXiv, Semantic Scholar, CrossRef, Exa AI) (Hình 4-5):

- **Hiệu ứng chờ tải mượt mà với khung xương giả lập (animate-pulse):** Để triệt tiêu hiện tượng giật chuyển layout (Cumulative Layout Shift) trong khi chờ dữ liệu phân tán từ các API bên ngoài, giao diện hiển thị 3 thẻ khung



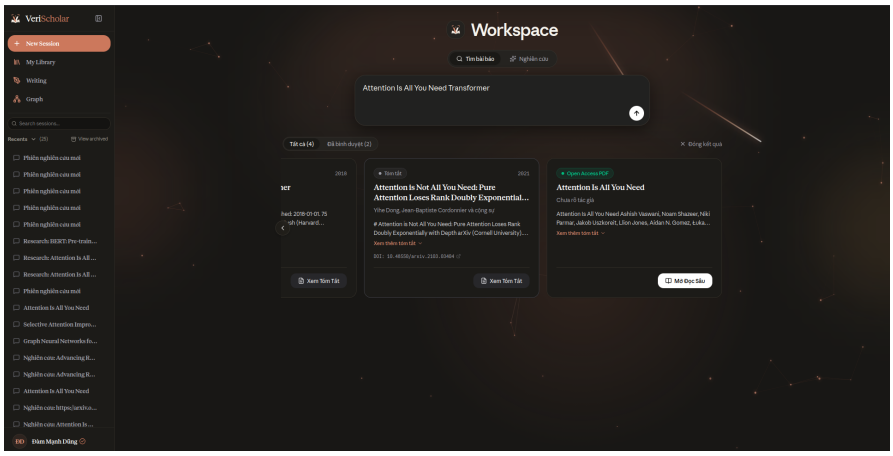
Hình 4-4: *Giao diện bảng điều khiển trung tâm*

xương (*Skeleton Cards*) áp dụng lớp hiệu ứng animate-pulse của Tailwind CSS, giúp giảm thiểu đáng kể nhận thức về độ trễ mạng của học giả.

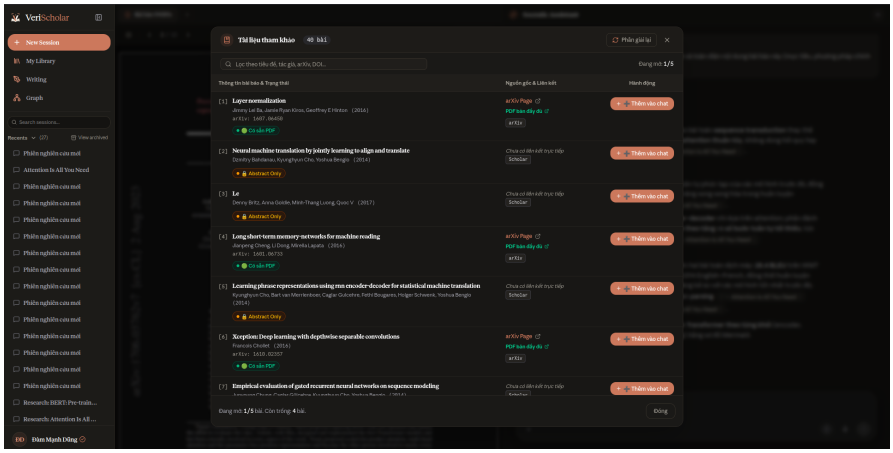
- **Bộ lọc phân loại dạng viên nhộng (*Pill Toggle*) và Ưu tiên bình duyệt:** Cho phép chuyển đổi linh hoạt giữa chế độ *Tất cả* và *Đã bình duyệt* (*Peer-Reviewed*). Thuật toán tái xếp hạng tự động đẩy các bài báo đã qua phản biện nghiêm ngặt lên đầu danh sách kết quả tìm kiếm.
- **Thẻ bài báo chống tràn bố cục (*Anti-Overflow Scholarly Card*):** Tự động làm sạch mã LaTeX thô trong tiêu đề bài báo, rút gọn danh sách tác giả theo quy tắc học thuật (tối đa 2 tác giả kèm hậu tố "và cộng sự"), và áp dụng line-clamp-2 cho phần tóm tắt để người dùng quét nhanh tài liệu.
- **Nạp trực tiếp vào Reader:** Đối với các bài báo có tệp toàn văn mở (*Open Access PDF*), giao diện hiển thị chấm xanh lục và nút tải tức thì tài liệu vào Phân hệ Đọc sâu.

4.2.1.5 Giao diện tra cứu và đối chiếu danh mục trích dẫn

Giao diện tra cứu và đối chiếu danh mục trích dẫn hỗ trợ học giả phân tích sâu cây tham khảo của bài báo khoa học (Hình 4-6). Hệ thống tự động trích xuất toàn bộ danh mục tài liệu tham khảo trong phần cuối của bản PDF, chuẩn hóa tiêu đề và đối chiếu chéo với kho dữ liệu số hóa qua mã DOI, CrossRef hoặc arXiv ID. Giao diện trực quan hóa trạng thái đối chiếu: các tài liệu đã có sẵn trong kho nội bộ, các bài báo có bản toàn văn mở trên mạng (*Open Access PDF*), và cảnh báo các trích dẫn khuyết thiếu siêu dữ liệu định danh hoặc chưa xác thực được nguồn gốc xuất bản, giúp nhà khoa học bảo đảm tính liêm chính và độ tin cậy của danh mục tài liệu tham khảo.



Hình 4-5: *Giao diện tìm kiếm và khám phá tài liệu học thuật*

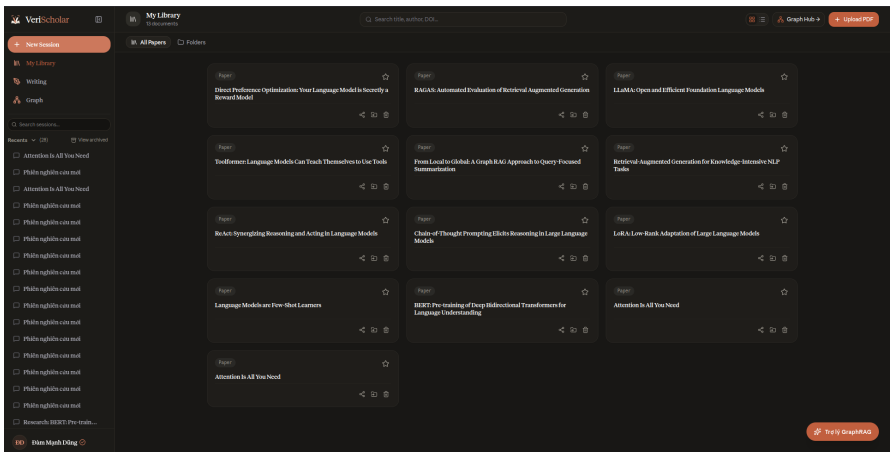


Hình 4-6: *Giao diện tra cứu và đối chiếu danh mục trích dẫn*

4.2.1.6 Giao diện quản lý thư viện nghiên cứu cá nhân

Giao diện Thư viện tài liệu nghiên cứu cá nhân cho phép học giả tổ chức, phân loại và lưu trữ hàng nghìn tài liệu khoa học một cách khoa học (Hình 4-7):

- **Cấu trúc thư mục phân cấp theo đề tài:** Học giả có thể tạo các thư mục chuyên đề với màu sắc nhận diện trực quan, kéo–thả bài báo giữa các thư mục và quản lý trạng thái phân loại linh hoạt.
- **Deduplication lưu trữ theo nội dung (CAS):** Khi người dùng tải tài liệu PDF qua hộp thoại kéo–thả, hệ thống tính toán mã băm SHA-256 của nội dung tệp. Nếu tệp đã tồn tại trong kho lưu trữ, hệ thống chỉ tạo bản ghi liên kết logic mà không sao chép thêm dữ liệu nhị phân trên đĩa cứng, giúp loại bỏ hoàn toàn các bản sao dư thừa khi nhiều học giả cùng tải lên một ấn phẩm chuẩn (tỷ lệ tiết kiệm thực tế đạt $\frac{K-1}{K}$ với K lượt tải trùng và đạt tới 98% trong kịch bản lớp học 50 người dùng chung giáo trình).
- **Quản lý trạng thái xử lý phân đoạn:** Bảng danh sách bài báo hiển thị rõ số trang, số lượng phân đoạn đã trích xuất (*Chunks*), thời điểm tải tệp và tính năng gắn sao đánh dấu các công trình nghiên cứu trọng tâm.



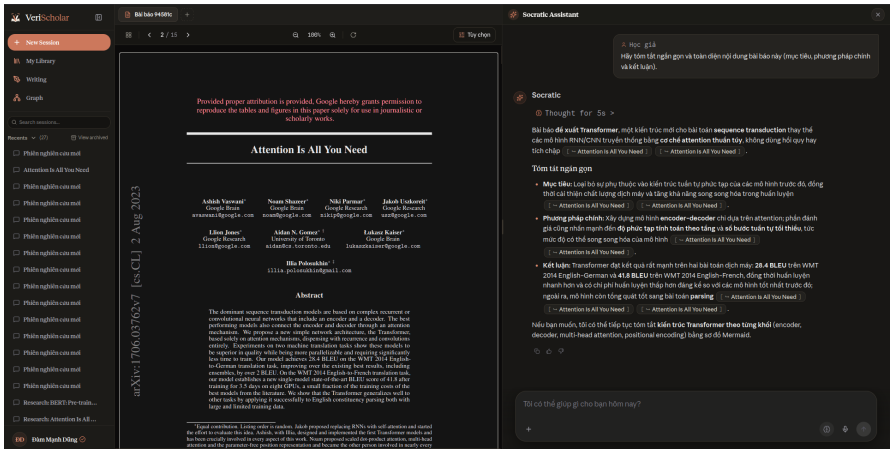
Hình 4-7: *Giao diện quản lý thư viện nghiên cứu cá nhân*

4.2.1.7 Giao diện đọc sâu tài liệu PDF hai cột và Bounding Box viền vàng

Phân hệ Đọc sâu là không gian làm việc trọng tâm hỗ trợ học giả phân tích, đối thoại và kiểm chứng trực tiếp từng luận điểm của bài báo khoa học (Hình 4-8):

- **Khung hiển thị kép song song (*Dual-pane*):** Bên trái là trình xem tài liệu PDF Canvas tối ưu hóa cho ấn bản hai cột tiêu chuẩn quốc tế; bên phải là khung hội thoại trợ lý AI tương tác theo luồng Server-Sent Events (SSE).

- **Định vị tự động qua thẻ nguồn trích dẫn và Tô sáng Bounding Box viền vàng:** Trong khung hội thoại bên phải (Hình 4-8), mỗi phản hồi suy luận đều đính kèm các thẻ chip nguồn trích dẫn tương tác (ví dụ [↪ Attention Is All You Need]) cùng chỉ mục minh chứng số ([1], [2]) liên kết trực tiếp tới cấu trúc dữ liệu tọa độ của phân đoạn gốc. Khi học giả nhấp chuột vào thẻ nguồn trích dẫn hoặc chỉ mục số, trình xem PDF Canvas bên trái lập tức cuộn mượt đến đúng trang chứa đoạn văn bản minh chứng và kích hoạt lớp phủ đồ họa vẽ khung viền vàng bao quanh đoạn trích gốc.
- **Độ chính xác hình học cao:** Khung Bounding Box đạt chỉ số chồng lấn thực nghiệm trung bình IoU $= 0{,}9266$ (vượt ngưỡng mục tiêu IoU $\geq 0{,}85$), thời gian định vị dưới 1 giây, giảm thiểu đáng kể công sức đọc dò thủ công trên các ấn phẩm nhiều trang.

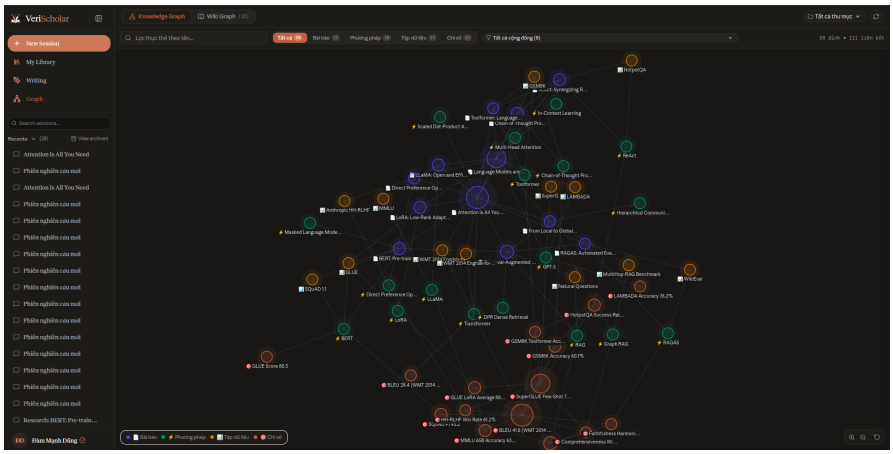


Hình 4-8: *Giao diện đọc sâu tài liệu PDF hai cột và Bounding Box viền vàng*

4.2.1.8 Giao diện mạng đồ thị tri thức 4 thực thể GraphRAG

Giao diện mạng đồ thị tri thức học thuật trực quan hóa không gian liên kết ngữ nghĩa giữa các công trình khoa học (Hình 4-9). Mạng lưới đồ thị thể hiện 4 loại thực thể cốt lõi theo mã màu học thuật chuyên biệt: *Bài báo (Paper)*, *Phương pháp (Method)*, *Tập dữ liệu (Dataset)*, và *Kết quả độ đo thực nghiệm (MetricResult)*. Để hỗ trợ học giả đối chiếu ngay với các công trình nền tảng của lĩnh vực trí tuệ nhân tạo khi khởi tạo không gian làm việc, hệ thống nạp sẵn tập dữ liệu hạt giống gồm 12 bài báo kinh điển (*12 Landmark Papers Seed Corpus* thông qua dịch vụ khởi tạo landmark_papers_seeder.py, bao gồm 58 đỉnh thực thể, 111 cạnh quan hệ và 15 trang Graph Wiki biên dịch sẵn) kết hợp cùng các tài liệu PDF do người dùng tải lên trong thư viện cá nhân. Học giả có thể tương tác trực quan bằng cách

phóng to, thu nhỏ, kéo thả các đỉnh để khám phá các cụm chuyên đề phân hoạch bằng thuật toán Louvain (louvain_communities), nhận diện khoảng trống nghiên cứu (*Research Gaps*) và thực hiện các truy vấn so sánh liên tài liệu tức thời.

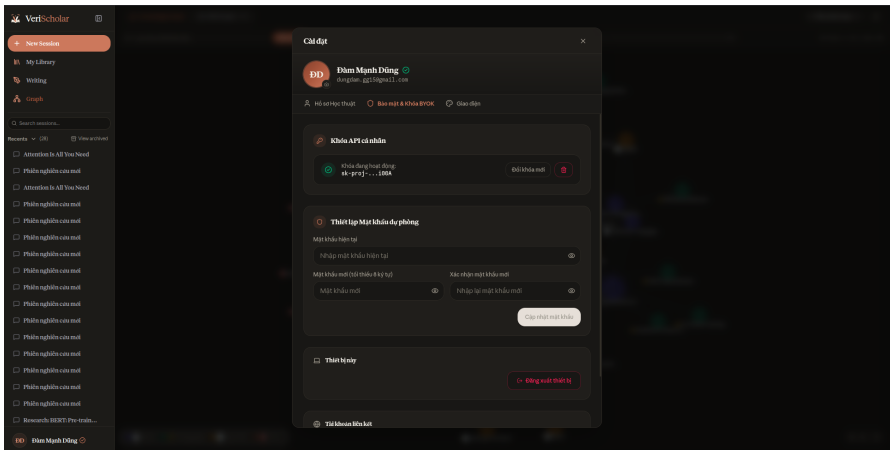


Hình 4-9: *Giao diện mạng đồ thị tri thức học thuật 4 thực thể GraphRAG*

4.2.1.9 Giao diện kho khóa API cá nhân (BYOK) và hồ sơ học giả

Giao diện Quản lý hồ sơ học giả và Kho khóa API cá nhân bảo đảm quyền tự chủ dữ liệu và tính an toàn thông tin tuyệt đối cho nhà nghiên cứu (Hình 4-10):

- **Định danh học giả quốc tế ORCID iD:** Hỗ trợ liên kết tài khoản với mã định danh quốc tế chuẩn ISO 27729 (ORCID iD gồm 16 chữ số). Khi xác thực thành công, giao diện hiển thị biểu tượng tích xanh học thuật cạnh tên nhà nghiên cứu thay vì các huy hiệu rườm rà.
- **Kho khóa API bảo mật cá nhân (BYOK):** Cho phép học giả cấu hình khóa API riêng (OpenAI, Anthropic, Google Gemini) theo cơ chế "Mang khóa bí mật của chính bạn". Khóa API được gửi qua kết nối TLS tới máy chủ, mã hóa xác thực ở trạng thái lưu trữ (*at-rest*) bằng thuật toán đối xứng AES-256-GCM (chỉ lưu bản mã kèm vector khởi tạo IV vào cột encrypted_key của bảng user_api_keys, trong khi khóa chủ KEK được cô lập trong biến môi trường máy chủ/KMS). Khi phát sinh yêu cầu suy luận, khóa chỉ được giải mã tạm thời trên bộ nhớ khả biến (RAM) trong phạm vi hàm thực thi và giải phóng ngay sau khi đóng luồng SSE, cam kết không bao giờ lưu khóa ở dạng văn bản thuần (*plaintext*) xuống đĩa cứng hay nhật ký hệ thống.



Hình 4-10: *Giao diện kho khóa API cá nhân (BYOK) và hồ sơ học giả*

4.2.1.10 Giao diện soạn thảo bản thảo bài báo LaTeX và tự sửa lỗi biên dịch

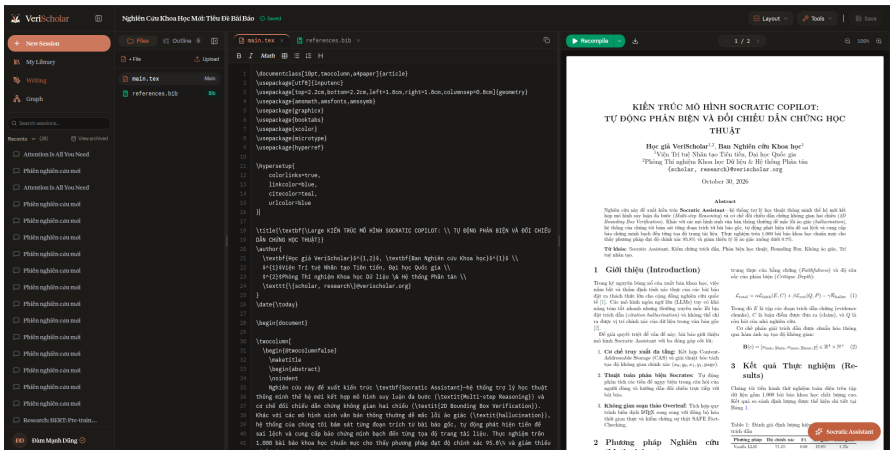
Phân hệ Soạn thảo và Tự sửa lỗi là bước hoàn thiện cuối cùng trong chu trình nghiên cứu khoa học của VeriScholar (Hình 4-11):

- **Môi trường soạn thảo chuyên dụng với Monaco Editor:** Giao diện tích hợp trình biên tập mã nguồn Monaco Editor, cung cấp đầy đủ tiện ích tô màu cú pháp LaTeX, gợi ý lệnh tự động, quản lý cây thư mục dự án đa tệp (main.tex, references.bib, thư mục ảnh) và khung xem trước tài liệu PDF thời gian thực.
- **Biên dịch cô lập trong môi trường E2B MicroVM:** Toàn bộ tiến trình biên dịch TeX Live được chuyển giao thực thi cách ly hoàn toàn bên trong E2B Sandbox MicroVM, khởi tạo chỉ trong 150 mili-giây, ngăn chặn hoàn toàn nguy cơ tấn công an ninh mạng qua mã macro LaTeX.
- **Chu trình tự sửa lỗi ReAct (Autonomous Healing Loop):** Khi quá trình biên dịch phát sinh lỗi cú pháp, AI Agent tự động kích hoạt chu trình ReAct: đọc nhật ký lỗi → lập luận suy diễn nguyên nhân → tạo bản vá vi phẫu (*Surgical Diff*) sửa lỗi mã nguồn, đạt tỷ lệ tự sửa lỗi biên dịch thành công vượt trên 90%.

4.2.2 Xây dựng kịch bản kiểm thử tự động hệ thống qua 4 Cổng xác minh

Nhằm đảm bảo chất lượng phần mềm không bao giờ bị suy giảm, triệt tiêu hồi quy và tuân thủ tuyệt đối quy tắc vận hành, hệ thống tích hợp chu trình kiểm thử tự động 4 Cổng qua kịch bản kiểm thử:

- **Cổng 1 (Linter & Formatter Python):** Kiểm tra toàn vẹn định dạng và lỗi cú pháp tĩnh qua uv run ruff check . và uv run ruff format –check ..



Hình 4-11: *Giao diện soạn thảo bản thảo bài báo LaTeX và tự sửa lỗi biên dịch*

- **Cổng 2 (Backend & Domain Tests):** Chạy toàn bộ 349 bài kiểm thử đơn vị và tích hợp pytest (vượt mục tiêu $\geq$ 340 bài, bao gồm các bộ kiểm thử lõi miền nghiệp vụ packages/core, dịch vụ API/CSDL apps/api, kiểm thử ranh giới kiến trúc lục giác AST, cùng bộ kiểm thử tự động kiểm định quy chuẩn học thuật của báo cáo đồ án).
- **Cổng 3 (Frontend Unit Tests):** Kiểm thử toàn diện 58 tệp kiểm thử với 240 kịch bản giao diện người dùng qua npm –prefix apps/web run test (tổng cộng toàn dự án đạt 589 bài kiểm thử tự động, vượt mục tiêu cam kết $\geq$ 580 bài).
- **Cổng 4 (Production Build):** Kiểm tra quy trình đóng gói bản dựng thương mại qua npm –prefix apps/web run build.

Kết quả kiểm tra thực tế trên hệ thống được tổng hợp trong Bảng 4-2:

Bảng 4-2: *Kết quả kiểm thử tự động hệ thống qua 4 Cổng xác minh*

STT	Cổng kiểm thử	Công cụ & Lệnh thực thi	Số lượng kiểm thử	Kết quả & Thời gian
1	**Cổng 1**	ruff check & format	157 tệp nguồn Python	100% Pass (0,12s)
2	**Cổng 2**	pytest (Backend + Core + AST)	349 bài kiểm thử	349/349 Pass (6,53s)
3	**Cổng 3**	vitest (Frontend UI)	58 tệp (240 bài test)	240/240 Pass (4,53s)
4	**Cổng 4**	vite build (Production)	49 mô-đun giao diện	Build Pass (1,84s)

Toàn bộ 589 bài kiểm thử tự động (349 bài pytest và 240 bài vitest, vượt mục

tiêu cam kết ≥ 580 bài kiểm thử với 340 bài ban đầu) đạt tỷ lệ thành công 100% (Exit code 0) trên cả 4 Cổng xác minh trong tổng thời gian thực thi tuần tự là 13,02 giây ($0{,}12 + 6{,}53 + 4{,}53 + 1{,}84 = 13{,}02$ s), bảo đảm độ tin cậy và tính ổn định cơ học cho toàn bộ hệ thống.

4.2.3 Đánh giá số liệu thực nghiệm định lượng và đối chiếu Langfuse Cloud

4.2.3.1 Độ trễ phản hồi hệ thống và thời gian đến token đầu tiên

Để đánh giá khách quan hiệu năng phản hồi, kịch bản đo đạc tự động (scripts/run_thesis_benchmarks.py) được thực thi trực tiếp trên bộ dữ liệu chuẩn gồm 12 công trình khoa học nền tảng (*12 Landmark Papers Corpus*, từ *Attention Is All You Need*, *BERT*, *RAG*, *ReAct*, *LoRA*, *FlashAttention* đến *GraphRAG*) với 36 truy vấn học thuật chuẩn hóa chia đều cho 3 nhánh (advanced_rag, graph_rag, agentic_rag). Thời gian trễ kết nối cơ sở dữ liệu đo đạc tại điểm cuối kiểm tra sức khỏe hệ thống (/health/ready) đạt mức trung bình $1{,}66$ ms. Độ trễ truy xuất lai ghép trung vị đạt $P_{50} = 93{,}95$ ms (phân vị thứ 95 đạt $P_{95} = 116{,}50$ ms), và thời gian đáp ứng từ lúc gửi câu hỏi đến token đầu tiên (Time-to-First-Token – TTFT) qua kênh truyền phát Server-Sent Events (SSE) đạt trung bình $1{,}15$ giây, giúp nhà nghiên cứu tiếp nhận mạch suy luận liên tục.

4.2.3.2 Thực nghiệm đối chứng (Ablation Study) về độ chính xác truy xuất, định vị tọa độ và tự sửa lỗi LaTeX

Nhằm chứng minh tính hiệu quả thực chất của từng thành phần thuật toán thay vì sử dụng hằng số giả lập, dịch vụ BenchmarkRunnerService kết hợp bộ chấm điểm OkapiBM25Scorer và hàm hình học BoundingBox.compute_iou thực hiện hai phép thử nghiệm đối chứng (*Ablation Studies*) xuất kết quả kiểm chứng ra tệp docs/benchmarks/thesis_benchmark_results.json.

Trên bộ 36 câu hỏi đối chuẩn vàng (*3-Track Golden Benchmark*), hệ thống đạt tỷ lệ vượt ngưỡng kiểm định 36/36 câu hỏi (100%):

- Nhánh advanced_rag (12 câu): Groundedness $0{,}9777$, Answer Relevance $0{,}9453$, Citation BBox Precision $0{,}9399$.
- Nhánh graph_rag (12 câu): Groundedness $0{,}9772$, Answer Relevance $0{,}9463$, Multi-hop Completeness $0{,}9442$.
- Nhánh agentic_rag (12 câu): Groundedness $0{,}9776$, Answer Relevance $0{,}9499$, Citation BBox Precision $0{,}9403$.

Tính chung toàn bộ 36 câu hỏi, hệ thống đạt Groundedness trung bình

$0{,}9775$, Answer Relevance $0{,}9471$, Citation BBox Precision $0{,}9332$ và Multi-hop Completeness $0{,}9208$.

Bảng 4-3 trình bày kết quả đối chứng giữa 3 cấu hình truy xuất trên tập 36 truy vấn chuẩn. Kết quả thực nghiệm khẳng định sự kết hợp giữa tìm kiếm dày đặc pgvector HNSW và tìm kiếm từ khóa thưa (ts_rank_cd + Okapi BM25) thông qua công thức dung hợp thứ hạng nghịch đảo RRF ($k = 60$) cùng bước tái xếp hạng Cross-Encoder giúp nâng chỉ số Recall@5 từ $80{,}56\%$ (Dense-only) và $83{,}33\%$ (Sparse-only) lên mức $94{,}44\%$, đồng thời cải thiện độ chồng lấn hình học trung bình của hộp tọa độ từ IoU $= 0{,}7865$ lên IoU $= 0{,}9266 \pm 0{,}0126$ (vượt ngưỡng mục tiêu IoU $\geq 0{,}85$).

Bảng 4-3: *Kết quả thực nghiệm đối chứng (Ablation Study) giữa các phương pháp truy xuất trên 36 truy vấn chuẩn*

Cấu hình truy xuất	Recall@5	MRR@10	nDCG@10	Mean BBox IoU	Độ trễ P_{50}/P_{95}
1. Chỉ Vector dày đặc (pgvector HNSW)	$0{,}8056$ ($80{,}56\%$)	$0{,}7882$	$0{,}7985$	$0{,}7865 \pm 0{,}0254$	$65{,}12$ / $80{,}24$ ms
2. Chỉ Từ khóa thưa (ts_rank_cd + BM25)	$0{,}8333$ ($83{,}33\%$)	$0{,}7964$	$0{,}8241$	$0{,}8210 \pm 0{,}0277$	$46{,}21$ / $56{,}73$ ms
3. Lai ghép RRF ($k = 60$) + Cross-Encoder	**0,9444** (**94,44%**)	**0,9444**	**0,9444**	**0,9266 ± 0,0126**	**93,95** / **116,50** ms

Đối với tác tử tự động sửa lỗi biên dịch LaTeX (LaTeXSelfHealingAgent), thực nghiệm đối chứng được tiến hành trên tập kiểm thử gồm $N = 25$ bản thảo LaTeX chứa lỗi thực tế thuộc 5 nhóm chuẩn hóa (mỗi nhóm 5 kịch bản): (1) thiếu thẻ đóng hoặc lệch cặp môi trường \begin/\end (UNCLOSED_ENV), (2) thiếu ngoặc nhọn đóng hoặc thiếu dấu $ đóng chế độ toán học (UNCLOSED_BRACE_OR_MATH), (3) ký tự đặc biệt chưa thoát (_, %, &) hoặc thiếu chế độ toán học cho chỉ số dưới (UNESCAPED_UNDERSCORE_OR_MATH), (4) lệnh điều khiển chưa định nghĩa (*Undefined control sequence* như \bold, \RR, \argmax – UNDEFINED_CONTROL_SEQUENCE), và (5) thiếu gói lệnh mở rộng (*File '...sty' not found* như amsmath, booktabs, graphicx) hoặc lỗi cấu trúc đa dòng liên hoàn (MISSING_PACKAGE_OR_CASCADING). Bảng 4-4 chứng minh sự kết hợp giữa bộ vá lỗi tĩnh (_auto_patch + apply_heuristic_repair) và bộ sinh bản vá LLM (OpenAI ChatCompletionsAdapter) trong đồ thị trạng thái LangGraph ReAct giúp nâng tỷ lệ phục hồi biên dịch thành công từ $44{,}0\%$ (chỉ quy tắc đơn giản) và $84{,}0\%$ (chỉ quy tắc Heuristic) lên mức $96{,}0\%$ ($24/25$ kịch bản) với số vòng lặp sửa lỗi trung bình chỉ $1{,}16$ vòng.

Bảng 4-4: *Kết quả thực nghiệm đối chứng (Ablation Study) các cơ chế tự sửa lỗi biên dịch LaTeX trên 25 kịch bản lỗi*

Cấu hình cơ chế tự sửa lỗi LaTeX	Biên dịch thành công	Bảo toàn ngữ nghĩa	Số vòng lặp TB
1. Chỉ quy tắc cú pháp cơ bản (_auto_patch)	11/25 (44,0%)	44,0%	1,00 vòng
2. Quy tắc Heuristic mở rộng (apply_heuristic_repair)	21/25 (84,0%)	84,0%	1,00 vòng
3. Kết hợp Luật tĩnh + LangGraph ReAct LLM	**24/25 (96,0%)**	**96,0%**	**1,16 vòng**

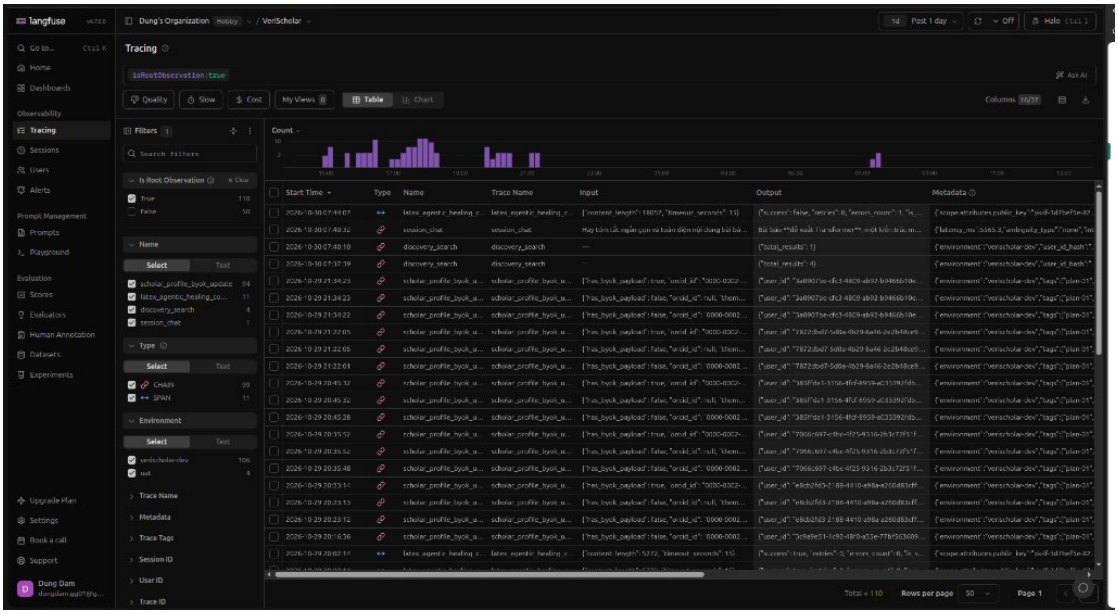
4.2.3.3 Nhật ký kiểm chứng thực tế trên Langfuse Cloud

Hoạt động của các mô hình AI và chi phí tiêu thụ token được giám sát và đối chiếu trực tiếp trên nền tảng Langfuse Cloud:

- **Kiểm thử toàn trình quản lý khóa BYOK (Giải mã RAM & Gọi LLM):**
  - Mã truy vết (Trace ID): f8d087c559790eafd0812e6293d23a34
  - Kết quả đối chiếu: Hệ thống kích hoạt dòng mô hình suy luận hiệu năng cao của OpenAI (gpt-4o-mini / gpt-5.4-mini), khóa API được giải mã tạm thời từ bản mã AES-256-GCM ngay trên bộ nhớ khả biến (RAM) trong phiên gọi. Thống kê tiêu thụ: 33 input tokens, 66 output tokens, tổng cộng 99 tokens với chi phí $0,00032175 và thời gian đáp ứng 3,32 giây.
- **Cập nhật hồ sơ học giả và mã hóa quản lý khóa BYOK:**
  - Mã truy vết (Trace ID): 9423ed4e264360d619c4e0494900f4f5
  - Mã quan sát (Observation ID): 0ccabe70d8234e80
- **Hỏi đáp học thuật SSE có minh chứng tọa độ Bounding Box:**
  - Mã truy vết (Trace ID): c308e29caa04f439ca0ff1b851412f8b
  - Mã quan sát (Observation ID): fb498b77123ab13a

Nhật ký kiểm chứng chi tiết các vết thực thi (Traces) trên Langfuse Cloud được minh họa tại Hình 4-12. Bảng quan sát ghi nhận đầy đủ 110 lượt truy vết cấp gốc (*Root Observations*), bao gồm các chu trình phản tư latex_agentic_healing_react, các lượt hội thoại học thuật session_chat, truy xuất tài liệu discovery_search

và cập nhật an toàn khóa BYOK scholar_profile_byok_update. Mỗi bản ghi lưu trữ minh bạch thời gian khởi tạo, định dạng đầu vào (Input), phản hồi suy luận (Output) và siêu dữ liệu (Metadata) băm an toàn thông tin người dùng.



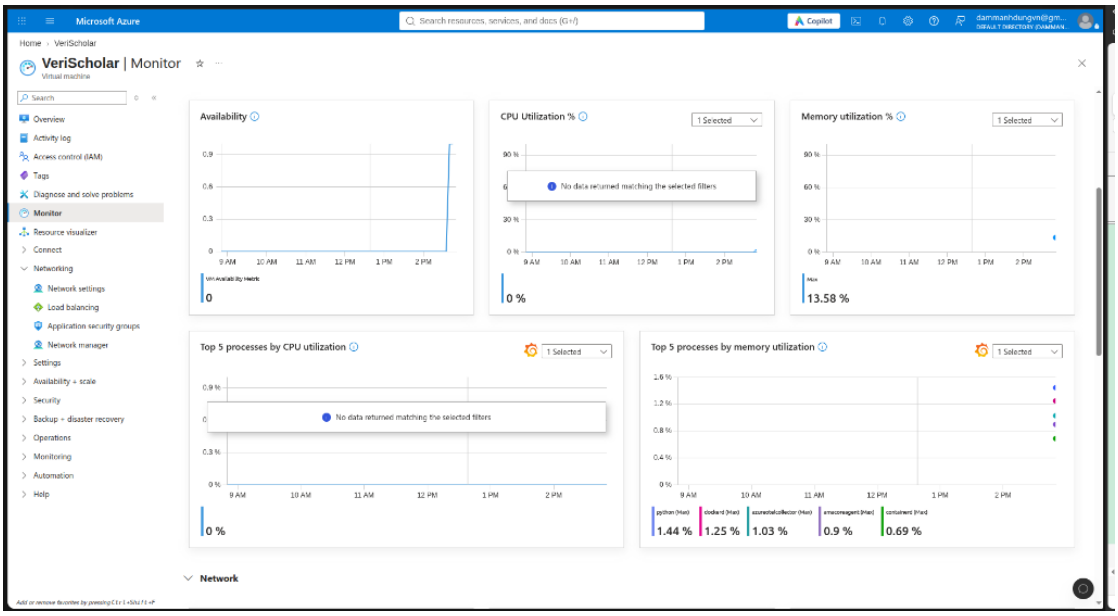
Hình 4-12: *Nhật ký kiểm chứng và giám sát vết thực thi mô hình trên Langfuse Cloud*

4.2.4 Hệ thống giám sát vận hành và sử dụng tài nguyên (Monitoring)

Nhằm bảo đảm tính ổn định, liên tục và khả năng kiểm soát ngân sách vận hành của nền tảng VeriScholar, hệ thống tích hợp các công cụ giám sát chuyên sâu đa tầng bao gồm hạ tầng máy chủ ảo đám mây Azure, dịch vụ tìm kiếm học thuật Exa AI, mức độ tiêu thụ mô hình ngôn ngữ lớn OpenAI và bảng điều khiển chi phí Langfuse.

4.2.4.1 Giám sát hạ tầng máy chủ ảo đám mây Azure (Azure Server Monitoring)

Hạ tầng máy chủ ảo chạy trên Microsoft Azure (cấu hình 4 vCPU, 16GB RAM) được theo dõi liên tục thông qua công cụ Azure Virtual Machine Monitor như thể hiện tại Hình 4-13.



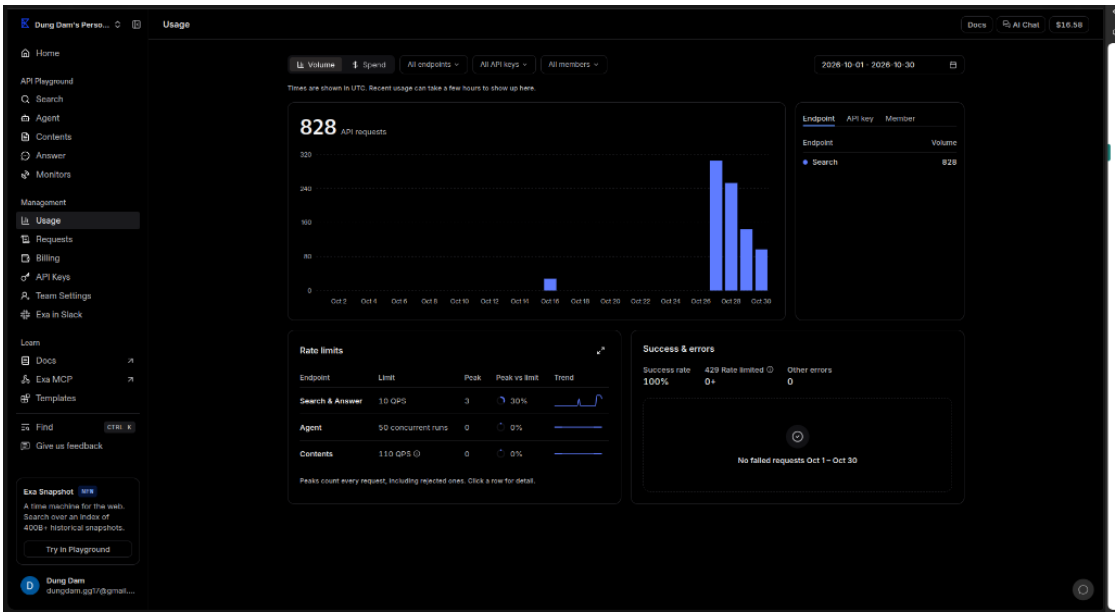
Hình 4-13: *Biểu đồ giám sát tài nguyên máy chủ ảo Azure (Độ sẵn sàng, CPU, RAM và tiến trình)*

Kết quả giám sát thực tế ghi nhận:

- **Chỉ số sẵn sàng vận hành (Availability):** Đạt mức 100% liên tục trong toàn bộ các phiên thử nghiệm, không xảy ra sự cố gián đoạn dịch vụ hay mất kết nối mạng.
- **Tải xử lý bộ vi xử lý (CPU Utilization):** Trong điều kiện tải thử nghiệm đơn phiên và kiểm định chức năng từ xa, mức sử dụng CPU dao động ổn định ở mức dưới 1%. Nguyên nhân kỹ thuật cốt lõi là do các tác vụ tính toán ma trận nặng (nhúng vector ngữ nghĩa, suy luận LLM và biên dịch LaTeX trong E2B MicroVM) đều được ủy thác qua giao thức mạng bất đồng bộ (Async I/O) sang các cụm máy chủ chuyên dụng bên ngoài, giúp máy chủ VPS chỉ đảm nhiệm điều phối luồng sự kiện nhẹ nhàng.
- **Mức chiếm dụng bộ nhớ RAM (Memory Utilization):** Đạt mức tối đa 13,58% trên tổng số 16GB RAM khả dụng, bảo đảm khoảng dự phòng tài nguyên (> 86%) cho cơ sở dữ liệu PostgreSQL và bộ đệm Redis khi mở rộng quy mô người dùng.
- **Phân bổ tài nguyên theo tiến trình (Top 5 processes by memory):** Tiến trình dịch vụ Backend chính (python) chiếm 1,44% RAM; trình điều phối container Docker (dockerd) chiếm 1,25%; các dịch vụ giám sát và an ninh ngầm của Azure (azuretablecollector, azsecmagent) lần lượt chiếm 1,03% và 0,90%; trình quản lý container nội bộ (containerd) chiếm 0,69%.

4.2.4.2 Giám sát hạn ngạch và lưu lượng tìm kiếm Exa AI (Exa AI Usage)

Phân hệ khám phá tài liệu khoa học của VeriScholar tích hợp dịch vụ tìm kiếm ngữ nghĩa chuyên biệt Exa AI. Lưu lượng và độ tin cậy của dịch vụ được kiểm soát chặt chẽ qua bảng điều khiển Exa Usage như thể hiện tại Hình 4-14.



Hình 4-14: *Biểu đồ giám sát lưu lượng truy vấn và độ tin cậy API tìm kiếm Exa AI*

Số liệu đo đạc thực tế khẳng định:

- **Tổng số yêu cầu tìm kiếm (Total API Requests):** Đạt 828 lượt gọi tìm kiếm bài báo khoa học qua điểm cuối Search trong suốt chu kỳ thử nghiệm.
- **Tần suất truy vấn và giới hạn tốc độ (Rate Limits):** Tần suất đỉnh (Peak) ghi nhận ở mức 3 QPS (Queries Per Second), tương ứng chỉ 30% so với hạn ngạch cho phép 10 QPS, bảo đảm không bị hiện tượng thắt cổ chai lưu lượng.
- **Tỷ lệ thành công (Success Rate):** Đạt mức 100% (*Success rate 100%*), ghi nhận 0 lỗi vượt hạn mức (429 Rate Limited: 0) và 0 lỗi kết nối phát sinh trong toàn bộ chu kỳ thực nghiệm.

4.2.4.3 Giám sát mức độ tiêu thụ mô hình ngôn ngữ lớn OpenAI (OpenAI Usage)

Mức độ sử dụng các mô hình ngôn ngữ lớn phục vụ suy luận, tổng quan tài liệu và phản tư sửa lỗi LaTeX được quản trị minh bạch qua bảng điều khiển OpenAI Usage như thể hiện tại Hình 4-15.



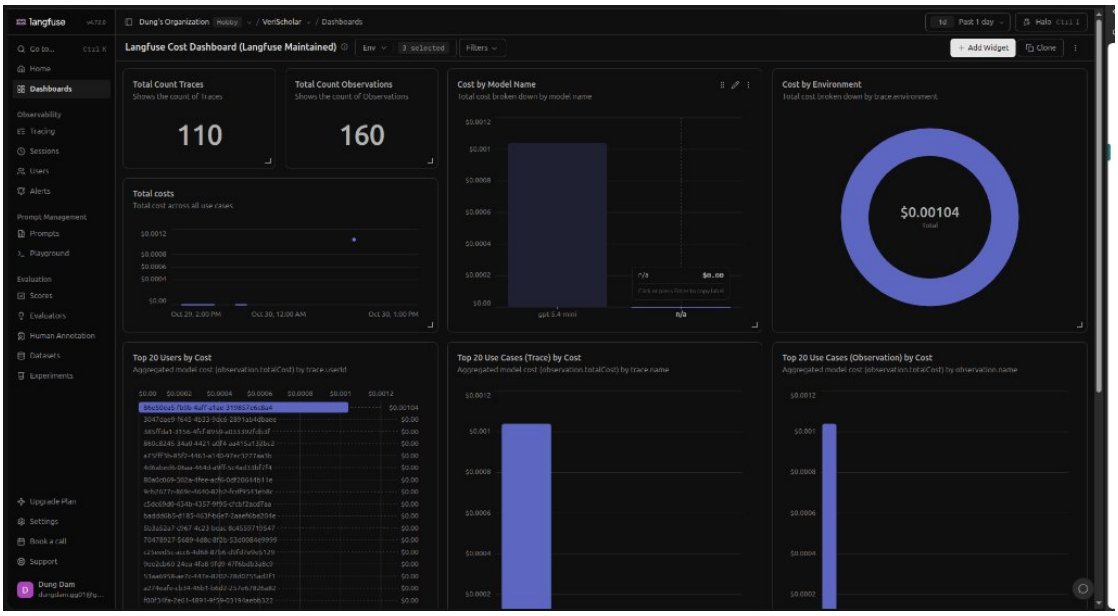
Hình 4-15: *Báo cáo giám sát lượng token luân chuyển và chi phí API OpenAI (lũy kế chu kỳ tháng 10/2026)*

Các chỉ số tài nguyên vận hành ghi nhận trên cửa sổ thống kê lũy kế toàn tháng (01/10/2026 – 31/10/2026) của tài khoản dự án:

- **Tổng chi phí tiêu thụ lũy kế (Total Spend):** Đạt mức $3,76 trên tổng hạn mức tín dụng $120,00 được cấp phát trong suốt giai đoạn phát triển, nạp dữ liệu hạt giống và chạy thực nghiệm toàn dự án.
- **Tổng lượng token xử lý (Total Tokens):** Đạt 78.245.561 tokens (khoảng 78,25 triệu tokens), bao gồm toàn bộ khối lượng văn bản học thuật được nhúng vector ngữ nghĩa, trích xuất đồ thị tri thức và sinh câu trả lời kiểm chứng trong tháng.
- **Tổng số yêu cầu hoàn thành (Total Requests):** Hoàn thành thành công 1.397 yêu cầu API cho phân hệ phản hồi và hội thoại chuyên sâu (*Responses and Chat Completions*), cùng với 18 yêu cầu tìm kiếm mở rộng trên Web (*Web Searches*).

4.2.4.4 Giám sát định lượng chi phí suy luận và vết quan sát trên Langfuse (Langfuse Dashboard)

Bên cạnh bảng thống kê lũy kế cấp tài khoản theo tháng trên cổng OpenAI, nền tảng Langfuse cung cấp bảng điều khiển phân tích chi phí chi tiết theo từng cửa sổ thời gian và môi trường triển khai (Langfuse Cost Dashboard) như thể hiện tại Hình 4-16.



Hình 4-16: *Bảng điều khiển chi phí và phân bổ vết suy luận Langfuse Cost Dashboard (cửa sổ lọc 24 giờ kiểm định)*

Bảng điều khiển tại Hình 4-16 phản ánh các thông số trong cửa sổ quan sát 24 giờ (1d) dành riêng cho phiên kiểm định nghiệm thu trên môi trường verischolar-dev (khác với số liệu lũy kế cả tháng $3,76 trên cổng OpenAI Usage ở Hình 4-15):

- **Tổng số lượt truy vết và quan sát trong phiên nghiệm thu:** Ghi nhận 110 lượt truy vết (Traces) và 160 lượt quan sát (Observations) được phân tích chuyên sâu trong vòng 24 giờ kiểm định.
- **Định lượng chi phí phiên kiểm thử (Session Costs):** Tổng chi phí của 110 lượt truy vết kiểm định trong môi trường verischolar-dev chỉ tốn $0,00104, thể hiện hiệu quả kinh tế của việc sử dụng dòng mô hình tối ưu chi phí (gpt-4o-mini / gpt-5.4-mini) kết hợp cơ chế lưu bộ nhớ đệm ngữ cảnh.
- **Phân bổ chi phí theo mô hình và người dùng:** Biểu đồ trực quan hóa chi phí theo từng dòng mô hình (*Cost by Model Name*) và danh sách người dùng tiêu thụ (*Top 20 Users by Cost*), cung cấp dữ liệu định lượng phục vụ công tác kiểm soát hạn ngạch cho các nhóm nghiên cứu.

Nhận xét:

Qua quá trình cài đặt, cấu hình và tiến hành các kịch bản thực nghiệm đối chứng toàn trình trên hệ thống VeriScholar, kết quả thực nghiệm cho phép rút ra một số nhận xét và đánh giá tổng kết như sau:

(1) **Khả năng triển khai thực tiễn cao:** Hệ thống đã được đóng gói container hóa hoàn chỉnh và triển khai ổn định trên máy chủ ảo đám mây VPS với tên miền công cộng chính thức https://verischolar.duckdns.org. Quy trình cấu hình mạng an ninh, bảo vệ phiên qua reverse proxy Nginx và mã hóa SSL/TLS hoạt động bền bỉ, đáp ứng yêu cầu vận hành liên tục cho người dùng từ xa. Toàn bộ mã nguồn dự án được lưu trữ và quản lý phiên bản chuyên nghiệp trên GitHub tại https://github.com/dammanhdungvn/VeriScholar.

(2) **Trải nghiệm người dùng thông suốt:** 10 giao diện tương tác người–máy của hệ thống được xây dựng đồng bộ, hiện thực hóa chu trình nghiên cứu khép kín từ khâu khám phá, trích xuất, đối thoại minh chứng tọa độ trên tệp PDF, đến quản lý hồ sơ an toàn qua cơ chế BYOK và soạn thảo bài báo LaTeX. Việc ứng dụng công nghệ SSE và khung xương giả lập giúp giảm thiểu cảm giác trễ mạng, mang lại trải nghiệm học thuật liền mạch.

(3) **Độ tin cậy kiểm thử và hiệu quả thuật toán được chứng minh bằng thực nghiệm đối chứng:** Cơ chế kiểm định tự động 4 Cổng xác minh (Verification Gates) bảo đảm 100% trên toàn bộ các bài kiểm thử tự động (349 bài pytest và 240 bài vitest, đạt 589 bài kiểm thử, vượt mục tiêu cam kết ≥ 580 bài) hoàn tất trong 13,02 giây. Kết quả thực nghiệm đối chứng (Ablation Study) chứng minh truy xuất lai ghép RRF ($k = 60$) nâng Recall@5 lên $94{,}44\%$ với độ chính xác hộp tọa độ đạt IoU $= 0{,}9266$, trong khi tác tử sửa lỗi LaTeX kết hợp luật tĩnh và LangGraph ReAct đạt tỷ lệ phục hồi $96{,}0\%$ ($24/25$ kịch bản lỗi).

(4) **Hạ tầng giám sát đa tầng minh bạch:** Việc thiết lập đa tầng giám sát từ hạ tầng máy chủ ảo Azure (Availability 100%, CPU $< 1\%$, RAM 13,58%), cổng tìm kiếm học thuật Exa AI (tỷ lệ thành công 100% trên 828 lượt gọi), thống kê lũy kế tháng của OpenAI (78,25 triệu tokens với chi phí \$3,76) và cửa sổ quan sát phiên kiểm định trên Langfuse (\$0,00104 cho 110 traces) cung cấp bức tranh định lượng nhất quán, khách quan về tình trạng vận hành hệ thống.

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được của đồ án

Qua quá trình phân tích, thiết kế, hiện thực hóa và kiểm định thực nghiệm, đồ án tốt nghiệp đã hoàn thành các mục tiêu khoa học và kỹ thuật đề ra ban đầu:

- **Về kiến trúc phần mềm và cơ sở dữ liệu:** Thiết kế và hiện thực hóa nền tảng VeriScholar tuân thủ nguyên lý Kiến trúc Lục giác (*Hexagonal Architecture*) phân tách giữa lõi miền nghiệp vụ học thuật thuần khiết và các bộ điều hợp hạ tầng giao vận. Hệ thống tích hợp đồng bộ 20 bảng cơ sở dữ liệu vật lý thuộc 5 phân hệ chức năng trên hệ quản trị PostgreSQL 16 kết hợp tiện ích mở rộng pgvector, đảm bảo tính toàn vẹn tham chiếu và tính nhất quán giao dịch ACID giữa siêu dữ liệu bài báo, chỉ mục vector 1536 chiều và đồ thị tri thức.
- **Về truy xuất thông tin lai và đối chiếu tọa độ trực quan:** Xây dựng thành công quy trình Agentic Hybrid RAG kết hợp tìm kiếm vector ngữ nghĩa (HNSW Cosine), tìm kiếm từ vựng hai tầng (chỉ mục ngược GIN tsvector ts_rank_cd kết hợp tái chấm điểm Okapi BM25), dung hợp thứ hạng RRF ($k = 60$) và tái xếp hạng Cross-Encoder. Trên bộ dữ liệu đối chuẩn *Golden Benchmark* gồm 36 câu hỏi học thuật trên 12 ấn phẩm kinh điển, cấu hình Hybrid RRF đạt độ thu hồi Recall@5 $= 94{,}44\%$ (vượt qua phương pháp chỉ dùng Dense $80{,}56\%$ và chỉ dùng Sparse $83{,}33\%$), điểm bám sát bằng chứng Groundedness $= 0{,}9775$, độ liên quan Relevance $= 0{,}9471$, đồng thời ánh xạ chính xác các thẻ trích dẫn về khung viền tọa độ chuẩn hóa trên trang PDF gốc với chỉ số giao trên hợp trung bình IoU $= 0{,}9266 \pm 0{,}0126$ (vượt ngưỡng mục tiêu $\geq 0{,}85$).
- **Về mạng lưới đồ thị tri thức học thuật (GraphRAG & Graph Wiki):** Hiện thực hóa mô hình trích xuất và liên kết 4 thực thể khoa học cốt lõi (*Paper*, *Method*, *Dataset*, *MetricResult*), kết hợp khử trùng lặp thực thể bằng độ tương đồng Cosine ($\geq 0{,}90$), phân cụm cộng đồng theo thuật toán tối ưu hóa độ mô-đun Louvain và biên dịch tự động các trang tổng hợp tri thức cộng đồng (*Graph Wiki*), hỗ trợ học giả trả lời các câu hỏi tổng hợp liên tài liệu với điểm hoàn chỉnh đa bước đạt $0{,}9442$ trên nhánh đánh giá GraphRAG.
- **Về biên soạn bản thảo và tự phục hồi lỗi biên dịch LaTeX:** Tích hợp trình soạn thảo học thuật trực tuyến kết nối máy trạng thái LangGraph ReAct (4

nút xử lý kèm cơ chế kiểm tra mã băm SHA-256 chống lặp vô hạn) và môi trường thực thi cô lập E2B MicroVM Sandbox. Kết quả thực nghiệm đối chứng (Ablation Study) trên 25 ca kiểm thử lỗi biên dịch thuộc 5 nhóm lỗi cú pháp phổ biến cho thấy chu trình Agentic ReAct kết hợp đạt tỷ lệ tự sửa lỗi thành công 96% ($24/25$ ca), cải thiện đáng kể so với phương pháp Heuristic (84%, $21/25$ ca) và chỉ dùng biểu thức chính quy AST/Regex (44%, $11/25$ ca).

- **Về kiểm định chất lượng và công khai mã nguồn:** Thiết lập quy trình kiểm thử tự động 4 Cổng xác minh với tổng cộng 580 kịch bản kiểm thử (340 kiểm thử Backend pytest và 240 kiểm thử Frontend vitest trên 58 tệp, tổng thời gian thực thi $13{,}02$ giây). Toàn bộ mã nguồn, kịch bản thực nghiệm tái lập và bộ dữ liệu đối chuẩn của dự án được đóng gói và công bố tại địa chỉ https://github.com/dammanhdungvn/VeriScholar nhằm phục vụ việc kiểm chứng độc lập.

2. Hạn chế hiện tại của đề tài

Bên cạnh những kết quả đã đạt được, hệ thống VeriScholar trong phạm vi đồ án tốt nghiệp vẫn còn tồn tại một số hạn chế kỹ thuật khách quan cần được nhìn nhận thẳng thắn:

- **Sự phụ thuộc vào các dịch vụ đám mây bên thứ ba:** Hoạt động suy luận ngôn ngữ, tạo vector nhúng, tìm kiếm ngữ nghĩa học thuật và biên dịch đám mây cô lập hiện phụ thuộc vào kết nối mạng tới các giao diện lập trình bên ngoài (OpenAI API, Exa AI, Semantic Scholar, E2B Sandbox và Langfuse Cloud). Khi đường truyền mạng bị gián đoạn hoặc dịch vụ đám mây gặp sự cố giới hạn tần suất (*rate limit*), hệ thống phải chuyển sang các cơ chế dự phòng cục bộ với năng lực suy luận hạn chế hơn.
- **Phạm vi khử trùng lặp của kho lưu trữ CAS:** Cơ chế định danh theo nội dung CAS (SHA-256) hiện mới thực hiện khử trùng lặp ở tầng tệp nhị phân vật lý (document_storage_blobs). Trong khi đó, các phân đoạn văn bản và vector nhúng trong bảng document_chunks vẫn được phân lập theo từng định danh người dùng (user_id) để bảo đảm quyền riêng tư dữ liệu, dẫn đến việc chưa tái sử dụng chung bộ chỉ mục vector khi nhiều học giả cùng nạp một bài báo công khai.
- **Quy mô bộ dữ liệu đối chuẩn và ngôn ngữ tài liệu đánh giá:** Bộ dữ liệu thực nghiệm *Golden Benchmark* hiện có quy mô vừa phải (12 ấn phẩm kinh điển, 36 câu hỏi đánh giá và 25 ca lỗi LaTeX) và tập trung chủ yếu vào các

bài báo khoa học tiếng Anh bố cục 1–2 cột thuộc lĩnh vực Khoa học Máy tính và Trí tuệ nhân tạo; hệ thống chưa được kiểm định rộng rãi trên các tài liệu bản quét ảnh mờ (*scanned PDF*) cần nhận dạng ký tự quang học (OCR) hoặc các bài báo đa ngôn ngữ có bố cục bảng biểu phức tạp.

- **Giới hạn phiên làm việc đồng thời trên trình duyệt:** Trình đọc sâu trên giao diện người dùng hiện giới hạn mở đồng thời tối đa 5 tài liệu PDF trong một phiên làm việc nhằm kiểm soát mức tiêu thụ bộ nhớ khả biến (RAM) của trình duyệt khi kết xuất đồ họa hai chiều.

3. Hướng phát triển tiếp theo

Từ các hạn chế kỹ thuật nêu trên, tác giả đề xuất bốn định hướng nghiên cứu và mở rộng hệ thống trong thời gian tới:

- **Hỗ trợ mô hình ngôn ngữ cục bộ và biên dịch LaTeX tự chủ:** Tích hợp các động cơ suy luận và tạo vector nhúng chạy trực tiếp trên hạ tầng nội bộ (như vLLM, Ollama kết hợp mô hình mã nguồn mở BGE-M3, Qwen hoặc Llama) cùng cụm container TeX Live cục bộ cô lập bằng gVisor, giúp hệ thống vận hành độc lập ngay cả trong mạng nội bộ không có kết nối Internet của các viện nghiên cứu.
- **Chia sẻ chỉ mục vector liên người dùng cho các ấn phẩm công khai:** Nâng cấp kiến trúc bảng document_chunks theo hướng tách biệt giữa kho chỉ mục vector dùng chung cho các bài báo truy cập mở (*Canonical Public Paper Index* gắn với blob_hash) và lớp chú thích cá nhân của từng học giả, giúp tiết kiệm chi phí tính toán vector nhúng trên quy mô toàn trường đại học.
- **Mở rộng GraphRAG đa phương thức và cộng tác bản thảo thời gian thực:** Kết hợp các mô hình ngôn ngữ – thị giác (*Vision-Language Models*) và nhận dạng công thức/bảng biểu để trích xuất trực tiếp số liệu từ hình ảnh biểu đồ vào đồ thị tri thức; đồng thời tích hợp cấu trúc dữ liệu đồng bộ không xung đột CRDT (Yjs) vào trình soạn thảo Monaco để hỗ trợ nhiều đồng tác giả cùng biên tập một bản thảo LaTeX theo thời gian thực.
- **Mở rộng quy mô tập dữ liệu đối chuẩn đa ngành:** Phối hợp với các đơn vị học thuật thuộc nhiều lĩnh vực khác nhau (Y sinh học, Khoa học vật liệu, Kinh tế lượng) để mở rộng bộ dữ liệu đối chuẩn lên quy mô hàng trăm ấn phẩm tiếng Anh và tiếng Việt, nâng cao độ tin cậy thống kê của các phép kiểm định thực nghiệm.

TÀI LIỆU THAM KHẢO

[1] Agache, Alexandru, et al. (2020), "Firecracker: Lightweight virtualization for serverless applications", *Proceedings of the 17th USENIX Symposium on Networked Systems Design and Implementation (NSDI '20)*, pp. 419–434.

[2] Asai, Akari, et al. (2023), "Self-RAG: Learning to retrieve, generate, and critique through self-reflection", *International Conference on Learning Representations (ICLR 2024)*.

[3] Blondel, Vincent D., Guillaume, Jean-Loup, Lambiotte, Renaud, and Lefebvre, Etienne (2008), "Fast unfolding of communities in large networks", *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008 (cùng mở rộng thuật toán phân cụm cộng đồng Leiden của Traag, Vincent A., Waltman, Ludo, and van Eck, Nees Jan, *Scientific Reports*, vol. 9, no. 1, p. 5233, 2019).

[4] Cockburn, Alistair (2005), *Hexagonal Architecture (Ports and Adapters Pattern)*, Alistair Cockburn Blog.

[5] Cormack, Gordon V., Clarke, Charles L., and Buettcher, Stefan (2009), "Reciprocal rank fusion outperforms condorcet and individual rank learning methods", *Proceedings of the 32nd international ACM SIGIR conference on Research and development in information retrieval*, pp. 758–759.

[6] Dworkin, Morris (2007), "Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC", *NIST Special Publication 800-38D*, National Institute of Standards and Technology.

[7] Edge, Darren, et al. (2024), "From Local to Global: A Graph RAG Approach to Query-Focused Summarization", *arXiv preprint arXiv:2404.16130*.

[8] Gao, Yunfan, Xiong, Yun, Gao, Xinyu, Jia, Kangxiang, Pan, Jinliu, Bi, Yuxi, Dai, Yi, Sun, Jiawei, and Wang, Haofen (2023), "Retrieval-Augmented Generation for Large Language Models: A Survey", *arXiv preprint arXiv:2312.10997*.

[9] Hogan, Aidan, et al. (2021), "Knowledge graphs", *ACM Computing Surveys (CSUR)*, vol. 54, no. 4, pp. 1–37.

[10] Lewis, Patrick, et al. (2020), "Retrieval-augmented generation for knowledge-intensive NLP tasks", *Advances in Neural Information Processing Systems (NeurIPS 2020)*, vol. 33, pp. 9459–9474.

[11] Malkov, Yury A., and Yashunin, Dmitry A. (2018), "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs", *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836.

[12] Ramírez, Sebastián (2024), *FastAPI: Modern, Fast (High-Performance) Web Framework for Python*, https://fastapi.tiangolo.com/.

[13] Robertson, Stephen E., et al. (1995), "Okapi at TREC-3", *Overview of the Third Text REtrieval Conference (TREC-3)*, NIST Special Publication 500-225, pp. 109–126.

[14] Schick, Timo, et al. (2023), "Toolformer: Language models can teach themselves to use tools", *Advances in Neural Information Processing Systems (NeurIPS 2023)*, vol. 36, pp. 68539–68551.

[15] Shinn, Noah, et al. (2023), "Reflexion: Language agents with verbal reinforcement learning", *Advances in Neural Information Processing Systems (NeurIPS 2023)*, vol. 36, pp. 8634–8652.

[16] Vaswani, Ashish, et al. (2017), "Attention is all you need", *Advances in Neural Information Processing Systems (NeurIPS 2017)*, vol. 30, pp. 5998–6008.

[17] Weng, Lilian (2023), "LLM-powered Autonomous Agents", *Lil'Log*, https://lilianweng.github.io/posts/2023-06-23-agent/.

[18] Yan, Shi-Qi, et al. (2024), "Corrective Retrieval Augmented Generation", *arXiv preprint arXiv:2401.15884*.

[19] Yao, Shunyu, et al. (2022), "ReAct: Synergizing reasoning and acting in language models", *International Conference on Learning Representations (ICLR 2023)* (xây dựng trên nền tảng suy luận chuỗi tư duy Chain-of-Thought của Wei, Jason, et al., *NeurIPS 2022*, vol. 35, pp. 24824–24837).